



Threat Containment Strategies for Compromised xApps in Open RAN

An undergraduate project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfillment of the requirements for the

**Degree of the Bachelor of the Science of Engineering
Honours**

by

M.K.J. Dewmith - RU/E/2021/4474
M.D.K. Dilshan - RU/E/2021/4485
L.L.D. Sewinda - RU/E/2021/4807
T.K. Thilohith - RU/E/2021/4832

.....
Dr. Chatura Seneviratne
(Supervisor)

.....
Prof. Dr. An Braeken
(Co-Supervisor)

.....
Mr. Pramitha Fernando
(Co-Supervisor)

Abstract

The Open Radio Access Network (O-RAN) disaggregates traditional base station functions into cloud-native software components operating on commodity hardware, enabling multi-vendor programmability at every layer of the network. Central to this architecture is the Near-Real-Time RAN Intelligent Controller (Near-RT RIC), which hosts containerized applications known as xApps that perform closed-loop control actions within a 10 ms to 1 s window. While this programmability unlocks operational flexibility, it introduces a critical security risk: a compromised or malicious xApp executes with privileged access to live network telemetry and E2 interface control commands, creating a direct path to service disruption, lateral movement and data exfiltration within the RIC platform. This project designs, implements and experimentally validates ZT-XGuard, a zero-trust security framework that addresses a scoped version of this threat through four coordinated layers. First, hardware-rooted platform attestation establishes a cryptographically verified trust anchor for the monitored hosting nodes before protected workloads are deployed. Second, a zero-trust secure onboarding pipeline followed by workload identity attestation ensures that only xApps which are cryptographically verified, receive a short-lived X.509 identity. Third, a dual-channel runtime monitor pairs kernel-level behavioral detection through Falco with a multivariate statistical resource-anomaly detector that evaluates per-xApp container telemetry, so that both hard exploitation events and sustained resource-pressure anomalies that evade simple threshold-based monitoring are covered. Fourth, on confirmed compromise an automated containment orchestrator withdraws the xApp's runtime identity binding so its short-lived SVID is not renewed, applies a NetworkPolicy backed by a node-level packet-filter block, isolates its service endpoints and confirms the xApp is unreachable. The framework was evaluated across two environments: a three-VM Solution Provider-SMO-RIC testbed for secure onboarding and workload-identity issuance, and a separate NIST O-RAN Automation Tool testbed comprising srsRAN, Open5GS and a Kubernetes-based O-RAN Software Community Near-RT RIC for runtime monitoring, and containment.

Acknowledgment

We would like to express our sincere gratitude to our supervisor, Dr. Chatura Seneviratne, from the Department of Electrical and Information Engineering, University of Ruhuna, for his continuous guidance, encouragement, and valuable insights throughout this project. His expertise provided a strong foundation for our research on improving the security of xApps in O-RAN environments. We also extend our heartfelt thanks to our co-supervisors, Prof. An Braeken and Mr. Pramitha Fernando, from the Department of Industrial Engineering, Vrije Universiteit Brussel (VUB), Belgium, for their valuable advice, technical support, and constructive feedback. Their knowledge and guidance on security mechanisms, and Open RAN architecture greatly contributed to the quality and direction of this work. We are deeply grateful to the Department of Electrical and Information Engineering, University of Ruhuna, for providing the laboratory facilities and computational resources necessary for the implementation and experimental validation of our proposed security architecture. These facilities enabled us to conduct testbed evaluations, performance analysis, and simulations under realistic conditions. Finally, we would like to thank all the faculty members, colleagues, and friends who supported and encouraged us throughout this journey. Their collaboration, insightful discussions, and moral support were invaluable to the successful completion of this research.

M.K.J. Dewmith - RU/E/2021/4474
M.D.K. Dilshan - RU/E/2021/4485
L.L.D. Sewinda - RU/E/2021/4807
T.K. Thilohith - RU/E/2021/4832

Contents

Abstract	ii
Acknowledgment	iii
Contents	vii
List of Figures	x
List of Tables	xii
Acronyms	xiii
1 Introduction	1
1.1 Background	2
1.1.1 Open RAN Architecture	2
1.1.2 Near-RT RIC Architecture and Security of xApps	4
1.1.3 Threat Containment Mechanisms for xApps	5
1.2 Problem Statement	5
1.3 Threat Model	6
1.3.1 System and Trust Assumptions	6
1.3.2 Adversary Model and Capabilities	6
1.3.3 In-Scope and Out-of-Scope Threats	6
1.3.4 Threat-to-Defense Mapping	7
1.3.5 Evasion	7
1.4 Objectives and Scope	8
1.4.1 Objectives	8
1.4.2 Scope	8
1.5 Novelty Statement	9
2 Literature Review	10
2.1 Previous Work	10
2.1.1 The Vulnerability of Disaggregated Architectures	10
2.1.2 Existing Security Mechanisms	14
2.2 Gaps in Literature	16
3 Methodology	18
3.1 Overview of the Methodology	18
3.2 Research and Requirement Analysis	19
3.3 Tools and Technologies	20
3.4 Hardware-Rooted Platform Attestation	22
3.5 Secure Onboarding and Workload Identity Attestation	23

3.5.1	Secure Onboarding Architecture	23
3.5.2	Cryptographic Packaging at the Solution Provider	24
3.5.3	SMO Secure Ingestion Pipeline	25
3.5.4	RIC Deployment and SPIRE Workload Attestation	26
3.6	Runtime Detection Architecture	27
3.7	Continuous System Observability	28
3.7.1	Behavioral Monitoring	28
3.7.2	Resource Monitoring	29
3.8	Behavioral Threat Detection with Falco	29
3.8.1	Zero-Trust Rule Design Philosophy	29
3.8.2	Representative Falco Rule Implementation	30
3.8.3	The Detection Rule Catalog	31
3.8.4	What the Rules Treat as Legitimate	32
3.8.5	Handling False Positives	32
3.8.6	Decision-Synchronous Evaluation and Its Limits	33
3.9	Statistical Resource-Anomaly Detector	34
3.9.1	Design Rationale and Development Process	35
3.9.2	Metric Selection and Feature Engineering	37
3.9.3	Robust Centering and Scale Estimation	38
3.9.4	Directional Excess Transformation	39
3.9.5	Sliding Window Mean Smoothing	39
3.9.6	MEWMA Tracking	40
3.9.7	Ledoit-Wolf Shrinkage Covariance Estimation and Hotelling's T^2 Fusion	41
3.9.8	Calibration of Control Limits	48
3.9.9	Dual-Gate Decision Logic	49
3.10	Three-State Policy Engine and Escalation Logic	50
3.10.1	Deterministic Transition Guards	51
3.11	Automated Multi-Lock Containment and Verified Isolation	52
3.12	Experimental Setup	55
3.12.1	Secure Onboarding Testbed	55
3.12.2	Runtime Monitoring and Containment Testbed	56
4	Results	57
4.1	Deployment of srsRAN and 5G Core	57
4.1.1	Deployed Components	57
4.2	Near-RT RIC Deployment	58
4.2.1	Kubernetes-Based Near-RT RIC	58
4.3	Hardware-Rooted Platform Attestation	58
4.3.1	Component Deployment and Registration	59
4.3.2	Trusted Provisioning and Integrity Verification	60
4.3.3	Integrity Violation Detection	62
4.3.4	System Admission Overhead	62
4.4	Secure Onboarding and Workload Identity Attestation	63
4.4.1	Infrastructure Bootstrapping	63
4.4.2	Initial Selector-Based Onboarding Validation	64
4.4.3	Man-in-the-Middle Simulation	68
4.4.4	Onboarding Latency and Performance Overhead	68

4.4.5	Security Implementations and Threat Mitigation	69
4.5	Behavioral Threat Detection	71
4.5.1	Detection and Containment Effectiveness	72
4.5.2	Detection and Containment Latency	72
4.5.3	Per-Mechanism Containment Latency	73
4.5.4	Behavioral Monitoring Overhead	74
4.6	Statistical Resource-Anomaly Detection	74
4.6.1	Evaluation Dataset Overview	75
4.6.2	Clean Baseline Specificity	76
4.6.3	Resource Exhaustion Adversary Scenarios	77
4.6.4	Burst-Duration Boundary Characterization	81
4.6.5	Ablation: Contribution of Each Detector Stage	83
4.6.6	Deployed Detection and Automated Containment	84
4.7	Automated Containment and Isolation	86
4.7.1	Verified Removal from the RIC Control Loop	87
4.7.2	Resilience of the Isolation to the Compromised xApp	87
4.7.3	Monitoring and Orchestration Overhead	88
4.7.4	Experimental Reset Between Trials	88
4.8	Scalability Evaluation	88
4.8.1	Evaluation Scope and Industry Context	88
4.8.2	Protected-Workload Detector Scalability	89
4.8.3	Policy Decision-Plane Throughput and Saturation	91
4.8.4	Concurrent Enforcement on Deployed xApps	93
4.8.5	Scalability Summary and Validity Boundaries	94
4.9	Runtime Overhead of the Policy Engine	95
5	Discussion	97
5.1	Deployment of srsRAN and the 5G Core	97
5.2	Near-RT RIC Deployment	97
5.3	Impact and Analysis of Remote Attestation	98
5.4	Secure Onboarding and Workload Attestation	98
5.5	Runtime Behavioral Monitoring	99
5.6	Statistical Resource-Anomaly Detection	100
5.7	Automated Multi-Lock Containment	101
5.8	Challenges and Solutions	102
5.8.1	Getting Real KPM Telemetry across the E2 Interface	102
5.8.2	Separating Attacks from Legitimate RIC Control Traffic	103
5.9	O-RAN WG11 Threat Coverage	103
5.10	Limitations	104
5.11	Summary of Discussion	106
6	Timeline and Resources Required	107
6.1	Timeline	107
6.2	Resources Required	109
7	Conclusion and Future Work	110
7.1	Conclusion	110

7.2 Future Work	111
References	112

List of Figures

1.1	Open RAN architecture [1]	2
1.2	Near-RT RIC architecture [2]	4
3.1	ZT-XGuard full system architecture with policy engine and containment orchestrator	19
3.2	Remote attestation using Keylime open source software tool	22
3.3	Secure xApp onboarding and workload-attestation sequence.	23
3.4	Workflow at the solution provider during secure onboarding	24
3.5	Workflow at the SMO during verification and secure chart preparation	25
3.6	Workflow at the Near-RT RIC during xApp deployment and workload attestation	26
3.7	ZT-XGuard runtime detection architecture.	27
3.8	Distribution of the M1 CPU-utilization metric under benign baseline operation.	36
3.9	Raw standardized z-score vs rectified score contribution.	37
3.10	CPU rate-gate floor derived from the maximum of the pooled clean CPU samples ($n = 5400$), interpreted as a one-sided Wilks tolerance bound with at least 95% confidence and 99.9% population coverage under the calibration distribution, giving 56.638 mCPU.	49
3.11	ZT-XGuard containment architecture.	53
4.1	Kubernetes-based Near-RT RIC deployment: running pods in the <code>ricplt</code> namespace	58
4.2	Keylime agent startup and registration log on the srsRAN node, confirming successful AIK binding to the registrar	59
4.3	Keylime registrar log confirming agent registration and secure storage of the hardware identity database entry	59
4.4	Keylime verifier continuous monitoring loop, issuing nonce-based TPM quote challenges and logging periodic node integrity checks	60
4.5	Successful integrity handshake.	60
4.6	Raw PCR measurements captured from the agent node's vTPM.	61
4.7	PCR-quote verification success confirmation.	61
4.8	PCR manipulation on the agent node, extending PCR 23 with an unauthorized value.	62
4.9	Transitioning to FAIL status immediately following the PCR 23 manipulation.	62
4.10	System admission overhead comparing the native baseline against continuous attestation polling intervals.	63

4.11	cert-manager pods running in the <code>cert-manager</code> namespace, confirming that the upstream CA for the SPIRE trust domain is operational	64
4.12	SPIRE server, agents and CSI driver pods running in the <code>spire-system</code> namespace	64
4.13	SPIRE node attestation	64
4.14	Secure transfer of the signed xApp descriptor tarball and its GPG detached signature from solution provider to SMO	65
4.15	Successful execution of the initial selector-based SMO secure ingestion pipeline	66
4.16	Initial onboarding validation: successful xApp deployment and selector-based SPIRE workload attestation on the Near-RT RIC	67
4.17	Man-in-the-middle simulation result	68
4.18	Comparison of xApp onboarding latency across 200 trials for baseline vs. zero-trust pipeline.	69
4.19	Behavioral channel per attack scenario: detection decision latency (left axis) and attack-onset-to-four-lock-application latency (right axis) across the 400 trials.	73
4.20	Per-mechanism containment and network-unreachability latency (mean with 95% confidence interval) across the 400 behavioral trials. . . .	74
4.21	MEWMA/ T^2 pressure score on held-out clean datasets D1 (top) and D2 (bottom) across 1745 scored samples each	77
4.22	A1 development dataset (sustained CPU exhaustion at 20% utilization), used with A3 to select the CPU rate-gate window and rate.	78
4.23	A2 held-out evaluation (combined CPU + memory exhaustion) . . .	80
4.24	A3 development and boundary-characterization dataset (4-step stealth intensity staircase)	81
4.25	A4 raw pulse-train evaluation	82
4.26	Deployed statistical channel per scenario: attack-onset-to-detection (left axis) and post-detection containment latency (right axis). The fixed 30 s dwell that precedes isolation is constant across scenarios and is omitted for clarity.	85
4.27	Reversible CPU throttle after escalation to suspicious from the resource channel.	86
4.28	xApp E2 connectivity carrying RIC indication and reporting.	87
4.29	Detector per-cycle scoring latency (p50, p95, p99 and worst case) against the number of independent detector contexts.	91
4.30	Policy decision-plane end-to-end deadline-miss rate against offered evaluation rate.	92
4.31	Supporting evidence for the decision-plane saturation knee.	93
4.32	Per-stage containment latency against the number K of deployed xApp workloads subjected to simultaneous controlled attack injection. .	94

4.33	ZT-XGuard policy-engine overhead on the Near-RT RIC during a 15-minute steady-state measurement at 1 Hz. The upper plot compares CPU usage for the vanilla RIC and the ZT-XGuard-enabled RIC, while the lower plot compares resident-memory usage. The CPU overhead remains small, whereas the policy engine introduces an approximately 98 MB fixed resident-memory increase.	96
------	--	----

List of Tables

2.1	Comparison on the implemented scope with closely related O-RAN security work	16
3.1	ZT-XGuard behavioral detection rules, their severity and the trust state each drives	31
3.2	The four-stage resource-anomaly scoring pipeline	34
3.3	Complete Telemetry Metric Extraction and Model Role Mapping	38
3.4	Noise-suppression ratio and total pipeline latency (s) across candidate window lengths and MEWMA half-lives	40
3.5	Notation used in the Ledoit-Wolf shrinkage covariance derivation	42
3.6	Directional-excess calculation for the first sample of the worked example	46
3.7	Eight-sample excess window used to calculate $\mathbf{x}_1$	46
3.8	Formal trust-state transition table of the ZT-XGuard policy engine	51
3.9	Three-VM testbed configuration for secure onboarding evaluation	55
3.10	NIST O-RAN Automation Tool testbed configuration for runtime monitoring and containment evaluation	56
4.1	Attack scenarios and points of failure in the zero-trust onboarding pipeline	70
4.2	Evaluated attacks mapped to MITRE ATT&CK techniques	71
4.3	Behavioral-channel detection and containment across the eight attack scenarios (50 trials each, mean)	72
4.4	Per-mechanism containment latency from attack onset (mean across the 400 behavioral trials)	73
4.5	Evaluation dataset inventory: roles, sizes and relationship to model fitting, calibration and evaluation phases	75
4.6	False positive rate on clean and benign-elevated-load evaluation datasets (n = total scored samples after warmup exclusion)	76
4.7	Confusion matrix for the three sustained scenarios (A1 and A3 development; A2 held out)	78
4.8	A4 per-pulse outcomes around the frozen dual-gate boundary	82
4.9	Deployed statistical-channel detection and post-dwell containment per scenario.	85
4.10	Detector computational scalability.	90
4.11	Mean enforcement-milestone completion time (seconds, attack-onset origin) by concurrency level, over the four deployed xApps. 100 formal trials per K	94
5.1	Coverage of O-RAN WG11 xApp threat categories by ZT-XGuard	104

6.1	Project Timeline	108
-----	----------------------------	-----

Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
BBU	Baseband Unit
CIA	Confidentiality, Integrity and Availability
CNF	Cloud-native Network Function
COTS	Commercial Off-The-Shelf
CPRI	Common Public Radio Interface
DTLS	Datagram Transport Layer Security
E2AP	E2 Application Protocol
E2SM	E2 Service Model
E2T	E2 Terminator
High-PHY	Higher Physical Layer
IPsec	Internet Protocol Security
KPI	Key Performance Indicator
Low-PHY	Lower Physical Layer
MAC	Medium Access Control
MIMO	Multiple Input Multiple Output
MitM	Man-in-the-Middle
ML	Machine Learning
MOI	Managed Object Instance
Near-RT RIC	Near-Real-Time RAN Intelligent Controller
Non-RT RIC	Non-Real-Time RAN Intelligent Controller
O-CU	O-RAN Control Unit
O-DU	O-RAN Distributed Unit
O-RAN	Open Radio Access Network

O-RU	O-RAN Radio Unit
OAM	Operations, Administration and Maintenance
OSC	O-RAN Software Community
PCR	Platform Configuration Register
PDCP	Packet Data Convergence Protocol
R-NIB	RAN Network Information Base
RA	Remote Attestation
RAN	Radio Access Network
RIC	RAN Intelligent Controller
RLC	Radio Link Control
RMR	RIC Message Router
RRC	Radio Resource Control
RU	Radio Unit
SCTP	Stream Control Transmission Protocol
SDAP	Service Data Adaptation Protocol
SDL	Shared Data Layer
SMO	Service Management and Orchestration
TCG	Trusted Computing Group
TEE	Trusted Execution Environment
TPM	Trusted Platform Module
UE	User Equipment
UE-NIB	User Equipment Network Information Base
VNF	Virtual Network Function

Chapter 1

Introduction

The evolution of Radio Access Network (RAN) toward the Open Radio Access Network (O-RAN) establishes a paradigm shift in cellular network architecture. O-RAN enables the transition from proprietary, vendor-locked, monolithic systems to disaggregated, virtualized and software-defined infrastructures [3]. Open RAN utilizes standardized and open interfaces combined with Commercial Off-The-Shelf (COTS) hardware to support flexibility [4]. By incorporating multi-vendor interoperability, programmability and cloud-native deployment, Open RAN enhances reconfigurability of the RAN and coordination among network nodes, reducing costs and fostering innovation. One of the most important aspects of O-RAN is RAN Intelligent Controllers (RICs) and their applications—xApps and rApps. These applications introduce programmable, data-driven control over radio resources [3]. However, the openness and programmability of the O-RAN also expand the attack surface, introducing critical security vulnerabilities, especially in the Near-Real-Time RAN Intelligent Controller (Near-RT RIC) where xApps execute [4]. Since xApps are third-party applications that are running on virtualized and containerized environments, they are prone to security threats such as supply chain attacks, code tampering and unauthorized modifications. A compromised xApp can lead to severe network disruptions, including service degradation, data breaches and unauthorized control over radio functions and resources [4]. Within the related-work corpus analyzed in Chapter 2, we found no prior system that reports the same combination of supply-chain verification, cryptographic workload identity, per-xApp behavioral and resource-anomaly detection and probe-verified multi-lock containment for the Near-RT RIC. This project implements and evaluates ZT-XGuard, a zero-trust security framework specifically designed for xApps in the Near-RT RIC, integrating cryptographic onboarding verification, SPIFFE/SPIRE workload attestation, runtime behavioral and resource monitoring and automated containment. Critical behavioral compromise initiates immediate isolation, whereas statistically confirmed compromise provides a 30-second operator-intervention window before automatic isolation.

The Near-RT RIC is deployed at the network edge and it interacts directly with O-CUs and O-DUs via the E2 interface [1]. It hosts xApps that subscribe to streams of network data (Key Performance Measurements-KPMs) and then compute and send control actions back to the RAN. This enables closed-loop automation for use cases like dynamic handover optimization, traffic steering and radio resource management. The Non-Real-Time RAN Intelligent Controller (Non-RT RIC) operates on a timescale greater than 1 second. This controller is part of the Service Management and Orchestration (SMO) framework. It focuses on higher-layer policy deployment, AI/ML model training and management and service life-cycle orchestration. The Non-RT RIC provides guidance, policies and enrichment information to Near-RT RIC via the A1 interface [1].

An important aspect of the O-RAN architecture is the definition of standardized, open interfaces that allow disaggregated components to communicate seamlessly across a multi-vendor landscape. For instance, in traditional RAN, the S-interface or X2-interface often contained proprietary extensions that prevented hardware from Vendor A, from communicating with software from Vendor B. O-RAN reduces this proprietary coupling by specifying interoperable interfaces and protocols at defined architectural reference points [3] [1]. Among these interfaces, the E2 interface plays a critical role in enabling near-real-time optimization of radio network behavior. The E2 interface connects the Near-RT RIC with RAN nodes, specifically the O-CU and O-DU. This interface supports the exchange of fine-grained telemetry data and the issuance of control commands, facilitating advanced functions such as dynamic load balancing, interference coordination and mobility optimization through standardized E2 Service Models (E2SMs). Although the E2 interface enhances network programmability, it also introduces critical security vulnerabilities. WG11 defines interface-security requirements and protocol controls for O-RAN; the listed interface protocols are mandatory for vendors to support, while their use by an operator can remain deployment and regulation-dependent [7]. Implementation studies nevertheless report access-control and signaling-validation gaps in O-RAN SC E2 components, creating concrete paths for unauthorized signaling and denial of service [8].

The A1 interface provides the logical communication connection between the Near-RT RIC and the Non-RT RIC. It is mostly used to communicate high-level, intent-based policies, also known as A1 policies that direct near-real-time control loop behavior. These policies might have goals like giving particular network slices priority, imposing power limits or modifying quality-of-service (QoS) standards. These high-level instructions are converted into specific control actions by the Near-RT RIC, which are then sent to RAN nodes through the E2 interface. The authenticity and integrity of communications over the A1 interface are crucial to prevent misconfiguration or policy injection attacks, since A1 policies have the ability to affect network-wide behavior [1]. The O1 interface facilitates the Management and Orchestration (OAM) functions, including configuration, fault monitoring and software life-cycle management. In a multi-vendor environment, the O1 interface ensures that the SMO framework can monitor the health of an O-DU from one vendor just as easily as an O-RU from another [9] [1].

The transition from proprietary interfaces to standardized, open protocols changes

the RAN security model. Although the openness enables a “plug-and-play” ecosystem in which operators can deploy the best components, it also dissolves the traditional trust boundary enforced by single-vendor systems. As a result, security can no longer rely on implicit trust based on physical location or vendor. Every message exchanged across an open interface must be explicitly authenticated, authorized and protected to ensure integrity and confidentiality. This shift requires a security-by-design approach in which the network operates under the assumption that any connected component may be compromised unless its identity and behavior are continuously verified through cryptographic and policy-based mechanisms [3] [10]. The O-Cloud is the collective virtualization infrastructure for computing, storage and networking and the software platform that supports O-RAN functions. O-CU and O-DU are designed as virtualized network functions (VNFs) or cloud-native network functions (CNFs) that can be deployed on COTS servers, often at the network edge. This cloud-native approach enables elasticity, scalable resource sharing and automated lifecycle management [1].

1.1.2 Near-RT RIC Architecture and Security of xApps

The Near-Real-Time RAN Intelligent Controller (Near-RT RIC) is an important software platform in the O-RAN architecture. It operates within strict latency limits of 10 milliseconds to 1 second. It creates the control loop between network intelligence and the physical RAN elements (O-CU and O-DU). Its primary function is to host and execute lightweight, containerized applications known as xApps [2]. xApps analyze live network telemetry and issue control commands. This transforms the RAN from a statically configured infrastructure into a dynamically programmable and adaptive system. As a result, it enables AI-driven traffic steering, dynamic spectrum sharing and proactive handover management [3]. However, this architecture gives third-party xApps considerable power and access, creating a critical need for robust threat containment mechanisms.

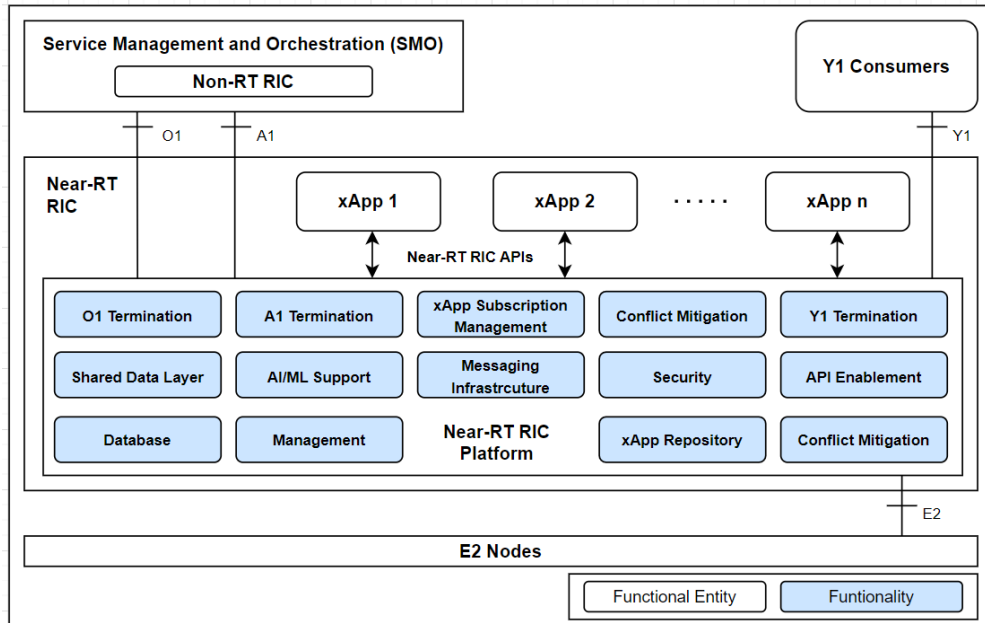


Figure 1.2: Near-RT RIC architecture [2]

The Near-RT RIC is a microservice-based platform that provides the necessary runtime environment and services for xApps. The internal architecture of the Near-RT RIC consists of several key subsystems that facilitate communication, management and control. There are several core internal components in the Near-RT RIC: RIC Message Router (RMR) is the internal messaging bus for communication between xApps, services and interface endpoints. E2 Terminator (E2T) terminates the E2 interface connections from RAN nodes (O-CU/O-DU). Subscription Manager handles xApp requests to subscribe to specific data streams from RAN functions via E2. Shared Data Layer (SDL) is a shared database (e.g., Redis-based) storing RAN/UE context in RAN Network Information Base (R-NIB) and User Equipment Network Information Base (UE-NIB). Conflict mitigation resolves conflicts when multiple xApps issue conflicting control commands for the same RAN resource. xApp Management provides life-cycle APIs for onboarding, deploying and terminating xApps [3].

An xApp is a containerized microservice deployed as a Docker container within the RIC platform that implements specific control logic. It interacts with the RAN through a well-defined workflow enabled by the E2 interface and its Service Models (E2SMs). The security of the Near-RT RIC is inherently complex due to its design aspects. xApps operate within the control plane of the network with direct access to sensitive operational data (UE identities, cell load, performance counters) and the ability to alter RAN behavior. Third-party xApps introduce supply chain risks where malicious code or vulnerable libraries can be introduced. The shared platform resources like SDL, RMR and the potential for xApp conflicts create a large attack surface. A malfunctioning or hostile xApp can destabilize the RIC platform itself or manipulate data of other xApps in the SDL [8].

1.1.3 Threat Containment Mechanisms for xApps

ZT-XGuard responds to confirmed compromise through a four-lock containment sequence comprising workload-identity binding withdrawal, network quarantine, Service-endpoint isolation and an independent node-level packet-filter block. An active TCP-connect probe verifies that the target is unreachable before containment is recorded as complete. Critical behavioral compromise initiates this response immediately, whereas statistically confirmed compromise uses the operator-intervention dwell described in Section 3.10. The implementation of each isolation lock is detailed in Section 3.11.

1.2 Problem Statement

The O-RAN design allows third-party xApps to execute control actions within the Near-Real-Time RIC. WG11 specifies security requirements and controls for Near-RT RIC APIs, xApp authentication and authorization, image validation, interface protection, monitoring and application lifecycle management [7, 11]. The problem addressed by this project is therefore not an absence of O-RAN security specifications. It is an implementation and integration gap: the reviewed systems do not connect supply-chain verification and workload identity to per-xApp runtime compromise assessment and verified containment as one coordinated architecture.

Once deployed, a compromised xApp may misuse shared resources, interfere with other xApps, move laterally within the RIC environment, expose sensitive data or degrade RAN performance. Resource exhaustion is particularly difficult to handle safely because legitimate xApps can also produce bursty CPU, memory and network behavior. Within the related-work corpus examined in this report, no prior system reports the same combination of per-xApp multivariate resource-pressure detection, independent kernel-level behavioral detection and verified containment. ZT-XGuard is designed to fill that experimentally testable gap while implementing, the relevant standards-level controls.

1.3 Threat Model

1.3.1 System and Trust Assumptions

We consider a Kubernetes-hosted Near-RT RIC in which xApps run as containers beside the RIC platform services (E2 termination, RMR messaging, the shared data layer and subscription management). ZT-XGuard trusts the Kubernetes control plane, the SPIRE identity infrastructure and its own monitoring and policy-engine components, which together form the trusted computing base. An attacker who controls the Kubernetes API server or the SPIRE root can subvert any policy layered above them, so their integrity is a precondition rather than something the framework defends. The trusted components may still deliver noisy or delayed telemetry and the design tolerates that.

1.3.2 Adversary Model and Capabilities

The adversary compromises a single xApp, through a subverted container image or software supply chain, exploitation of a vulnerable running xApp or abuse of the permissions the xApp was legitimately granted. We assume the attacker gains code execution at the xApp’s own privilege level, which is root inside the container but without host access and inherits the xApp’s RIC identity and network reachability. From there the attacker can exhaust CPU, memory or network bandwidth; open unauthorized outbound connections; execute unexpected processes or an in-container shell; read sensitive files, ServiceAccount tokens or SPIFFE key material; tamper with the xApp’s configuration; or attempt to escape the container.

1.3.3 In-Scope and Out-of-Scope Threats

In scope are runtime resource abuse and system-level exploitation launched from a compromised xApp, because these produce evidence at the layers ZT-XGuard monitors: container resource telemetry and kernel system calls. The following are out of scope, each for a stated reason:

- Attacks expressed only through the xApp’s legitimate RAN-control semantics, such as malicious but well-formed control logic, E2 data poisoning and stealthy control-plane manipulation. These need application-level models of correct control behavior rather than the system-level evidence we monitor.

- Container escape that has already reached host-level control. The behavioral channel detects the escape *attempt* at the system-call level and contains the xApp before host compromise completes; an attacker already holding the host belongs to the trusted base.
- Control-plane-wide denial of service against the Kubernetes API itself.
- Physical-layer and radio attacks.

1.3.4 Threat-to-Defense Mapping

In-scope threats are handled through independent behavioral and statistical decision routes that operate independently without a mutual corroboration requirement. A critical behavioral signal produces **COMPROMISED** and immediate isolation. A statistical signal first produces **SUSPICIOUS** when partial evidence is observed and produces **COMPROMISED** only when its internal dual-gate confirmation rule is satisfied. Statistical compromise then enters an operator-intervention dwell before automatic isolation. The two detection channels are detailed in Sections 3.8 and 3.9, the three-state decision logic in Section 3.10 and the four-lock containment mechanism in Section 3.11.

Critical behavioral violations, including in-container shell execution, credential or SVID access, sensitive-file access, unauthorized outbound connections, configuration tampering, container-escape attempts and malicious tool execution, raise the affected xApp directly to **COMPROMISED** and trigger isolation. Lower-confidence behavioral events, such as suspicious tooling, package management, binary drops, permission changes or unexpected Kubernetes API access, place the xApp in **SUSPICIOUS**, where additional scrutiny and the reversible CPU throttle are applied. Resource-exhaustion attacks, including sustained CPU/memory abuse and duty-cycled or stealthy workloads, are evaluated by the statistical detector: partial evidence produces **SUSPICIOUS** and CPU throttling, whereas satisfaction of the complete dual gate raises the xApp to **COMPROMISED**; following a fixed 30-second operator-intervention dwell, isolation is initiated automatically unless the operator isolates the xApp earlier. This intervention dwell delays the containment action, not the **COMPROMISED** decision itself.

1.3.5 Evasion

A resource-abuse adversary may attempt to remain below the statistical confirmation gates or mimic benign bursts. The evaluation therefore examines both low-and-slow and duty-cycled resource abuse. The selected CPU rate gate detects the tested below-floor-mean burst pattern because it measures the fraction of recent samples above the calibrated CPU floor rather than relying only on mean load. The pulse-duration experiment further brackets an observed transition between the tested 5 s and 10 s pulses, but this does not establish a universal minimum detectable duration or intensity. An adaptive attacker that keeps either the CPU-rate condition or the T^2 persistence condition below its confirmation threshold may still evade the statistical channel.

The behavioral channel has a complementary limitation: an exploitation path

whose runtime actions do not match any deployed Falco rule may escape behavioral detection. The independent statistical channel can provide additional coverage when such an attack also produces anomalous CPU, memory or network-resource behavior; however, an attack that is both behaviorally uncovered and resource-stealthy remains outside the demonstrated detection scope.

1.4 Objectives and Scope

1.4.1 Objectives

The primary objective of this project is to implement a security framework that enhances the resilience of Near-RT RIC against threats from compromised xApps.

This objective is achieved through the following specific goals:

1. Implement a workload identity attestation mechanism for xApps.
2. Develop an automated isolation pipeline that is triggered by security violations of xApps.
3. Experimentally validate the proposed framework within a testbed.

1.4.2 Scope

This project aims to build a security framework to protect xApps in Open RAN. The main focus is on the Near-RT RIC and our work is specifically bounded to securing the xApps running inside it.

This project covers the following areas.

1. Runtime remote attestation system to continuously monitor and verify the integrity of the xApp execution environment.
2. Implementation of workload identity attestation to enable continuous verification of xApp authenticity.
3. Development of a dual channel threat detection mechanism comprising behavioral and resource-abuse adversaries.
4. Automated containment and verified isolation of confirmed xApp compromise.

Project scope does not include:

1. The security of other O-RAN components like the O-CU, O-DU, O-RU, SMO or the Non-RT RIC is outside the scope of this project.
2. While our framework is designed for the Near-RT RIC, deploying it in a large-scale, commercial multi-vendor, production network is beyond the research and testing of this project.

1.5 Novelty Statement

This project designs a coordinated zero-trust framework architecture that links supply-chain verification, workload identity, runtime compromise assessment and automated threat containment for xApps in the Near-RT RIC.

The novelty claim is deliberately scoped to the related-work corpus analyzed in Chapter 2 and to the reported combination, not to the individual algorithms or tools. MEWMA, Hotelling’s T^2 , Ledoit-Wolf shrinkage, Falco, SPIFFE/SPIRE and Kubernetes policy enforcement are established mechanisms. ZT-XGuard’s first contribution is their xApp-specific statistical construction: a per-xApp resource exhaustion detector that combines robust baseline fitting, directional excess, Multivariate Exponentially Weighted Moving Average tracking, Ledoit-Wolf covariance estimation, Hotelling’s T^2 fusion and an explicit persistence/rate confirmation rule. Its second contribution is deterministic decision-level fusion of independent behavioral and multivariate resource-detection routes, each capable of establishing **COMPROMISED** under its own confirmation guard. Its third contribution is a measured, probe-verified multi-lock containment path that links the compromise decision to independently verified workload isolation.

The reviewed systems include xApp authorization, E2 signaling inspection and blocking, UE-oriented intrusion detection and secure slicing, fine-grained xApp access control, encrypted xApp inference and zero-trust segmentation. None reports this same per-xApp behavioral-plus-resource decision architecture coupled to the four-lock, probe-verified containment path. The experimental claim remains bounded: supply-chain verification and Pod-UID-bound identity issuance were evaluated on the three-domain onboarding testbed, whereas runtime monitoring, a separately instantiated revocable identity binding and containment were evaluated on the single-node O-RAN testbed. The experiments validate complementary halves rather than one continuous end-to-end deployment. The evaluated workflows operated without external network connectivity, demonstrating compatibility with deployments that impose an offline operating requirement.

Chapter 2

Literature Review

This chapter analyzes the existing knowledge regarding O-RAN security. It follows a structured approach, moving from the architectural vulnerabilities inherent in disaggregation to specific threat vectors within the Near-RT RIC and xApps. Next it discusses, existing security frameworks that have been addressed in previous work and finally evaluate the limitations of current defense mechanisms.

2.1 Previous Work

This section presents a comprehensive review of existing academic literature, technical standards and open-source frameworks relevant to the security of the O-RAN. A special focus is placed on the Near-RT RIC, its xApp ecosystem and the security mechanisms proposed for threat containment in cloud-native deployments.

2.1.1 The Vulnerability of Disaggregated Architectures

The transition from legacy networks to O-RAN represents a shift from “security by obscurity” to “security by design”. It introduces severe structural vulnerabilities in the O-RAN. In traditional monolithic RAN deployments (e.g., proprietary “black boxes”), the Baseband Unit (BBU) and Radio Unit (RU) were trusted implicitly because they were hardened by a single vendor. The physical perimeter of the cell site was the trust boundary in the traditional RAN.

The O-RAN architecture removes this implicit trust. As noted in Polese et al. [3], disaggregating the base station into the O-CU, O-DU and O-RU creates a multi-vendor environment. A scenario now exists where the O-DU from Vendor A must trust control messages from an xApp from Vendor B running on a RIC platform from Vendor C. The “Dissolution of Trust” means that components can no longer assume that a message originating from inside the network is legitimate. The attack surface has expanded from the network edge to the internal interfaces themselves [9].

The E2 Interface as an Attack Vector

The E2 interface is the critical link connecting the Near-RT RIC to the RAN nodes. It carries the E2 Application Protocol (E2AP) messages used to optimize network parameters. Although, the O-RAN security specifications mandate the use of Internet Protocol Security (IPsec) or Datagram Transport Layer Security (DTLS) to encrypt this traffic, early implementations often lack consistent enforcement.

Technical analyses show that the E2 interface is susceptible to Man-in-the-Middle (MitM) attacks if mutual authentication is not strictly enforced. An attacker who gains access to the transport network can inject malicious E2 control messages. An attack on the E2 interface can destabilize the control plane causing cell outages or handover failures across the entire cluster. Furthermore, the reliance on the Stream Control Transmission Protocol (SCTP) for E2 introduces specific vulnerabilities, since SCTP is susceptible to flooding attacks that can overwhelm the E2 Termination (E2T) service [8].

The xApp Threat Landscape

The introduction of the xApp ecosystem, creates a complex new attack surface. The O-RAN Alliance’s Security Working Group (WG11) provides the definitive threat analysis for the architecture. Their modeling identifies “Malicious or Compromised xApps” as a paramount concern, detailing specific attack vectors that directly threaten the Near-RT RIC’s integrity, availability and confidentiality. These threats can be systematically categorized into three primary groups based on the vulnerability origin and attack method [12].

- **Exploitation of Untrusted xApps:** The fundamental risk of third-party software creates a major threat category. WG11 identifies scenarios where an attacker exploits an xApp that originates from an untrusted or unmaintained source. It can be a compromised or malicious software supply chain. A single compromised xApp can be used to achieve multiple adversarial goals, as detailed in the related threat family [12]. Primary security impacts of exploitation of untrusted xApps include: Tampering (A malicious xApp can alter data transmitted over the A1 or E2 interfaces, disrupting RAN functions and violating data authenticity), Information Disclosure (Sensitive network configuration, user data or other confidential information can be extracted), Denial of Service (DoS) and Elevation of Privilege (An attacker can gain unauthorized control over other xApps or the entire Near-RT RIC, enabling actions beyond intended permissions). This threat model shows that the software supply chain for xApps is a critical vulnerability because a single exploitable component can compromise the core security tenets of CIA.
- **Breach of Container and Runtime Isolation xApps:** The second category addresses failures in the runtime security. xApps are typically deployed as containers within an O-Cloud environment, relying on isolation mechanisms for security. WG11 threats state that an attacker “can compromise xApp isolation and break out of xApp containment” by exploiting vulnerabilities in the host system [12]. A successful breach of containment has severe effects such as: Tampering with the Host (An attacker can tamper with the

underlying system, potentially manipulating other xApps or O-RAN components), Lateral Movement & Data Theft (An attacker can extract sensitive information from co-hosted xApps or the Near-RT RIC software, by breaching isolation), Service Degradation (Gained access can be used to degrade services, leading to a DoS condition) and Privilege Escalation (Compromising the host provides a platform to exploit other vulnerabilities potentially leading to full system control). These threats highlight that the security of the xApp runtime environment is as critical as the xApp code. A failure in isolation can lead to lateral movement and system wide compromise.

- **Abuse of the Control Loop via Malicious Policies:** The third category involves attacks that weaponize the control interfaces of the O-RAN. These threats describe how “false or malicious A1 policies from the Non-RT RIC” can be issued to modify the behavior of xApps on the Near-RT RIC [12]. The security impacts of control loop abusing include: Network Tampering (False policies can alter RAN performance. For instance by improperly steering user data to facilitate other attacks), Induced Denial of Service (Policies can be crafted to trigger a DoS condition, targeting specific UE, groups or entire cells) and Privacy Invasion (Using false policies to locate a subscriber. An attacker can track the location of a user, by isolating a subscriber’s traffic or manipulating beamforming data. This represents a major breach of confidentiality).

Malicious and Compromised xApps

The architectural openness of O-RAN introduces a critical xApp threat surface. The O-RAN Alliance WG11 study treats malicious or compromised xApps as a principal Near-RT RIC security issue because xApps occupy a privileged position with access to sensitive network data and control interfaces [13]. The exact numbering of key issues can change across specification versions, so this report relies on the threat category rather than a version-specific issue number. A malicious xApp is defined as one that possesses valid credentials to operate on the RIC platform but deliberately abuses its granted privileges to perform harmful actions [13]. These applications may be intentionally created as malicious from their start or may become compromised through vulnerabilities introduced in the software supply chain or via runtime exploits [1]. The threats can be categorized by their origin and method.

Attack Vectors and Methods

The abuse of xApp privileges can target different layers of the RIC and RAN, leading to a number of failures.

- **Resource Exhaustion and Denial of Service (DoS):** A primary attack involves resource starvation, where a malicious xApp intentionally consumes extreme amounts of shared CPU or memory to degrade or cripple the performance of neighboring, legitimate xApps [1] [13]. More aggressively, an xApp can target the RIC infrastructure itself. Research has found that an xApp can flood the E2 Termination (E2T) service with malformed or high-volume

of signaling messages, potentially causing the critical E2 interface to collapse and damaging the connection of RIC to the RAN nodes [1] [9].

- **Privilege Escalation and API Abuse:** The internal APIs of the Near-RT RIC, particularly those managing the E2 interface, lack robust, standardized access controls. Experimental research has revealed that attackers could exploit this gap to make unauthorized calls to privileged internal APIs [8] [9]. For instance, a documented attack involved calling an unprotected shutdown API over unencrypted HTTP, which forcibly disconnected all O-CU and O-DU nodes from the RIC, effectively paralyzing the network slice [9].
- **Policy Manipulation and Network Instability:** Beyond direct resource attacks, a sophisticated malicious xApp can exploit its role in the control loop. By sending radio resource control commands that deliberately contradict the policies of higher-priority xApps, it can create instability in the RAN [13]. For example, an xApp could override an emergency service Quality of Service (QoS) policy, leading to service degradation or failure for critical communications.
- **Lateral Movement and Data Exfiltration:** Once an xApp is compromised, it can serve as a foothold for lateral movement within the RIC environment. In deployments with inadequate container or network isolation, a malicious xApp can probe the internal RIC network to identify and attack other components [8]. A prime target is the Shared Data Layer (SDL), a common database (often Redis) used by xApps to store state. Breaching the SDL could allow theft of highly sensitive user equipment identifiers, such as International Mobile Subscriber Identity (IMSI) or Temporary Mobile Subscriber Identity (TMSI), enabling subscriber tracking or impersonation attacks [9].

The threat from malicious xApps is intensified by authentication and authorization weaknesses in Near-RT RIC APIs and internal communications identified by WG11 [13]. Weak or incompletely implemented fine-grained access control can allow a vulnerable benign xApp to be manipulated into actions beyond its intended scope, blurring the operational distinction between a compromised application and an intentionally malicious one. This report avoids attaching fixed key-issue numbers to those concerns because the numbering is version-specific.

Case Studies in Vulnerability: CVE Analysis

Public vulnerability records provide concrete evidence of availability failures in O-RAN SC E2 components, but the two records must be distinguished precisely:

- **CVE-2023-42358:** In the O-RAN SC `ric-plt-e2mgr` G-Release, a crafted request to the E2Manager API can cause denial of service. NVD associates the record with missing authorization and uncontrolled resource consumption; it does not describe malformed SCTP traffic or identify E2Term as the affected component [14].
- **CVE-2023-41628:** In the O-RAN SC E2 G-Release, incorrectly initiating the messaging procedure between E2Node and E2Term can cause denial of

service. The public record does not describe internal routing-table manipulation [15].

Together, these records demonstrate distinct denial-of-service paths in E2 management and termination workflows. They substantiate implementation-level availability risk; they do not by themselves establish privilege escalation, prove exploitability from every xApp deployment or justify claims about traffic redirection.

2.1.2 Existing Security Mechanisms

The security of the Near-RT RIC and its xApps has evolved in a security strategy, moving from static, perimeter-based models to dynamic, evidence-based frameworks aligned with zero-trust principles.

Static Isolation and Analysis

The first line of defense involves securing the deployment environment and the software supply chain itself, employing static controls established at configuration time.

- **Kubernetes Security Primitives:** Standard deployments utilize native orchestration features for initial isolation. These include namespaces for logical separation, network policies for micro-segmentation and resource quotas to prevent a single xApp from starving others of CPU or memory. However, as noted in security analyses, these are static configurations; they create a security baseline but cannot dynamically respond to novel runtime attacks that emerge after deployment [10].
- **Pre-Deployment Static Analysis:** To combat threats from the software supply chain, tools like Clair or Trivy are used to scan container images for known vulnerabilities before they are deployed on the RIC. While effective against a catalog of known threats, this method is fundamentally limited against zero-day exploits or runtime attacks such as memory buffer overflows, which are not present in the static image [10].

Runtime Monitoring and Enforcement

To address the limitations of static configurations, advanced runtime security focuses on detecting and preventing malicious behavior as it occurs.

- **System Call Restriction:** Security modules like Seccomp and AppArmor are used to define strict profiles that limit the system calls an xApp container can make. This proactively prevents many container breakout attempts by restricting actions at the kernel level.
- **Kernel-Level Behavioral Monitoring:** Modern research highlights eBPF (Extended Berkeley Packet Filter) as a powerful tool for low-overhead, deep-kernel monitoring [9] [10]. eBPF programs can be attached to kernel tracepoints to monitor events such as unexpected process spawning, suspicious file access or anomalous network connections in real-time. This enables the

detection of behavioral anomalies that indicate a compromise, providing the necessary signal for a response.

Zero Trust and Cryptographic Attestation

The most significant security aspect of the O-RAN is the shift towards a Zero Trust Architecture (ZTA), which operates on the principle of “never trust, always verify” [16] [17] [8]. This paradigm rejects implicit trust based on network location and requires continuous verification of identity and integrity.

- **Hardware-Rooted Trust with Remote Attestation:** In high-assurance scenarios, a TPM-rooted attestation protocol allows a verifier to authenticate a fresh quote over selected platform measurements and compare those measurements with an approved policy [10]. Measured-boot PCR evidence can establish assurance about the measured firmware, bootloader, kernel and enrolled files; it does not, by itself, prove that a running xApp container remains unaltered. Extending the trust decision to container code requires container/runtime measurements and an explicit policy link between platform attestation and workload-identity issuance. ZT-XGuard evaluates those mechanisms independently in separate testbeds and does not claim that Keylime gated SPIRE issuance.
- **Workload Identity with SPIFFE/SPIRE:** A core enabler of ZTA in microservices environments is the Secure Production Identity Framework for Everyone (SPIFFE) and its implementation SPIRE. SPIFFE provides a standardized way to issue cryptographically-verifiable identities to software workloads, like xApps, based on the properties of the software itself rather than mutable network attributes like IP addresses [8] [18]. SPIRE acts as the trust broker, performing attestation to validate the provenance of an xApp before issuing it a SPIFFE Verifiable Identity Document (SVID), which is then used for mutual TLS (mTLS) authentication with other services.

Statistical and Behavioral Runtime Anomaly Detection

Beyond policy-based enforcement, only few works have applied statistical and machine-learning techniques to detect runtime anomalies in cloud-native workloads. Multivariate control-chart methods, in particular the Hotelling’s T^2 statistic derived from EWMA [19] [20], are an established approach for detecting deviations in correlated multivariate telemetry and have been used for anomaly detection in cloud-native and server environments. These approaches exploit the correlated nature of system-level metrics where CPU, memory and network activity co-vary under normal operation to detect deviations that evade per-metric threshold monitoring. Within the O-RAN security corpus reviewed for this report, we found no prior evaluation of this specific multivariate process-control construction on per-container xApp resource telemetry, including an explicit persistence/rate gate and an empirically characterized pulse-duration boundary. Falco provides rule-based kernel-level behavioral detection for generic container environments but does not perform statistical characterization of resource usage patterns or provide the per-xApp telemetry fusion that sustained resource-exhaustion attacks require.

2.2 Gaps in Literature

Despite the advances above, the reviewed systems solve different portions of the problem: identity and authorization, E2 signaling validation, UE-oriented intrusion detection and slicing, xApp access control, data confidentiality or architectural segmentation. Table 2.1 compares their implemented scope directly. The gap claimed here is narrower and testable: within this corpus, no prior system combines per-xApp kernel-behavior and multivariate resource evidence with a trust-state decision and a four-lock containment action whose network result is verified by an active probe.

Table 2.1: Comparison on the implemented scope with closely related O-RAN security work

Approach	Primary control or evidence	Evaluated response	Gap relative to ZT-XGuard
XRF and hierarchical authorization [21, 22]	JWT-based xApp authentication, authorization and discovery	Token-mediated API access decisions	No runtime compromise detector or workload quarantine
E2 anomaly-traffic detector [23]	E2 state-machine validation and packet-field conformance checks	Blocks anomalous E2/RMR signaling and raises an alarm	Does not assess container resource behavior or perform multi-lock xApp containment
ZTRAN [24]	Service authentication, UE behavior profiling and secure slicing xApps	Denies or restricts malicious UEs through authentication and slice/resource control	Targets UE trust and RAN-resource access, not compromise of the hosting xApp
OZTrust [25]	Per-packet xApp tagging/verification and trace-derived access policies	Blocks unauthorized API access and lateral movement	Access-control enforcement without behavioral/resource compromise scoring or verified workload quarantine
ZT-RIC [26]	Inner-product functional encryption for xApp inference	Preserves RAN/UE data confidentiality while permitting encrypted inference	No runtime xApp compromise decision or containment path
Hybrid O-RAN zero-trust deployment [27]	Central PDP, distributed PEPs, enclaves and xApp/rApp sandboxing	Policy and segmentation enforcement in a Kubernetes/Istio proof of concept	Deployment architecture rather than per-xApp detection-to-probe-verified containment
Generic Falco runtime security [28]	Kernel-level behavioral rules	Emits runtime alerts	RIC-agnostic and does not provide integrated containment by itself
ZT-XGuard (this work)	Per-xApp behavioral and multivariate resource evidence	Three-state decision, four locks and active reachability probe	Evaluated on one protected xApp across separate onboarding and runtime testbeds

Absence of Verified Multi-Lock xApp Containment

Several prior systems do enforce security responses: Hung et al. block anomalous E2 signaling, ZTRAN restricts malicious UEs through secure slicing and OZTrust blocks unauthorized xApp communication [23–25]. The remaining gap is not an absence of response. It is the absence, in the reviewed corpus, of a compromised-xApp response that combines withdrawal of the workload-identity binding, CNI-level deny-all policy, Service-endpoint removal and an independent node-level packet-filter block, followed by an active reachability test.

Effectiveness of Forensic-Preserving Isolation

The reviewed work does not experimentally evaluate an isolation mode in which the compromised xApp is retained for evidence collection while its network and service reachability are removed and independently verified. ZT-XGuard evaluates that containment outcome without introducing a fourth trust state. The active probe verifies network unreachability; it does not prove immediate cryptographic invalidation of an already issued SVID, which remains valid until it expires.

Latency-Security Performance Trade-offs

Finally, the Near-RT RIC operates under strict timing requirements (less than 1 s control loops). There is a lack of empirical data on the performance overhead that continuous security checks (like attestation) impose on the system. It is currently unknown if strict zero trust enforcement causes the RIC to violate its latency limits, leading to RAN performance degradation. This project will provide the necessary experimental data to quantify this trade-off [9].

Limited Integrated Experimental Validation

The literature contains both conceptual proposals and experimentally evaluated component-level controls. A continuing gap is evaluation of the complete supply-chain-to-runtime lifecycle in one continuous deployment. ZT-XGuard advances the evidence base by evaluating onboarding and workload-identity issuance in one testbed and live runtime detection and verified containment in another, but it does not claim to close the continuous end-to-end deployment gap; integrating the two environments remains future work.

Absence of Evaluated Per-xApp Multivariate Resource Control

The reviewed frameworks do not report a per-xApp detector that jointly evaluates correlated container resource telemetry such as CPU, memory-growth and network-rate against a clean, workload-specific baseline using multivariate statistical process control. Static per-metric thresholds cannot represent cross-metric covariance and are vulnerable to benign bursts or distributed weak deviations. ZT-XGuard therefore evaluates a specific MEWMA/Hotelling T^2 construction with shrinkage covariance, directional excess and an explicit persistence/rate gate.

Chapter 3

Methodology

3.1 Overview of the Methodology

ZT-XGuard is a zero-trust security framework designed to address the detection and containment of a compromised xApp in the O-RAN Near-RT RIC, from the moment a package arrives from an untrusted third-party provider, through continuous runtime operation, to automated containment and forensic preservation when a compromise is detected. The framework is built around four coordinated security pillars that conceptually cover the xApp lifecycle. Their implementation was evaluated in the two complementary testbeds described below.

1. **Hardware-Rooted Platform Attestation:** Before any xApp is deployed, the infrastructure nodes hosting the Near-RT RIC are cryptographically attested using TPM-based remote attestation, establishing a verified trust anchor for the execution environment.
2. **Zero-Trust Secure Onboarding and Workload Identity Attestation:** Every xApp package is subjected to dual-signature cryptographic verification before it is admitted to the platform. Successfully onboarded xApps are issued short-lived, cryptographically verifiable workload identities that bind their runtime identity to their Kubernetes deployment attributes.
3. **Runtime Monitoring via Behavioral and Resource-Anomaly Detection:** Two independent observational channels operate continuously during xApp execution, kernel-level syscall monitoring through Falco, and a multivariate statistical resource-anomaly detector built on MEWMA tracking and Hotelling's T^2 scoring.
4. **Automated Multi-Lock Containment:** A critical behavioral decision initiates immediate automatic isolation. A statistically confirmed compromise opens a 30-second operator-intervention dwell, during which the operator may isolate early; otherwise, isolation begins automatically when the dwell expires. Both routes apply four independent locks and confirm unreachability with an active probe.

Figure 3.1 illustrates how these four pillars interact within the overall ZT-XGuard architecture, from the supply chain through to verified containment.

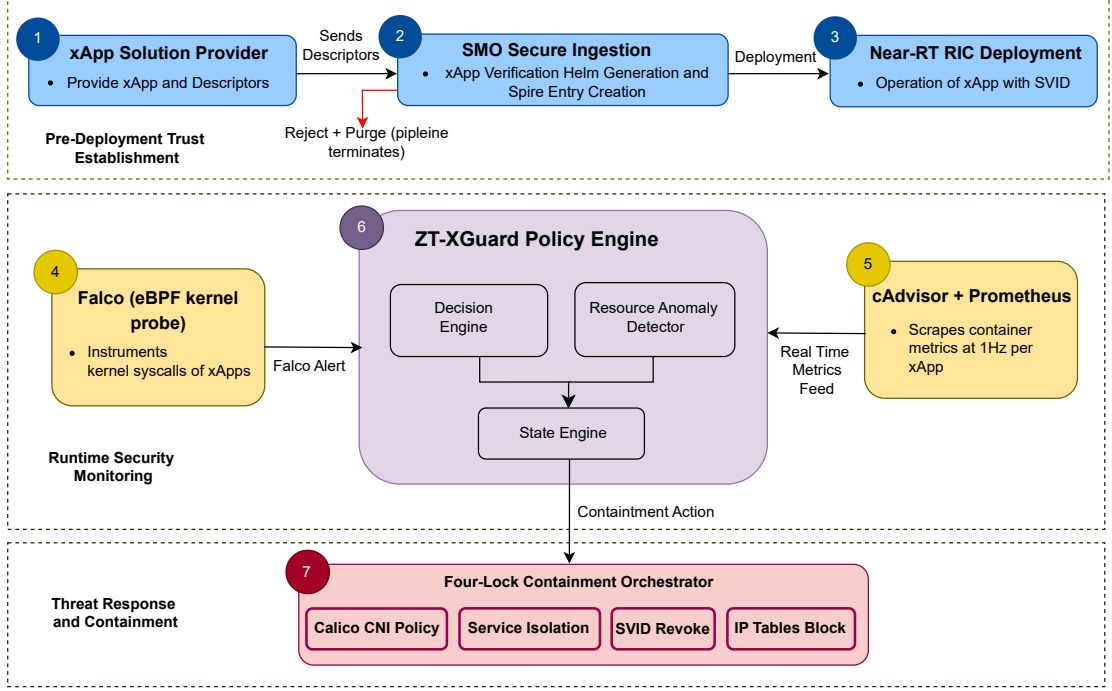


Figure 3.1: ZT-XGuard full system architecture with policy engine and containment orchestrator

The secure onboarding and workload attestation components are evaluated on a three-VM testbed that models the Solution Provider, SMO and Near-RT RIC as distinct administrative domains. It first establishes the selector-based, init-container identity flow used for the initial functional validation and then evaluates the final onboarding pipeline, which injects a `spiffe-helper` sidecar and creates a JIT (Just-in-Time) registration entry bound to the active Pod UID. The runtime monitoring and containment components are evaluated independently on the NIST O-RAN Automation Tool platform [29], which provides a fully integrated srsRAN [30], Open5GS [31] and O-RAN SC Near-RT RIC [32] [33] environment in a single VM. In this testbed, a declarative `ClusterSPIFFEID` selects the protected xApp through a label and supplies the revocable identity anchor used by containment. The two identity deployments are not presented as one certificate lifecycle: they independently validate identity issuance during onboarding and identity-binding withdrawal with renewal prevention during runtime response. This separation reflects a deliberate design principle: the onboarding pipeline is agnostic to the underlying RIC implementation, while the resource-anomaly detector and containment engine are tightly integrated with the specific telemetry and orchestration APIs of the Near-RT RIC deployment.

3.2 Research and Requirement Analysis

The functional requirements for ZT-XGuard were derived directly from the O-RAN Alliance WG11 threat model [12], which identifies malicious and compromised xApps as a principal threat class in the Near-RT RIC security landscape. The threat model describes three primary attack classes that a compromised xApp

can execute: exploitation of privileged E2 interface access to inject malicious control commands, resource exhaustion attacks that degrade the performance of co-located legitimate xApps and lateral movement through the SDL to exfiltrate subscriber identities or manipulate platform state.

From this analysis, four functional requirements were established that directly shaped the system architecture:

1. Pre-deployment cryptographic verification of xApp packages and container images.
2. Short-lived workload identity bound to the active xApp instance, with support for renewal withdrawal.
3. Independent behavioral and resource-anomaly detection during runtime.
4. Automated, probe-verified containment of confirmed xApp compromise.

A non-functional requirement imposed by this project was air-gapped operation, modelling deployments in which external connectivity is unavailable or prohibited. Every tool selected for ZT-XGuard was therefore required to support offline operation during the evaluated workflow. This constraint ruled out verification paths that depend on a live external transparency-log service and shaped the offline Cosign configuration used in the onboarding pipeline.

3.3 Tools and Technologies

The security framework described in this project depends on a stack of open-source tools, each selected for its alignment with this project’s operating constraints (offline deployment, multi-vendor interoperability and strict latency bounds) and its established use in cloud-native environments. The rationale for each key component is summarized below.

GPG (GNU Privacy Guard): GPG is used for offline verification of xApp descriptor archives. Its detached-signature mechanism allows the SMO to validate the origin of configuration files using a pre-trusted public key without network access.

Cosign (Sigstore): Cosign [34] provides cryptographic signing and verification of container images. It supports fully offline signing using a private key and produces a self-contained bundle that can be verified locally, making it ideal for the air-gapped RIC registry.

ChartMuseum: ChartMuseum is a lightweight Helm chart repository deployed as a single container. It serves as the trusted vault on the SMO, enabling the RIC to pull verified charts over the local network without direct SMO-to-cluster push access.

Helm: Helm is the standard package manager for Kubernetes. It templates, versions, and deploys xApp workloads and its ability to pull charts from a re-

note repository with explicit version pinning ensures repeatable, controlled deployments.

SPIRE/SPIFFE: SPIRE implements the SPIFFE standard to provide Kubernetes native workload identity. It uses the PSAT attester to validate pod provenance and issues short-lived X.509 SVIDs for mutual TLS, aligning with O-RAN community adoption and zero-trust principles.

cert-manager: cert-manager acts as the upstream Certificate Authority (CA) for the entire SPIRE trust domain. It automates the issuance and renewal of the CA certificate backed by a self-signed ClusterIssuer, ensuring a consistent chain of trust across all SPIRE-issued identities.

Keylime: Keylime [35] is a remote attestation framework built on the TPM [36] that continuously monitors node integrity. Its ability to cryptographically verify boot state and runtime measurements and to automatically revoke payloads from compromised agents provides a hardware-rooted trust anchor for the platform.

Falco: Falco [28] is a runtime security tool that uses Extended Berkeley Packet Filter (eBPF) to monitor system calls inside containers. Alerts are forwarded to the containment orchestrator through Falcosidekick [37]. It can detect anomalous behavior such as shell spawning or sensitive file access within RIC namespaces, supplying the trigger signal for automated containment actions.

cAdvisor + Prometheus: cAdvisor [38] performs resource monitoring and 1 Hz per-container telemetry scraping and Prometheus [39] provides the time-series aggregation for the statistical anomaly detector.

Kubernetes: Kubernetes [40] provides the orchestration layer for the Near-RT RIC. Its native primitives, namespaces, network policies and resource quotas, are extended with custom attestation-driven policies to implement identity-aware isolation of compromised xApps.

Statistical/Math Libraries: The scikit-learn library [41] is employed by the resource-anomaly detection engine to calculate the Ledoit-Wolf shrinkage covariance [42], the MEWMA and Hotelling's T^2 metrics. These tools facilitate the formulation of a unified pressure score to dynamically evaluate telemetry signals and avoid false positives.

Calico CNI Plugin: Calico [43] is the Container Network Interface (CNI) plugin that enforces the deny-all NetworkPolicy at the data plane, providing the micro-segmentation lock that severs all ingress and egress traffic from a compromised pod.

Node-level packet filter: Used by the containment engine to install the independent node-level network block, a drop rule in a dedicated chain in the host packet filter, that enforces isolation independently of the CNI policy pipeline as a defense-in-depth second network lock.

3.4 Hardware-Rooted Platform Attestation

Before any xApp is scheduled on the Near-RT RIC, the infrastructure nodes that will host it must be verified to be running an unmodified, trusted software stack. This requirement cannot be satisfied by software-only checks alone, since a compromised operating system or bootloader could trivially spoof the results of any in-band integrity verification. ZT-XGuard addresses this through Keylime, an open-source remote attestation framework that anchors its trust chain in hardware TPM measurements.

A Keylime Agent is installed on each monitored node. In this deployment, the srsRAN gNB node and the Near-RT RIC worker node. The agent interacts with the node's virtual TPM (vTPM) to collect Platform Configuration Register (PCR) values that record cryptographic measurements of the firmware, bootloader, operating system kernel and critical system files accumulated during the boot sequence. Since PCR values are computed as a chained hash of every measured component, any modification to the software stack at boot time produces a different PCR measurement that cannot be forged without knowledge of the TPM's endorsement key.

The attestation lifecycle operates as follows. During an initial trusted provisioning phase, a clean system boot is performed in a controlled lab environment. The resulting PCR values are captured and stored as the Reference Integrity Manifest (RIM) in the Keylime Registrar alongside the node's Attestation Identity Key (AIK). The Keylime Verifier then continuously challenges each registered agent by issuing a fresh nonce; the agent returns the current PCR values signed by the TPM over that nonce, proving both the integrity measurement and the freshness of the response against replay attacks. The Verifier compares the signed quote against the stored RIM and maintains the node's attestation status as either trusted or untrusted. Only nodes in a trusted state receive encrypted configuration payloads through the Keylime Tenant, ensuring that sensitive 5G configuration material is never delivered to a platform whose integrity cannot be verified.

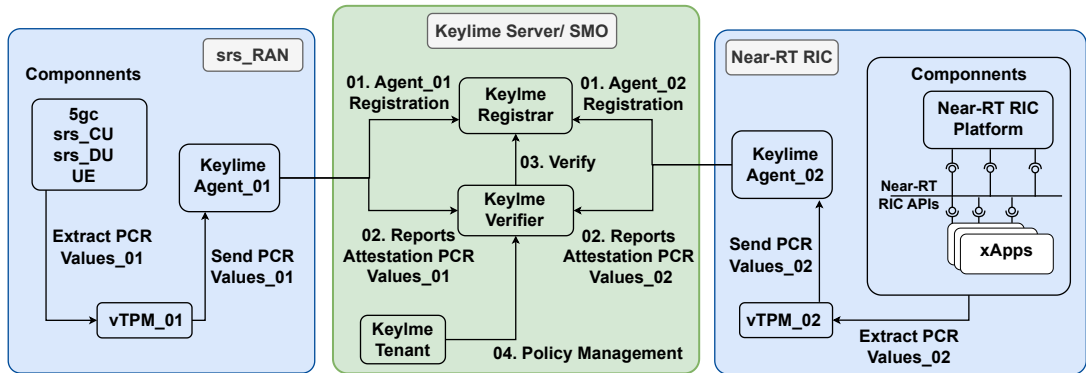


Figure 3.2: Remote attestation using Keylime open source software tool

This hardware-rooted trust anchor implements a platform-integrity mechanism aligned with WG11 secure-execution objectives and provides evidence that the monitored host's measured boot and load-time state matches the approved reference before protected workloads are deployed. This attestation covers the plat-

form’s measured boot and load-time integrity, meaning the firmware, bootloader, kernel and measured system files; it does not by itself detect runtime-only compromises that leave those measurements unchanged, which are the responsibility of the runtime detection channels described later.

3.5 Secure Onboarding and Workload Identity Attestation

The secure onboarding pipeline establishes cryptographic trust in an xApp before any of its code is permitted to execute on the Near-RT RIC. It spans three administrative domains: the xApp Solution Provider, the SMO and the Near-RT RIC and enforces two independent verification layers: a supply-chain integrity check that validates the origin and integrity of the xApp package and a runtime identity check that binds a cryptographically verifiable SPIFFE identity (SVID) to the running xApp pod.

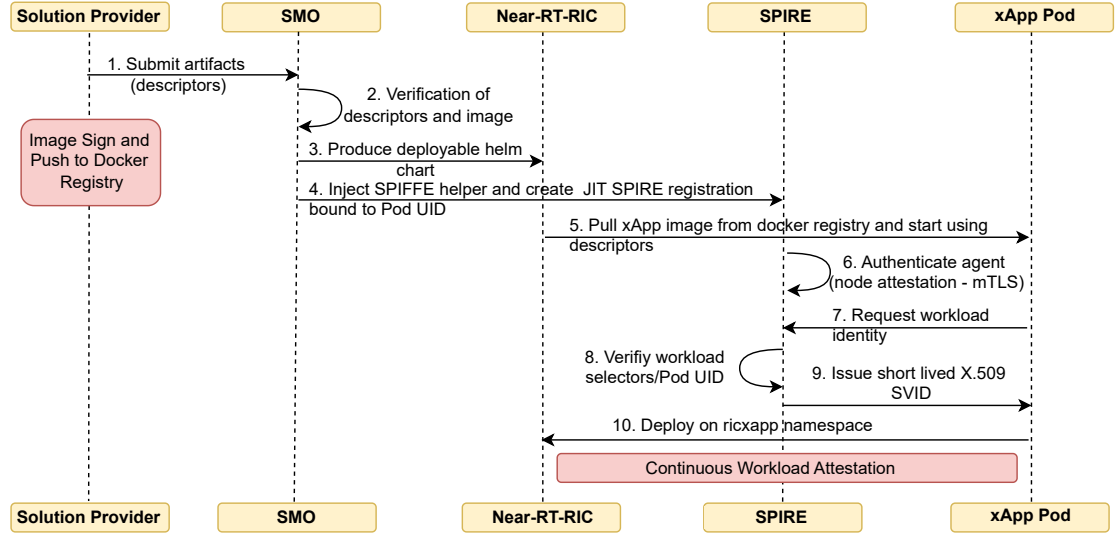


Figure 3.3: Secure xApp onboarding and workload-attestation sequence.

The identity procedure in this section belongs specifically to the three-VM onboarding testbed. It culminates in a JIT SPIRE entry bound to the active Pod UID and sidecar-managed SVID delivery. The separate runtime monitoring and containment testbed uses the `ClusterSPIFFEID`-based identity arrangement described in Section 3.6; no claim is made that an SVID issued in the onboarding testbed was transferred into, or revoked inside, the runtime testbed.

3.5.1 Secure Onboarding Architecture

The secure onboarding evaluation was implemented across three virtual machines representing the Solution Provider, the SMO platform and the Near-RT RIC as distinct administrative domains. The Solution Provider represents the external xApp developer, the SMO acts as the operator-controlled admission boundary that verifies the submitted descriptor and container-image evidence and prepares

the deployable package, and the Near-RT RIC hosts the Kubernetes environment in which an accepted xApp executes. SPIRE provides workload identity services to the deployed xApp, including short-lived X.509 SVID issuance and subsequent workload attestation. Figure 3.3 summarizes the interaction among these components, while the individual verification gates are described in Section 3.5.3.

3.5.2 Cryptographic Packaging at the Solution Provider

Before any packages are transferred, the Solution Provider and SMO will negotiate their public keys via a trusted out of band channel. The provider then constructs the xApp container image, and uploads the image to an external xApp container provider-managed or centralized O-RAN Docker registry. The digests of the image created by the digest function SHA-256 are signed using a Cosign private key in the offline bundle mode, which generates a `.bundle` file containing the image digest. A signature without using any external transparency log service. This design meets the air-gapped requirement as the signature can be verified based on the provider's public key and the bundle file.

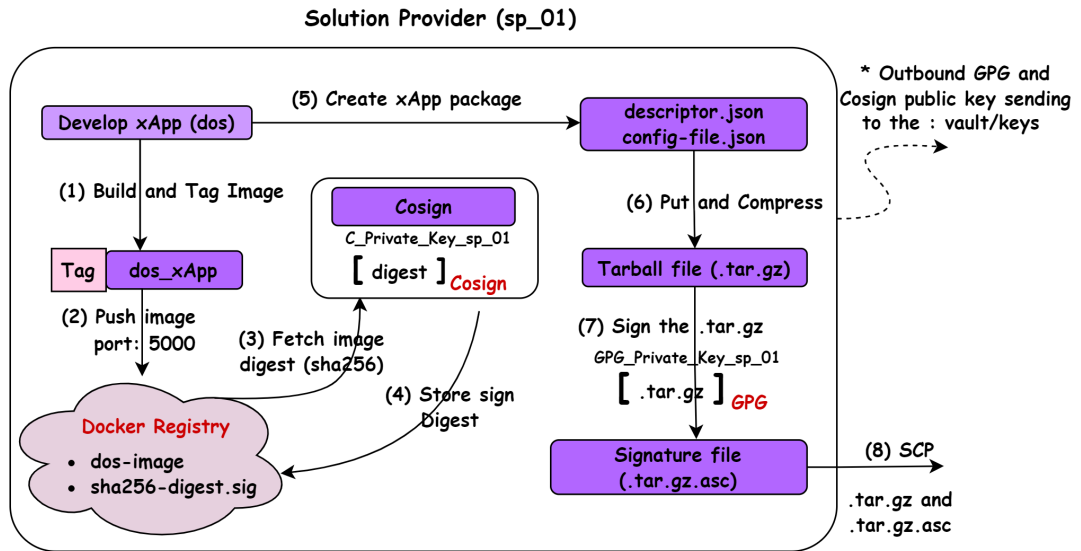


Figure 3.4: Workflow at the solution provider during secure onboarding

Each time, xApp descriptor files (`schema.json` and `config-file.json`) are bundled into a tarball and then signed with the provider's GPG private key, creating an `.asc` signature file. Only the detached signature and the signed tarball are securely moved to the SMO, the container image is still in the external registry until the SMO checks and syncs it to the RIC's private registry.

3.5.3 SMO Secure Ingestion Pipeline

The SMO executes `secure-ingestion.sh`, a pipeline script that enforces four sequential verification gates before any deployment artifact is produced. The pipeline terminates immediately and purges all intermediate files if any gate fails, ensuring that a tampered payload cannot persist in the ingestion environment.

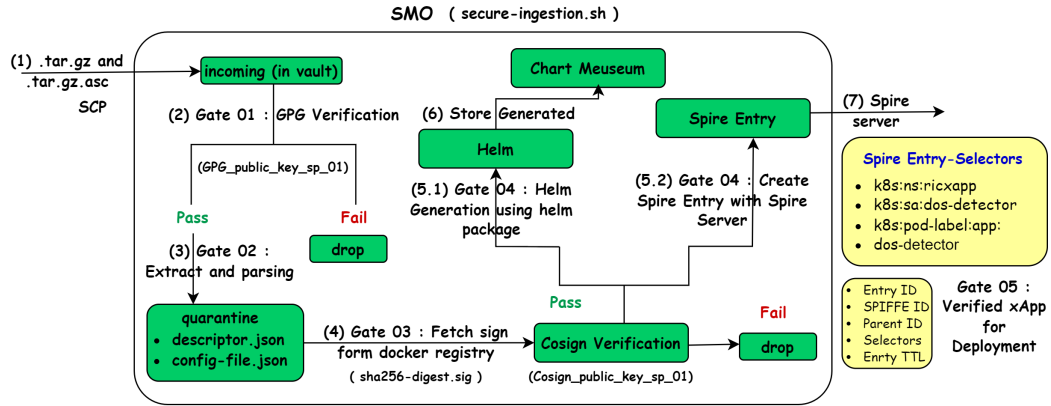


Figure 3.5: Workflow at the SMO during verification and secure chart preparation

Gate 1: GPG signature verification

The detached GPG signature of the descriptor tarball is verified against the provider's public key in the SMO keyring. A signature mismatch indicates that the descriptor files were modified after the provider signed them, regardless of whether the filename or archive structure was preserved. Any failure here terminates the pipeline before the tarball is even unpacked.

Gate 2: Descriptor extraction and parsing

The verified tarball is unpacked and the xApp name, version and container image reference are extracted from the descriptor JSON. These values are used in all subsequent pipeline steps.

Gate 3: Cosign container image verification

The `.bundle` file is transmitted to the provider and used to verify the referenced container image from the external provider managed registry against the Cosign public key provided by the provider. This assures the image in the external registry corresponds to the provider-built and signed image. If verification is successful, the image can be synchronized with the RIC's local private registry.

Gate 4: Sidecar injection and packaging

- The SMO ingestion pipeline automatically injects the required security infrastructure directly into the Helm templates before packaging the chart.
- A `spiffe-helper` sidecar container, a shared `/etc/svid` volume mount and the SPIRE Unix socket mounts are dynamically added to the xApp's Deployment definition.

- The finalized chart is then uploaded to the ChartMuseum Helm repository on the SMO.
- Unlike early iterations of this framework, no SPIRE registration entry is created at this stage; identity registration is deferred to a highly secure Just-In-Time (JIT) binding process during actual deployment.

3.5.4 RIC Deployment and SPIRE Workload Attestation

The RIC operator triggers deployment by executing `registration.sh`, which adds the SMO's ChartMuseum as a Helm repository and installs the xApp chart onto the Kubernetes cluster. As the pod is scheduled and enters its initialization phase (`Pending` → `Init:0/1`), the deployment script dynamically captures the pod's unique, active Kubernetes Pod UID. It immediately uses this identifier to create a Just-In-Time (JIT) entry in the SPIRE Server, binding the SPIFFE ID strictly to that exact pod instance (`k8s:pod-uid:<POD_UID>`).

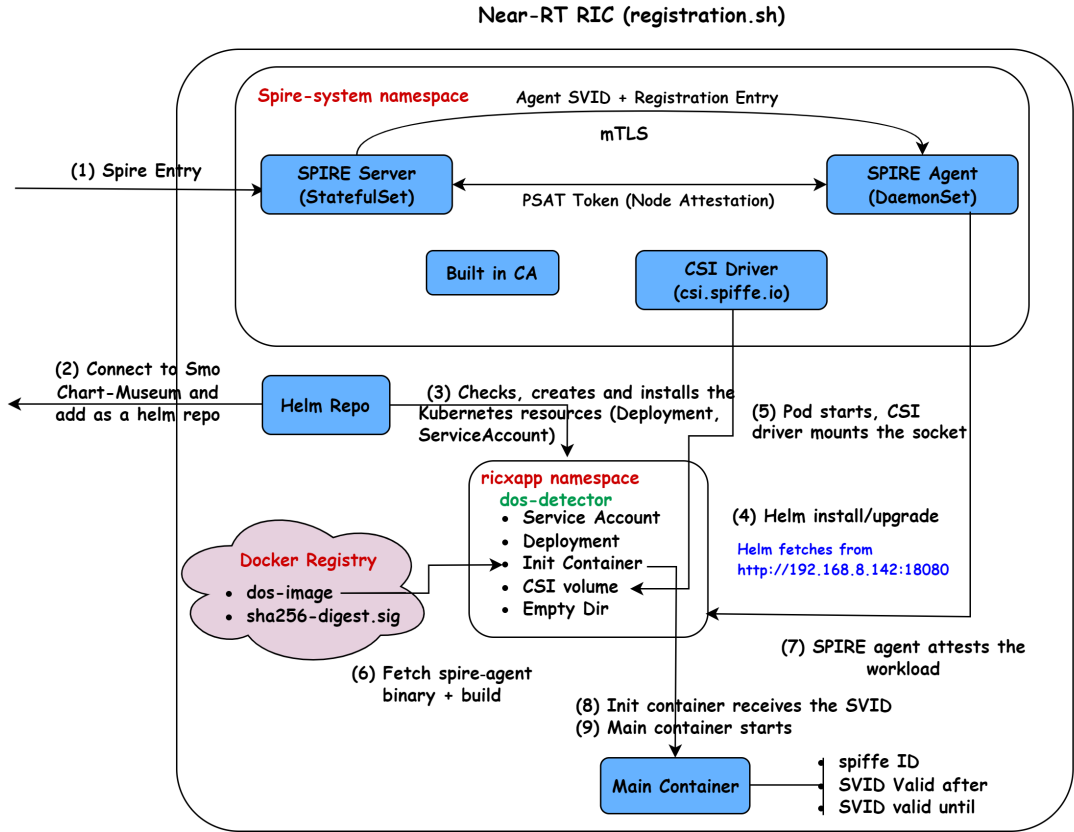


Figure 3.6: Workflow at the Near-RT RIC during xApp deployment and workload attestation

As the pod transitions to the running state (`2/2 Running`), both the main xApp container and the dynamically injected `spiffe-helper` sidecar launch simultaneously. Once running, the `spiffe-helper` sidecar establishes a handshake with the local SPIRE Agent over the shared Unix domain socket (`/var/run/spire/`

`sockets/spire-agent.sock`). The SPIRE Agent performs Kernel/Kubernetes-level attestation by verifying the calling process with the Kubernetes API Server using the captured Pod UID.

Upon successful attestation, the SPIRE server issues a short-lived X.509 SVID (five-minute TTL). The `spiffe-helper` sidecar receives this cryptographic material and writes the `ca.crt` (Root CA), `tls.crt` (X.509 Certificate with the Subject Alternative Name `spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector`), and `tls.key` (Private Key) directly into the shared `/etc/svid` volume. The main xApp container then utilizes this SVID for authenticated mutual TLS communication with RIC platform services. This architecture seamlessly transforms an unauthenticated xApp into a verified Zero-Trust workload without requiring any custom code changes inside the main application itself.

3.6 Runtime Detection Architecture

The workload-identity arrangement used in this section is local to the runtime monitoring and containment testbed. It is independent of the JIT Pod-UID entry used in the three-VM onboarding testbed. In the runtime testbed, a clusterSPIFFEID named `zt-xguard-ricxapp-identity` selects the protected xApp through the `svid-enabled` label, and a `spiffe-helper` sidecar maintains the short-lived SVID in `/etc/svid`. This declarative binding is also the revocation anchor: the responder removes the selecting label during containment, after which the SPIRE controller stops rendering the pod's registration entry and the current SVID is not renewed. Thus, the runtime experiments validate identity-backed containment independently of the onboarding experiment's identity-issuance path. Once an xApp is admitted and running, ZT-XGuard operates as a continuous monitoring and response layer around it. The runtime architecture is built on a single principle: evidence about an xApp's trustworthiness is gathered from two independent channels with deliberately different error profiles and that evidence is fused into one authoritative trust state by a central policy engine, which is the only component permitted to order containment. Figure 3.7 shows this arrangement.

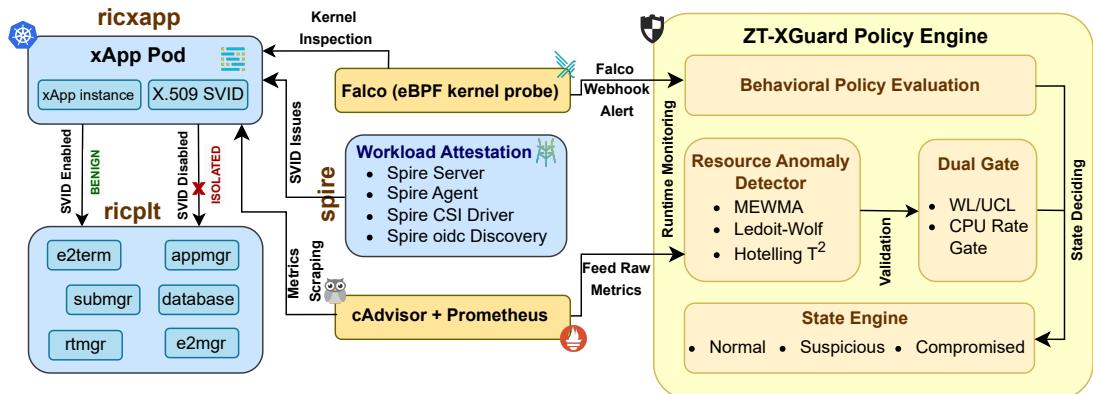


Figure 3.7: ZT-XGuard runtime detection architecture.

The runtime architecture combines two independent evidence paths: Falco/eBPF provides kernel-level behavioral evidence, while the statistical channel evaluates

cAdvisor/Prometheus resource telemetry. Both feed the central policy engine, which maintains the authoritative trust state and initiates the appropriate response. The two detection channels and the formal decision logic are detailed in Sections 3.8, 3.9 and 3.10, respectively.

The **policy engine** performs deterministic decision-level fusion. Each channel independently proposes a trust state, and the engine maintains exactly one of three states for every xApp:

$$T_x(t) \in \{\text{NORMAL}, \text{SUSPICIOUS}, \text{COMPROMISED}\}. \quad (3.1)$$

Isolation is a containment action applied after compromise and is not a fourth trust state.

3.7 Continuous System Observability

While secure onboarding establishes initial trust, the dynamic and privileged nature of xApps necessitates continuous runtime evaluation. Relying solely on static configurations is insufficient to prevent zero-day exploits, runtime tampering or novel resource-exhaustion attacks. Therefore, the foundation of the runtime security layer is the continuous aggregation of system-level and kernel-level telemetry. The framework employs two distinct observational channels to capture both behavioral security threats and anomalous resource utilization. The independence of these channels is architecturally intentional: a sophisticated attack may evade one channel while triggering the other and by treating them as separate evidence streams rather than a single composite signal, the framework avoids the failure modes that arise when all detection logic is fused into a single scoring path.

3.7.1 Behavioral Monitoring

Falco instruments the Linux kernel using eBPF probes attached to system-call tracepoints, so it observes the system calls made by processes inside monitored containers with negligible overhead on the data path. A system call is the interface a process uses to ask the kernel to do anything consequential, such as starting a program, opening a file or making a network connection. Watching this interface makes an attacker’s actions visible even when the exploit itself is new, because the attacker still has to spawn a shell, read a file or open a socket to make progress. The deployment scopes its rules to the `ricxapp` namespace so that only xApp activity is evaluated, which keeps the alert volume proportionate.

Each rule that fires produces a structured alert event carrying a stable signal name, a rule identifier and the pod, container and command context. Falcokick forwards the event over an HTTP POST webhook to the `/alert` endpoint, where it is evaluated by the policy engine. A critical event raises the xApp to **COMPROMISED** and initiates immediate isolation, whereas a warning-level event raises it to **SUSPICIOUS** without initiating full isolation. The rule set itself, its zero-trust allow-list design, the individual detection signals and how each maps to a trust-state decision, is described in Section 3.8.

3.7.2 Resource Monitoring

Container resource utilization is collected by cAdvisor, which runs as part of the Kubernetes kubelet on every node and exposes per-container metrics at 1 Hz through the kubelet endpoint. Prometheus is configured to scrape these metrics using the `kubernetes-nodes-cadvisor` target, aggregating the time-series data into the storage backend that the statistical anomaly detector reads from. The full set of 11 metrics collected per container is described in Section 3.9; only the four scoring features (CPU, memory growth rate, network RX rate, network TX rate) are fed into the anomaly detection model, while the remaining metrics serve as data quality guards (Note: In this scope, the detector is built only for the KPI monitoring xApp).

3.8 Behavioral Threat Detection with Falco

The behavioral channel is the framework’s answer to the class of attacks that a resource monitor cannot see. A compromised xApp that opens a shell, reads a secret, connects to an attacker-controlled server or tries to break out of its container is not necessarily consuming any unusual amount of CPU, memory or bandwidth, so it leaves no statistical footprint. What it does leave is a system call. This channel watches those system calls and reacts to the ones that a healthy xApp would never make.

3.8.1 Zero-Trust Rule Design Philosophy

Most intrusion-detection rule sets are written as a deny-list: they enumerate known-bad patterns and hope the attacker uses one of them. ZT-XGuard inverts this. Each rule encodes what is legitimate for a given xApp and it fires when the xApp does something outside that envelope, rather than when it matches a generic threat signature. This is the zero-trust position applied at the system-call layer: an xApp is not trusted to behave well simply because it was admitted, so any action it takes that its declared behavior does not account for is treated as evidence, even if that action is not on any published list of exploits.

Concretely, the rules are scoped tightly so that they evaluate only genuine xApp activity. They apply to the main xApp containers in the `ricxapp` namespace and deliberately exclude trusted platform sidecars and helper containers, including the `spiffe-helper`, because those would otherwise inflate the alert volume. Within that scope, each rule encodes a specific class of action that a healthy xApp never performs, so it fires on evidence of misuse rather than on ordinary operation.

The set of legitimate destinations an xApp needs, meaning the RIC platform services it contacts as part of its normal function, is enumerated explicitly, so that a connection to anything outside that set is treated as external egress. This matters because a RIC xApp such as the KPI monitoring xApp contacts platform services continuously, including E2 registration and subscription, the shared data layer and the metrics store and the framework must tell that legitimate control-plane traffic apart from an attacker’s outbound connection. Enumerating the

known-good destinations, rather than trying to blacklist bad ones, is what makes that distinction reliable.

The deployed rules are kept in agreement with the live cluster automatically. A background reconciler in the policy engine periodically re-checks the rules against the current cluster state, for example that a RIC platform service reached through a headless-service backing address has not moved after a pod restart and repairs any drift without human intervention. This keeps the rules accurate as the deployment changes and bounds the maintenance cost of the behavioral channel.

3.8.2 Representative Falco Rule Implementation

Listing 3.1 shows the deployed unexpected-shell rule applied to the KPI monitoring xApp, `kpimon-go`. The rule monitors `execve/execveat` events and detects shell binaries such as `sh` or `bash`, which are not part of normal `kpimon-go` operation. When triggered, it emits the `unexpected_shell` signal with `CRITICAL` severity, causing the policy engine to classify the xApp as `COMPROMISED` and initiate immediate containment.

Listing 3.1: ZT-XGuard unexpected-shell Falco rule applied to the KPI monitoring xApp

```
1 # kpimon-go is included in the controlled xApp scope
2 - macro: ztx_xapp_namespace
3   condition: k8s.ns.name = ricxapp
4
5 - macro: ztx_kpimon_go_pod
6   condition: >
7     k8s.pod.name startswith "ricxapp-kpimon-go"
8
9 - list: ztx_shell_binaries
10  items: [sh, bash, dash, zsh, ksh, ash]
11
12 # Representative critical rule applied to kpimon-go
13 - rule: ZTX xApp Unexpected Shell
14   desc: A shell was spawned inside a controlled xApp main container.
15   condition: >
16     evt.type in (execve, execveat) and
17     ztx_controlled_xapp and
18     proc.name in (ztx_shell_binaries) and
19     not ztx_declared_shell_entrypoint
20   output: >
21     ztx_signal=unexpected_shell
22     rule_id=ZTX-A1
23     xapp shell spawned
24     (user=%user.name
25      command=%proc.cmdline
26      pod=%k8s.pod.name
27      ns=%k8s.ns.name
28      container=%container.name
29      image=%container.image.repository)
30   priority: CRITICAL
31   tags: [zt-xguard, xapp, runtime, shell, compromise]
```

3.8.3 The Detection Rule Catalog

Table 3.1 lists the detection rules, the stable signal each emits, its severity and the trust-state decision it drives. The severity is not cosmetic: it is the bridge between an observed system call and the framework’s response. A **CRITICAL** rule maps to confirmed compromise and drives immediate isolation, because a single occurrence of the behavior it catches, an in-container shell or a read of workload-identity key material, is already strong evidence and any delay only widens the attacker’s window. A **WARNING** rule maps to the **SUSPICIOUS** state, where scrutiny rises and the reversible CPU throttle may apply but the xApp is not removed, because the behavior it catches, such as running a packaging tool, is suspicious but not on its own conclusive. The Falco severity is the rule’s own label; the policy engine (Section 3.10) makes the final trust decision and may escalate beyond it. The one such case is unauthorized execution of known attack or lateral-movement tooling: the Falco rule is warning-tier, but the policy engine places this signal in the same lateral-movement class as token theft and container escape and drives it directly to **COMPROMISED**.

Table 3.1: ZT-XGuard behavioral detection rules, their severity and the trust state each drives

Signal	Severity	Trust state	What it detects and why it matters
Unexpected shell	CRIT	COMP	A shell (sh , bash and similar) is started inside an xApp container. Production xApps never open interactive shells, so this signals a hands-on attacker or an exploit payload.
Sensitive file access	CRIT	COMP	An attempt to open /etc/shadow , /etc/sudoers , /etc/passwd or an SSH key path. These are the files an attacker reads to harvest credentials or escalate.
ServiceAccount token access	CRIT	COMP	A read of the pod’s mounted Kubernetes ServiceAccount token. A stolen token grants the attacker the pod’s own cluster API access.
SVID or SPIFFE material access	CRIT	COMP	Access to or tampering with SVID or SPIRE identity material (/etc/svid , /run/spire). This is the workload’s cryptographic identity, the anchor containment later revokes.
External egress attempt	CRIT	COMP	An outbound connection to any destination not on the xApp’s internal allow-list. This is the signature of data exfiltration or command-and-control.
Profile or config tamper	CRIT	COMP	A write or metadata change to the xApp’s verified profile or runtime configuration paths. Subverting an xApp’s own trusted configuration changes what it is allowed to do.
Container escape attempt	CRIT	COMP	Access to the container runtime socket (Docker, containerd or CRI-O). This is a well-known escape-to-host and privilege-escalation vector (MITRE ATT&CK T1611).
Malicious tool execution	WARN	COMP	Execution of known attack or lateral-movement tooling inside the container. The Falco rule is warning-tier, but the policy engine escalates this lateral-movement signal directly to compromise.

Signal	Severity	Trust state	What it detects and why it matters
Package manager execution	WARN	SUS	Execution of <code>apt</code> , <code>apk</code> , <code>pip</code> and similar. A running xApp installing software is a strong sign of tampering.
Binary drop in writable path	WARN	SUS	Writing an executable-looking artifact into <code>/tmp</code> , <code>/var/tmp</code> or <code>/dev/shm</code> . Typically staging a payload for later execution.
Permission tamper	WARN	SUS	Execution of <code>chmod</code> , <code>chown</code> or <code>setcap</code> . Changing permissions or capabilities to enable a follow-on step.
Kubernetes API contact	WARN	SUS	A connection to the Kubernetes API service on port 443. An xApp reaching for the cluster API is outside its normal role.
<i>Note:</i> CRIT = Critical, WARN = Warning, COMP = Compromised, SUS = Suspicious.			

Alongside these detection rules, a small set of INFORMATIONAL rules records legitimate behavior explicitly: normal identity renewal, expected in-cluster connections and the RIC platform contacts an xApp makes as part of its normal operation. These do not drive containment. They act as positive confirmation of healthy operation and prevent the framework from having to infer normality from the absence of an alert.

3.8.4 What the Rules Treat as Legitimate

The rules above are only as precise as the definitions of legitimate behavior they rest on. Those definitions are explicit. Credential-theft detection keys on a fixed list of sensitive paths (`/etc/shadow`, `/etc/sudoers`, `/etc/passwd` and SSH key locations); identity-theft detection keys on the SVID and SPIRE paths; configuration-tamper detection keys on the xApp’s profile and runtime-config paths; and payload-staging detection keys on writes of script or binary artifacts into the world-writable temporary directories. Tool-based signals key on named binaries: the shell interpreters, the read-and-exfiltrate tools (for example `cat`, `tar`, `base64`, `openssl`), the package managers and the permission-changing utilities.

Network legitimacy is defined by an allow-list of destinations rather than by blocking known-bad addresses. The list enumerates the real RIC platform services an xApp may contact (the E2 manager and termination, the application manager, the A1 mediator, the subscription manager, the route manager, Prometheus, SDL and the metrics database), the Kubernetes API and cluster DNS and the pod’s own loopback interface. Anything an xApp connects to that is not on this list is, by the zero-trust definition, external and worth flagging, which is exactly what the external-egress rule enforces.

3.8.5 Handling False Positives

A significant part of the behavioral-channel work was making the rules fire on genuine misuse and stay silent during the legitimate but unusual-looking things real xApps do at startup and during normal operation. Three cases are worth stating because they show the depth of the tuning.

First, a container’s own initialization looks superficially like an attack. The runtime’s `init` process and the xApp entrypoint legitimately read `/etc/passwd` and the pod’s ServiceAccount token as they start. The rules exempt the container’s PID-1 and entrypoint processes and, for the KPI monitoring xApp specifically, the two compiled binaries that its entrypoint forks and that load their own identity material at startup. The exemption is deliberately narrow: it only covers a read that comes from a process directly forked by the container’s own entrypoint, so a shell or any other differently parented process that reads the same files after a compromise is still flagged.

Second, metadata-changing system calls report their target differently from ordinary file opens. A `chmod`, `rename` or `unlink` does not populate the file-descriptor name field that a normal open does; it reports its target as a call argument instead. An earlier version of the tamper rules matched only the descriptor field, so a real permission- or rename-based tamper of identity material or configuration was silently missed. The rules were corrected to match both fields, which restored reliable detection of metadata-based tampering across every xApp.

Third, several legitimate RIC destinations are reached by raw IP address rather than by a resolvable name and some of them are backed by headless services whose addresses change when a pod restarts. Matching those destinations by name alone caused an xApp’s own healthy traffic to be misread as external egress. The destination allow-list therefore includes the stable service addresses and a small set of pod-backed addresses that a background reconciler keeps current automatically, so routine traffic does not drift into a false quarantine after a cluster event.

3.8.6 Decision-Synchronous Evaluation and Its Limits

The behavioral channel adds no scoring or aggregation stage between detection and response. The first rule that matches raises the xApp’s trust state and, for a CRITICAL rule, starts containment in the same step. This is what makes the channel fast: its latency is essentially the path from the kernel event through Falco to the decision endpoint, with no waiting for a trend to accumulate.

The channel has an inherent limit that follows from how it works. It observes new actions, such as a spawned process, an opened file or a new connection. An attacker who executes entirely inside an already-running xApp process, using code that is already loaded and spawning nothing new, produces a read that is indistinguishable at the system-call layer from a legitimate one. No purely syscall-based monitor can separate those two cases and ZT-XGuard does not claim to. This is one of the reasons the statistical channel exists as an independent second line: an in-process attack that avoids every behavioral rule still consumes resources and that is what the next section is designed to catch.

3.9 Statistical Resource-Anomaly Detector

Static threshold-based resource monitoring, for example, triggering an alert when a container’s CPU usage exceeds 80% is fundamentally unsuitable for the Near-RT RIC context. xApps that process burst RAN telemetry traffic legitimately exhibit sharp, transient CPU and network spikes that would trigger a static threshold repeatedly under normal operation, producing a false positive rate that makes the monitoring system operationally useless. The resource-anomaly detector in ZT-XGuard was designed specifically to overcome this limitation by evaluating resource metrics jointly through a multivariate statistical model calibrated on a clean xApp baseline and by requiring a sustained, physically corroborated deviation before emitting a threat signal. The statistical model is designed to be per xApp instance and the model built in this framework is designed for the KPI Monitoring xApp in the `ricxapp` namespace. Per-xApp calibration is necessary because a single global threshold cannot serve xApps with different normal resource profiles. A threshold set high enough not to flag a busy xApp would sit above the entire operating range of a quiet one, so an attack on the quiet xApp would never cross it, while a threshold low enough to protect the quiet xApp would fire continuously on the busy one. Fitting the baseline per xApp removes this tension. The limitation is that the model is established on one xApp; generalization of this fitting and calibration procedure to xApps with different normal behavior is argued by design rather than demonstrated here and is identified as future work.

Instead of evaluating metrics in isolation, the detector fuses multiple critical indicators of resource consumption, specifically CPU usage, memory growth rate, Network RX rate and Network TX rate into a unified statistical model.

The detector processes raw telemetry through four sequential stages, each taking the previous stage’s output as its input: (A) directional excess transformation, (B) an 8-second sliding-window mean, (C) MEWMA smoothing and (D) Hotelling’s T^2 fusion against the Ledoit-Wolf precision matrix. Table 3.2 summarizes the signal shape and cadence at each stage; the following subsections describe each stage in detail and Section 3.9.7 traces one real telemetry sample through all four stages numerically.

Table 3.2: The four-stage resource-anomaly scoring pipeline

Stage	Output shape	Description
Raw telemetry	1 vector (11-dim)/s	Unprocessed cAdvisor metrics
A: Directional excess	1 vector (4-dim)/sample	Rectified, standardized excess
B: Window mean	1 vector (4-dim)/s	8 s rolling arithmetic mean
C: MEWMA smoothing	1 vector (4-dim)/s	Exponentially smoothed state $\mathbf{Z}_t$
D: Hotelling T^2	1 scalar/s	Joint anomaly pressure score

3.9.1 Design Rationale and Development Process

Each component of the detector addresses a specific failure mode observed during development. The statistical scoring parameters were fitted and calibrated using clean data, whereas the downstream CPU rate-gate window and rate were selected using designated attack-development datasets. The complete pipeline was then frozen before any held-out dataset was evaluated.

Why a multivariate, memory-bearing statistic. A resource-abuse attacker inside an xApp does not need a single dramatic spike. It can drive a moderate, sustained elevation or duty-cycle on and off to keep its average load benign. This defeats both a static per-metric threshold, which the sustained-but-moderate attack stays under and an instantaneous multivariate score, which the duty-cycled attack evades between pulses. MEWMA’s exponential memory lets small persistent deviations accumulate smoothly instead of being averaged away at a window edge and its multivariate form fuses all resource channels while accounting for how they normally co-vary, so it raises an alarm only on jointly unusual behavior.

Why a statistical model rather than machine learning.

The resource-anomaly detector uses multivariate statistical process control instead of a learned classifier because the problem’s constraints select it. Supervised learning needs a large, representative set of labeled attacks, and no such corpus exists for resource abuse inside an O-RAN xApp. Manufacturing labels from a few synthetic stressors would teach the model only the patterns we happened to script and hide everything else behind a false sense of coverage. A statistical baseline avoids this entirely: it is fitted only on trusted clean telemetry, which is plentiful and can be gathered with confidence, and it flags any departure from normal rather than any resemblance to a known-bad example. Modelling normal behaviour and scoring deviations from it is the established approach when clean data is abundant and labeled anomalies are scarce [44].

The detector’s output also drives a consequential automatic action. A confirmed anomaly can remove a live control xApp from the RIC, so the decision has to be auditable and defensible after the fact. The Hotelling T^2 statistic is a single, mathematically defined distance from the calibrated baseline with a known reference distribution, so every escalation traces to which features deviated, by how much, and which gate confirmed it. For high-stakes decisions of this kind an interpretable model that can be reasoned about directly is preferable to an opaque one whose output cannot be explained or certified [45]. Its control limits are read straight from the clean score distribution, which makes the false-alarm rate a tunable design parameter rather than a byproduct of unavailable labels, and the frozen model is deterministic, runs offline, and adds about 0.004 of a core per context, well inside the 1 Hz edge budget.

Why robust centering. The baseline resource measurements are not normally distributed and can contain occasional high-valued spikes even during benign operation. This is illustrated in Figure 3.8, where the M1 CPU distribution is right-skewed with a long upper tail. The arithmetic mean (1.61 mCPU) is consequently shifted toward the tail, whereas the median (1.34 mCPU) remains close to the dense region of normal observations. For this reason, the baseline center and scale

are estimated using the median and median absolute deviation (MAD) rather than the mean and standard deviation, making the detector less sensitive to benign outliers. A scale floor, set to one tenth of the median feature scale, is additionally applied to prevent near-constant features from amplifying small measurement fluctuations into false anomaly evidence.

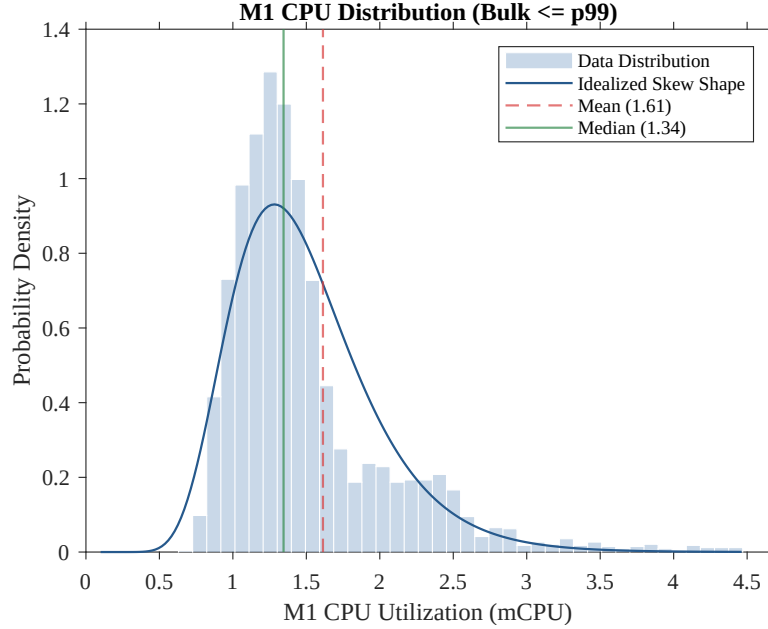


Figure 3.8: Distribution of the M1 CPU-utilization metric under benign baseline operation.

Why Ledoit-Wolf shrinkage covariance. Fusing the channels with a Hotelling T^2 statistic requires inverting the covariance matrix that describes how the features normally co-vary. A raw sample covariance keeps the correlation structure but can be numerically fragile to invert, while treating the features as independent would throw away exactly the joint-anomaly detection the detector exists for. Ledoit-Wolf shrinkage keeps the full correlation structure and guarantees a safely invertible matrix and the amount of correction is not a tuned hyperparameter but the output of a peer-reviewed formula that minimizes expected estimation error. On the fitted data the correction is small (intensity $\alpha^* = 2.5 \times 10^{-4}$), which follows from the large sample-to-feature ratio (about 3,490 states over four features) rather than from the raw matrix being already well conditioned; it improves the matrix’s condition number by roughly 28% as a safety margin while changing its values by only about 0.02%.

Why one-sided directional excess. An earlier version of the model used a symmetric distance from baseline and it produced false positives whenever legitimate traffic pushed the network RX or TX rate below baseline, which happens constantly and harmlessly when a lighter workload is scheduled. The frozen model replaces symmetric distance with a one-sided rectified excess, so only deviations above baseline contribute to the score. It is illustrated in Figure 3.9. This single correction removed a whole class of benign-drop false positives while keeping full sensitivity to the upward deviations that characterize resource abuse.

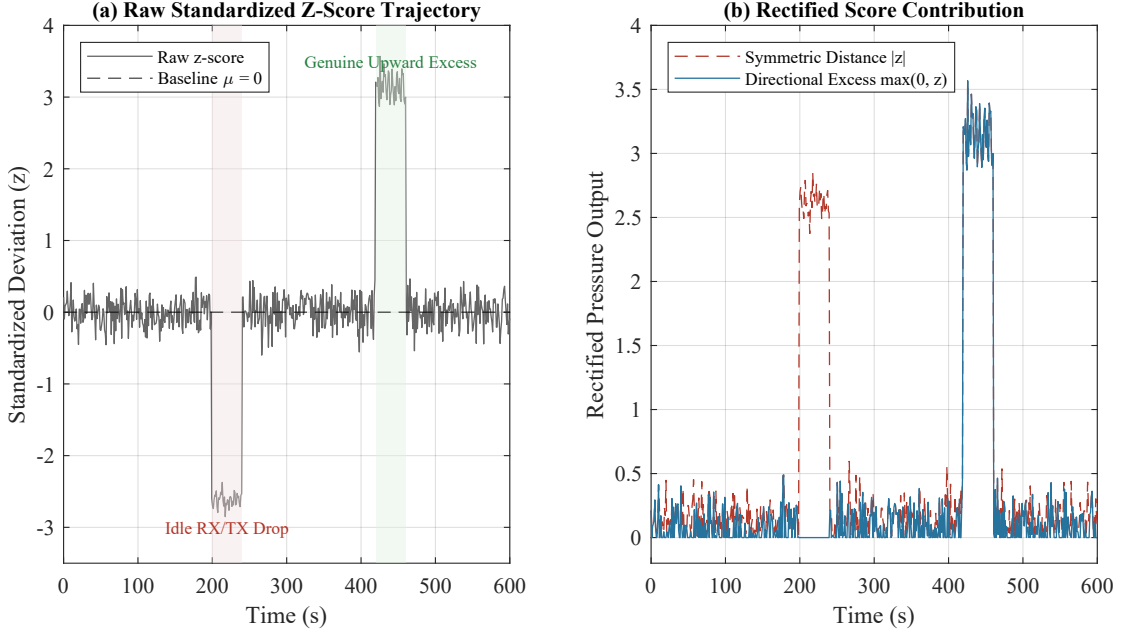


Figure 3.9: Raw standardized z-score vs rectified score contribution.

Dataset governance. The data were separated into fitting, calibration, development and held-out evaluation roles. Two clean baseline runs (N1 and N2) were used to fit the feature transforms, robust center and scale and shrinkage covariance. A third clean run (N3) was used to set the empirical control limits, and the CPU floor was derived from the pooled N1–N3 clean observations. A1 and A3 were designated development datasets and were used to select the downstream CPU rate-gate window ($L=60$) s and rate threshold ($r=0.15$). After that step, the complete scoring and decision pipeline was frozen. The held-out datasets A2, A4 and D1–D3 were not used to modify any model, threshold or gate parameter.

Development pipeline. The model was built in a fixed sequence: feature diagnostics to select the four scored channels from the eleven collected metrics; the transform and robust-centering step; the shrinkage-covariance fit; construction of the smoothing windows; the MEWMA and T^2 evaluation; nonparametric calibration of the control limits on N3; and selection of the CPU rate-gate window and rate on A1 and A3. Section 4.6 reports development results and held-out results separately.

3.9.2 Metric Selection and Feature Engineering

The cAdvisor telemetry stream provides 11 distinct metrics per container at 1 Hz. Table 3.3 describes the full set and the role each metric plays in the v5.0 detection model.

Four metrics are selected as scoring features for the T^2 model: M1 (CPU), M4 (memory growth rate), M5 (network RX rate) and M6 (network TX rate). The rationale for this selection is as follows. The absolute memory working-set metric (M3) was removed after empirical testing showed that legitimate xApp data caching behavior routinely caused it to exceed anomaly thresholds, producing false positives

Table 3.3: Complete Telemetry Metric Extraction and Model Role Mapping

ID	Column	Units	Meaning	Role in Model
M1	m1_cpu	mCPU	CPU usage	SCORE ($\log(1 + x)$)
M2	m2_cpu_throttle	mCPU	CFS throttle ratio	Quality guard only
M3	m3_mem_ws	Bytes	Working-set memory	Removed (caused FPs)
M4	m4_mem_growth	B/s	Memory growth rate	SCORE (identity)
M5	m5_net_rx	B/s	Network RX rate	SCORE ($\log(1 + x)$)
M6	m6_net_tx	B/s	Network TX rate	SCORE ($\log(1 + x)$)
M7	m7_fs_read	B/s	File System read rate	Deterministic guard
M8	m8_fs_write	B/s	File System write rate	Sparse-event guard
M9	m9_socket_count	count	Open socket count	Invariant guard
M10	m10_fd_count	count	Open file-descriptor count	Invariant guard
M11	m11_proc_count	count	Process and thread count	Invariant guard

regardless of the threshold level. Memory growth rate (M4) is used instead because it reflects the rate at which memory is being allocated rather than the level of retained allocations. Filesystem I/O metrics (M7, M8) and process and thread counts (M9–M11) are retained only as data quality guards used in the eligibility filtering stage; they do not enter the covariance matrix because they are either deterministic (filesystem activity is nearly absent during normal xApp operation) or invariant (thread and file-descriptor counts are stable for a given xApp version).

Because CPU usage and network throughput exhibit heavily right-skewed distributions with occasional extreme outliers driven by garbage collection pauses and scheduler jitter, a $\log(1 + x)$ transformation is applied to M1, M5 and M6 before any further processing. Memory growth (M4) is left untransformed because it naturally centers around zero and the log transformation would destroy the sign information that distinguishes memory growth from memory release.

3.9.3 Robust Centering and Scale Estimation

The four transformed features are normalized using robust statistics derived from the clean baseline datasets (N1 and N2):

$$X_{\text{std}} = \frac{X - \text{Median}}{1.4826 \times \text{MAD}} \quad (3.2)$$

where MAD is the Median Absolute Deviation [46] and the factor 1.4826 makes the estimator consistent with the standard deviation under a Gaussian distribution. The median and MAD are resistant to the heavy-tailed spikes in container telemetry in a way that the arithmetic mean and standard deviation are not; a

single outlier sample cannot shift the center estimate or inflate the scale estimate in a way that contaminates the baseline model.

A scale floor equal to 10% of the median scale across all four features is applied to prevent near-constant features from producing a scale estimate close to zero, which would amplify minor numerical noise into arbitrarily large normalized deviations.

The center and scale parameters derived from N1 and N2 are frozen after fitting and are never updated from any subsequent data, including the evaluation datasets. This is a hard methodological constraint: re-fitting the normalizer on any data that overlaps with the evaluation sets would constitute data leakage and would invalidate the reported detection metrics.

Figure 3.8 shows why the model uses median and MAD rather than mean and standard deviation for centering. The raw CPU feature is heavy-tailed, with a maximum many times its median, so a mean and variance baseline would be inflated by benign bursts and desensitized, while the robust estimator keeps the benign baseline tightly centered.

3.9.4 Directional Excess Transformation

Before entering the MEWMA tracker, each normalized feature is passed through a directional excess transform:

$$e_i(t) = \max(0, x_{\text{std},i}(t)) \quad (3.3)$$

This one-sided operation ensures that only upward deviations from the clean baseline contribute to the anomaly score. Downward drift, for example, a reduction in network throughput when a less demanding workload is scheduled, produces an excess of zero and does not increase the T^2 score. This design choice was motivated by a failure mode observed in an earlier two-sided formulation of the model: legitimate variations in the runtime environment between the baseline collection period and the evaluation period produced small systematic downward shifts in M5 and M6, which the two-sided statistic treated as equally suspicious to upward deviations, yielding a 0% NORMAL occupancy on clean evaluation datasets. (Figure 3.9) The directional excess formulation eliminates this failure mode while preserving full sensitivity to the upward deviations that characterize genuine resource-exhaustion attacks.

3.9.5 Sliding Window Mean Smoothing

The directional excess vector is smoothed over an 8-second sliding window before entering the MEWMA tracker:

$$\bar{e}_i(t) = \frac{1}{W} \sum_{k=t-W+1}^t e_i(k), \quad W = 8 \quad (3.4)$$

This step attenuates single-sample noise spikes that would otherwise propagate into the MEWMA state and produce transient T^2 excursions above the UCL that

do not reflect a genuine sustained anomaly.

3.9.6 MEWMA Tracking

The smoothed excess vector is processed by a Multivariate Exponentially Weighted Moving Average (MEWMA) tracker [19]:

$$\mathbf{Z}_t = \lambda \bar{\mathbf{e}}_t + (1 - \lambda) \mathbf{Z}_{t-1} \quad (3.5)$$

where $\lambda = 0.2063$ corresponds to a half-life of 3.0 seconds, meaning that the influence of any single observation decays to 50% of its initial weight after approximately three time steps. The MEWMA state $\mathbf{Z}_t$ therefore maintains an exponentially decaying memory of recent behavior rather than a hard window, allowing it to accumulate evidence of a moderate, sustained deviation over multiple seconds without requiring a single dramatic spike to trigger detection. This property is essential for detecting the low-and-slow resource-exhaustion attacks described in Scenario A3, in which the attack intensity is deliberately kept below simple threshold levels.

This specific combination of an 8-second window and $\lambda = 0.2063$ was not chosen arbitrarily; it was selected from a systematic trade-off analysis. Table 3.4 reports the noise-suppression ratio for candidate window lengths of 5–10 s and the resulting total pipeline latency (window length plus the time for the MEWMA state to reach 90% convergence, $\approx 3.32 \times H$) for half-lives H of 2–8 s. The noise-suppression ratio decreases monotonically with window size: a 10 s window always suppresses more single-sample noise than an 8 s window, which always suppresses more than a 5 s window, with no crossover or local optimum in this dimension. The 8 s window paired with $H = 3$ s ($\lambda = 0.206299$, highlighted) was selected as the operating point that balances adequate noise suppression (ratio = 0.2611) against a total pipeline latency (27 s) still well inside the sub-minute containment budget required by the Near-RT RIC.

Table 3.4: Noise-suppression ratio and total pipeline latency (s) across candidate window lengths and MEWMA half-lives

Window	Ratio	$H=2$	$H=3$	$H=4$	$H=5$	$H=6$	$H=8$
5 s	0.3467	18	24	31	38	44	58
6 s	0.3128	19	25	32	39	45	59
7 s	0.2842	20	26	33	40	46	60
8 s	0.2611	21	27	34	41	47	61
9 s	0.2420	22	28	35	42	48	62
10 s	0.2248	23	29	36	43	49	63

The recency-weighting factor λ is itself derived from the more intuitive half-life parameter H (the time after which a past observation’s influence decays to 50%) via

$$\lambda = 1 - 2^{-1/H} \quad (3.6)$$

which for $H = 3$ s gives $\lambda = 0.206299$, the frozen value used throughout the model.

Upon xApp startup, a warmup period of $\lceil \log(0.01)/\log(1 - \lambda) \rceil = 20$ samples is enforced during which no public state is emitted. This prevents the initialization transient, during which $\mathbf{Z}_t$ has not yet converged to a meaningful estimate of the xApp’s resource behavior, from producing spurious alerts.

Table 3.4 reports the latency and noise-suppression trade-off across the candidate settings. The frozen choice ($W = 8$ s, $H = 3$ s and $\lambda = 0.2063$) lies within the smooth monotonic grid rather than at an isolated discontinuity.

3.9.7 Ledoit-Wolf Shrinkage Covariance Estimation and Hotelling’s T^2 Fusion

The four-dimensional MEWMA state vector $\mathbf{Z}_t$ is fused into a single scalar anomaly score using Hotelling’s T^2 statistic [20]:

$$T^2 = \mathbf{Z}_t^T \boldsymbol{\Sigma}_{\text{lw}}^{-1} \mathbf{Z}_t \quad (3.7)$$

where $\boldsymbol{\Sigma}_{\text{lw}}^{-1}$ is the inverse of the Ledoit-Wolf shrinkage estimator of the covariance matrix of the MEWMA state, computed from the clean baseline data and frozen alongside the center and scale parameters.

A raw sample covariance matrix is not used directly because the four scoring metrics are highly correlated. CPU usage and network throughput co-vary strongly under normal xApp operation and the presence of off-diagonal terms with magnitudes differing by up to three orders of magnitude from the diagonal terms makes the sample covariance matrix numerically fragile to invert. The Ledoit-Wolf shrinkage estimator [42] addresses this by blending the sample covariance toward a scaled identity matrix target:

$$\boldsymbol{\Sigma}_{\text{lw}} = \alpha^* \mathbf{F} + (1 - \alpha^*) \mathbf{S} \quad (3.8)$$

Table 3.5 defines the notation used in the shrinkage derivation.

Table 3.5: Notation used in the Ledoit-Wolf shrinkage covariance derivation

Symbol	Name	Definition
n	Sample size	Pooled N1+N2 post-warmup MEWMA states ($n = 3490$)
p	Dimensionality	Number of scoring features ($p = 4$: M1, M4, M5, M6)
$\mathbf{x}_i$	One observation	The i -th MEWMA state, a p -dimensional vector
$\mathbf{S}$	Sample covariance	Raw covariance estimate of the pooled MEWMA states
$\mathbf{I}$	Identity matrix	The always-invertible shrinkage-target shape
$\text{Tr}(\mathbf{S})$	Trace	Sum of $\mathbf{S}$'s diagonal entries
μ	Shrinkage-target scale	$\mu = \text{Tr}(\mathbf{S})/p$, the average per-feature variance
$\mathbf{F}$	Shrinkage target	$\mathbf{F} = \mu\mathbf{I}$: the safe-fallback structure assuming equal variance and zero correlation
δ^2	Distance from target	$\delta^2 = \ \mathbf{S} - \mathbf{F}\ _F^2$, the squared Frobenius norm
$\bar{\beta}^2$	Sampling noise in $\mathbf{S}$	$\bar{\beta}^2 = \frac{1}{n^2} \sum_{i=1}^n \ \mathbf{x}_i \mathbf{x}_i^T - \mathbf{S}\ _F^2$
β^2	Capped noise term	$\beta^2 = \min(\bar{\beta}^2, \delta^2)$
α^*	Optimal shrinkage intensity	$\alpha^* = \beta^2 / \delta^2$
Σ_{lw}	Final shrunk covariance	$\alpha^* \mathbf{F} + (1 - \alpha^*) \mathbf{S}$, inverted to give the precision matrix

Worked example

The covariance model was fitted using the $n = 3490$ pooled post-warmup MEWMA states from N1 and N2. Each row contains the four scoring dimensions in the order M1, M4, M5 and M6. Thus, the fitted data matrix has the form

$$\mathbf{X} \in R^{3490 \times 4}$$

with the first five rows given by

$$\mathbf{X}_{1:5} = \begin{bmatrix} 0.7015 & 0.0611 & 21.9420 & 15.4992 \\ 0.7353 & 0.0485 & 23.5860 & 16.7787 \\ 0.7339 & 0.0385 & 24.8907 & 17.7941 \\ 0.7059 & 0.0306 & 23.9816 & 17.4509 \\ 0.6020 & 0.0243 & 21.2149 & 15.5067 \end{bmatrix}$$

Step 1: Calculate the sample covariance matrix $\mathbf{S}$.

For the fitted MEWMA states, the covariance entries are obtained from

$$S_{jk} = \frac{1}{n} \sum_{i=1}^n x_{ij}x_{ik}, \quad n = 3490. \quad (3.9)$$

For example, the first diagonal element represents the M1 contribution:

$$\begin{aligned} S_{11} &= \frac{1}{3490} \sum_{i=1}^{3490} x_{i,M1}x_{i,M1} \\ &= \frac{2273.5433}{3490} \\ &= 0.6514. \end{aligned}$$

Repeating this calculation for all four feature pairs gives

$$\mathbf{S} = \begin{bmatrix} 0.6514 & 0.2945 & 16.3256 & 10.3243 \\ 0.2945 & 0.3954 & 8.6105 & 5.5041 \\ 16.3256 & 8.6105 & 512.3698 & 312.9993 \\ 10.3243 & 5.5041 & 312.9993 & 197.8463 \end{bmatrix}.$$

Step 2: Calculate the shrinkage target $\mathbf{F}$.

The trace of $\mathbf{S}$ is the sum of its four diagonal entries:

$$\begin{aligned} \text{Tr}(\mathbf{S}) &= 0.6514 + 0.3954 + 512.3698 + 197.8463 \\ &= 711.2630. \end{aligned}$$

Since the model contains $p = 4$ scoring features, the average variance is

$$\begin{aligned} \mu &= \frac{\text{Tr}(\mathbf{S})}{p} \\ &= \frac{711.2630}{4} \\ &= 177.8157. \end{aligned} \quad (3.10)$$

The Ledoit–Wolf shrinkage target is therefore

$$\mathbf{F} = \mu \mathbf{I} = \begin{bmatrix} 177.8157 & 0 & 0 & 0 \\ 0 & 177.8157 & 0 & 0 \\ 0 & 0 & 177.8157 & 0 \\ 0 & 0 & 0 & 177.8157 \end{bmatrix}. \quad (3.11)$$

Step 3: Calculate the distance between $\mathbf{S}$ and $\mathbf{F}$

$$\mathbf{S} - \mathbf{F} = \begin{bmatrix} -177.1643 & 0.2945 & 16.3256 & 10.3243 \\ 0.2945 & -177.4204 & 8.6105 & 5.5041 \\ 16.3256 & 8.6105 & 334.5541 & 312.9993 \\ 10.3243 & 5.5041 & 312.9993 & 20.0306 \end{bmatrix} \quad (3.12)$$

The squared Frobenius distance is obtained by squaring every element of this difference matrix and summing them:

$$\delta^2 = \|\mathbf{S} - \mathbf{F}\|_F^2 = \sum_{j=1}^4 \sum_{k=1}^4 (S_{jk} - F_{jk})^2. \quad (3.13)$$

For the fitted baseline,

$$\delta^2 = 372085.295946$$

Step 4: Estimate the sampling noise in $\mathbf{S}$.

For each MEWMA observation $\mathbf{x}_i$, the outer product $\mathbf{x}_i \mathbf{x}_i^T$ gives that observation's contribution to the covariance structure. Its difference from the overall covariance matrix is therefore

$$\mathbf{x}_i \mathbf{x}_i^T - \mathbf{S}. \quad (3.14)$$

The sampling-noise estimate is

$$\bar{\beta}^2 = \frac{1}{n^2} \sum_{i=1}^n \|\mathbf{x}_i \mathbf{x}_i^T - \mathbf{S}\|_F^2. \quad (3.15)$$

Using all 3490 pooled MEWMA states gives

$$\bar{\beta}^2 = 92.905751$$

The Ledoit–Wolf procedure caps this term by δ^2 :

$$\begin{aligned} \beta^2 &= \min(\bar{\beta}^2, \delta^2) \\ &= \min(92.905751, 372085.295946) \\ &= 92.905751. \end{aligned} \quad (3.16)$$

Step 5: Calculate the shrinkage intensity α^*

The optimal shrinkage intensity is

$$\alpha^* = \frac{\beta^2}{\delta^2}. \quad (3.17)$$

Substituting the fitted values,

$$\begin{aligned} \alpha^* &= \frac{92.905751}{372085.295946} \\ &= 0.0002496894 \end{aligned}$$

This is the fraction by which the sample covariance is moved toward the scaled-identity target $\mathbf{F}$.

Step 6: Calculate the Ledoit–Wolf covariance matrix.

The final covariance estimate is

$$\Sigma_{\text{lw}} = \alpha^* \mathbf{F} + (1 - \alpha^*) \mathbf{S} \quad (3.18)$$

Using $\alpha^* = 0.0002496894$ gives

$$\Sigma_{\text{lw}} = \begin{bmatrix} 0.6957 & 0.2945 & 16.3216 & 10.3217 \\ 0.2945 & 0.4397 & 8.6083 & 5.5027 \\ 16.3216 & 8.6083 & 512.2863 & 312.9212 \\ 10.3217 & 5.5027 & 312.9212 & 197.8413 \end{bmatrix}.$$

The inverse of this matrix is the precision matrix used by the Hotelling T^2 score:

$$\Sigma_{\text{lw}}^{-1} = \begin{bmatrix} 6.3699 & -0.1638 & 0.0006 & -0.3287 \\ -0.1638 & 3.4997 & 0.0192 & -0.1191 \\ 0.0006 & 0.0192 & 0.0578 & -0.0919 \\ -0.3287 & -0.1191 & -0.0919 & 0.1709 \end{bmatrix}.$$

Step 7: Derivation of the MEWMA State and Hotelling T^2 Score.

The Hotelling T^2 calculation uses the MEWMA state rather than the instantaneous resource measurements. The worked example therefore follows the same sequence used by the detector: directional excess, an 8-second window mean, MEWMA recursion and finally the T^2 calculation.

Stage A: directional excess.

For an example, at $t = 635.004$ s, the four scoring features produce the directional-excess values shown in Table 3.6. M1, M5 and M6 are below their respective

baseline centres and therefore contribute zero, whereas M4 contributes a positive excess of 148.3251.

Table 3.6: Directional-excess calculation for the first sample of the worked example

Feature	Raw	Transform	Centre	Scale	Excess
M1 CPU	1.056 mCPU	0.7208	0.85229	0.14962	0
M4 Memory growth	493759.4 B/s	493759.4	906.57	3322.79	148.3251
M5 RX	536.581 B/s	6.2871	6.51111	0.00767	0
M6 TX	838.783 B/s	6.7331	6.88395	0.00767	0

Stage B: first 8-second window vector.

The detector uses an 8-second sliding mean with a one-second stride:

$$\mathbf{x}_t = \frac{1}{8} \sum_{j=0}^7 \mathbf{e}_{t-j}. \quad (3.19)$$

For the first complete window, the eight directional-excess samples are

Table 3.7: Eight-sample excess window used to calculate $\mathbf{x}_1$

t (s)	M1	M4	M5	M6
635.004	0.0000	148.3251	0.0000	0.0000
636.002	0.0000	140.0996	0.0000	0.0000
637.003	0.0000	67.7692	0.0000	0.0000
638.002	2.6976	64.1169	51.7391	38.0660
639.002	0.0000	22.8044	0.0000	0.0000
640.003	0.0000	19.6975	0.0000	0.0000
641.003	19.2213	17.6107	0.0000	0.0000
642.002	29.7683	16.8784	0.0000	0.0000
Mean	6.4609	62.1627	6.4674	4.7583

For example, the M1 component is

$$\frac{0 + 0 + 0 + 2.6976 + 0 + 0 + 19.2213 + 29.7683}{8} = 6.4609,$$

Similarly,

$$\mathbf{z}_1 = \begin{bmatrix} 6.4609 \\ 62.1627 \\ 6.4674 \\ 4.7583 \end{bmatrix}$$

Stage C: MEWMA state calculation.

The MEWMA recursion is

$$\mathbf{Z}_t = \lambda \mathbf{z}_t + (1 - \lambda) \mathbf{Z}_{t-1}, \quad (3.20)$$

with the frozen value

$$\lambda = 0.206299, \quad 1 - \lambda = 0.793701, \quad \mathbf{Z}_0 = \mathbf{0}.$$

The first MEWMA state is therefore

$$\begin{aligned} \mathbf{Z}_1 &= 0.206299 \begin{bmatrix} 6.4609 \\ 62.1627 \\ 6.4674 \\ 4.7583 \end{bmatrix} + 0.793701 \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \\ &= \begin{bmatrix} 1.333 \\ 12.824 \\ 1.334 \\ 0.982 \end{bmatrix}. \end{aligned}$$

Because the sliding window advances by one second, the following window means $\mathbf{x}_2$, $\mathbf{x}_3$ and $\mathbf{x}_4$ are processed in the same way:

$$\begin{aligned} \mathbf{Z}_2 &= 0.206299 \mathbf{x}_2 + 0.793701 \mathbf{Z}_1, \\ \mathbf{Z}_3 &= 0.206299 \mathbf{x}_3 + 0.793701 \mathbf{Z}_2, \\ \mathbf{Z}_4 &= 0.206299 \mathbf{x}_4 + 0.793701 \mathbf{Z}_3. \end{aligned}$$

Applying this recursion to the successive window-mean vectors gives the state used by the T^2 calculation:

$$\mathbf{Z}_4 = \begin{bmatrix} 7.723 \\ 22.039 \\ 7.111 \\ 4.602 \end{bmatrix}$$

Stage D: Hotelling T^2 .

Using the previously derived Ledoit–Wolf precision matrix,

$$T^2 = \mathbf{Z}_4^T \Sigma_{\text{lw}}^{-1} \mathbf{Z}_4. \quad (3.21)$$

By substituting,

$$\begin{aligned} \Sigma_{\text{lw}}^{-1} \mathbf{Z}_4 &= \begin{bmatrix} 6.3699 & -0.1638 & 0.0006 & -0.3287 \\ -0.1638 & 3.4997 & 0.0192 & -0.1191 \\ 0.0006 & 0.0192 & 0.0578 & -0.0919 \\ -0.3287 & -0.1191 & -0.0919 & 0.1709 \end{bmatrix} \begin{bmatrix} 7.723 \\ 22.039 \\ 7.111 \\ 4.602 \end{bmatrix} \\ &= \begin{bmatrix} 44.077 \\ 75.454 \\ 0.415 \\ -5.031 \end{bmatrix}. \end{aligned}$$

The final scalar score is therefore

$$\begin{aligned} T^2 &= 7.723(44.077) + 22.039(75.454) + 7.111(0.415) + 4.602(-5.031) \\ &= 340.41 + 1663.00 + 2.95 - 23.16 \\ &= 1983.17 \end{aligned}$$

3.9.8 Calibration of Control Limits

The Warning Limit (WL) and Upper Control Limit (UCL) are derived from the empirical quantiles of the T^2 score distribution computed on the N3 clean baseline dataset, which was held out from the fitting and normalization phases specifically for this purpose:

$$\text{WL} = Q_{0.975}(\text{N3 } T^2 \text{ scores}) = 10.4364 \quad (3.22)$$

$$\text{UCL} = Q_{0.995}(\text{N3 } T^2 \text{ scores}) = 15.3084 \quad (3.23)$$

The CPU floor used in the rate gate is the maximum of the pooled N1, N2 and N3 clean CPU observations ($n = 5400$). Interpreted as a one-sided, distribution-free Wilks tolerance bound [47], this sample maximum provides at least 95% confidence that at least 99.9% of the underlying clean CPU distribution lies at or below the bound, assuming the future observations are drawn from the same distribution as the calibration data:

$$\text{CPU floor} = 56.638 \text{ mCPU} \quad (3.24)$$

Figure 3.10 shows the resulting bound. The tolerance statement concerns coverage of the calibration population; it does not guarantee zero exceedances under

arbitrary workload or distribution shift. The nominal distribution-free Wilks interpretation also assumes independent calibration observations. Because 1 Hz container telemetry can be temporally correlated, the tolerance bound is treated as a conservative calibration rationale rather than an exact guarantee for the deployed time series. Operational specificity is assessed separately at the run level on the held-out clean datasets in Section 4.6.

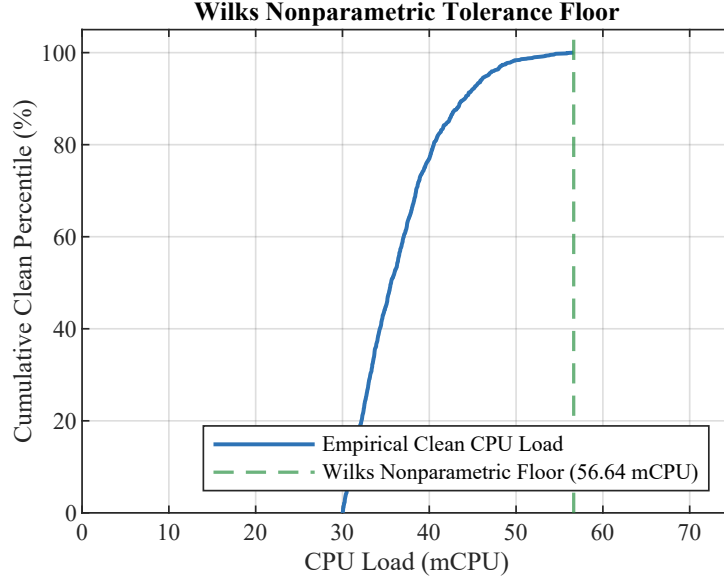


Figure 3.10: CPU rate-gate floor derived from the maximum of the pooled clean CPU samples ($n = 5400$), interpreted as a one-sided Wilks tolerance bound with at least 95% confidence and 99.9% population coverage under the calibration distribution, giving 56.638 mCPU.

All clean-derived scoring and calibration parameters were frozen after derivation. The gate window length ($L = 60$ s) and rate threshold ($r = 0.15$) were subsequently selected using A1 and A3 as designated development datasets. The complete pipeline was then frozen before A2, A4 and D1–D3 were evaluated. The CPU floor itself was derived from clean data alone.

3.9.9 Dual-Gate Decision Logic

A single T^2 score breaching the UCL is not sufficient to emit a threat signal. Two gates must both be satisfied simultaneously:

Gate A: 6 of 8 persistence rule

At least 6 of the 8 most recent per-second T^2 scores must meet or exceed the UCL. This prevents single transient noise spikes from triggering the state machine and ensures that the anomaly is sustained rather than momentary.

Gate B: CPU rate gate

For a full history of $L = 60$ one-second samples, define the CPU exceedance count

$$N_{\text{CPU}}(t) = \sum_{j=t-L+1}^t 1[\text{CPU}(j) > 56.638 \text{ mCPU}]. \quad (3.25)$$

The CPU rate gate is satisfied only when

$$\frac{N_{\text{CPU}}(t)}{60} > r, \quad r = 0.15. \quad (3.26)$$

Because the comparison is strict, the minimum integer count that can satisfy this gate is

$$N_{\min} = \lfloor rL \rfloor + 1 = \lfloor 0.15 \times 60 \rfloor + 1 = 10. \quad (3.27)$$

Thus, exactly nine exceedances produce a rate of 0.15 and do not pass the gate; at least ten above-floor samples are required once the 60-sample history is full. This is a necessary CPU-corroboration condition, not a sufficient end-to-end detection condition: the 6-of-8 T^2 persistence gate must be true at the same time. The rate formulation targets duty-cycling strategies by counting individual above-floor observations rather than comparing only a windowed mean. A duty cycle strictly greater than 15% can satisfy the CPU gate, subject to sampling alignment, whereas an exactly 15% cycle cannot.

When both gates are satisfied, the detector emits a state transition signal to the ZT-XGuard policy engine.

Because Gate B is a *necessary* condition, the statistical route to **COMPROMISED** is explicitly CPU-corroborated. An exhaustion strategy that drives memory or network pressure without ever raising CPU above the floor can still accumulate T^2 evidence and satisfy the persistence gate, so it is surfaced as **SUSPICIOUS** for operator attention, but it does not auto-escalate to **COMPROMISED** through this channel. The suspicious-or-compromised true-positive rates reported in Section 4.6 therefore measure detection coverage of the attack window; they are not a claim that non-CPU pressure alone reaches full statistical confirmation. Extending the corroboration gate to additional resource dimensions is identified as future work.

3.10 Three-State Policy Engine and Escalation Logic

ZT-XGuard maintains exactly one of three trust states for every monitored xApp x :

$$T_x(t) \in \{\text{NORMAL}, \text{SUSPICIOUS}, \text{COMPROMISED}\}. \quad (3.28)$$

NORMAL indicates that no active evidence violates the xApp’s expected profile. **SUSPICIOUS** indicates warning-level behavioral evidence or partial statistical evidence that requires increased scrutiny but has not satisfied a compromise guard. **COMPROMISED** indicates that either a critical behavioral guard or the statistical detector’s internal dual-gate confirmation guard has been satisfied.

Isolation is not a trust state. It is a containment action applied after an xApp has entered **COMPROMISED**. Consequently, successful isolation does not introduce an **ISOLATED** state into the policy engine; the isolated workload remains classified as **COMPROMISED** for the scope of the incident.

3.10.1 Deterministic Transition Guards

To evaluate the trust state of an xApp, we define two levels of threat severity based on combined behavioral and statistical evidence:

- **Critical Threat (C):** A profile-matched critical behavioral violation or a confirmed statistical anomaly.
- **Warning Threat (W):** A warning-level behavioral event or an elevated statistical anomaly.

The next trust state of the xApp, $S(t+1)$, is determined by a top-down priority evaluation of its current state, $S(t)$, and the active threats:

$$S(t+1) = \begin{cases} \text{COMPROMISED,} & \text{if } S(t) = \text{COMPROMISED or } C \text{ is true} \\ \text{SUSPICIOUS,} & \text{if } W \text{ is true} \\ \text{NORMAL,} & \text{otherwise} \end{cases} \quad (3.29)$$

Equation 3.29 removes ambiguity from state transitions. Because it is evaluated sequentially, any critical threat immediately forces and locks the state to **COMPROMISED**. If no critical threat exists, warning-level evidence shifts the state to **SUSPICIOUS**. If the environment is completely clear, the xApp remains **NORMAL**. Cross-channel corroboration is not strictly required; a strong signal from either the behavioral or statistical channel is sufficient to trigger a transition.

Table 3.8: Formal trust-state transition table of the ZT-XGuard policy engine

From	Trigger (signal or event)	Guard	To	Timing / action
N	Benign signal (clean baseline, heartbeat)	Resource use profile-consistent	N	Monitor
Any	Out-of-scope event	Not a controlled-xApp event	IGNORED	Posture unchanged
N	Falco suspicious signal (package manager, binary drop, permission tamper, K8s API contact)	—	S	CPU throttle, evidence
N	Statistical elevated (<code>resource_anomaly_t2_elevated</code>)	$T^2 \geq \text{WL}$, dual gate not fully met	S	CPU throttle, evidence
N	Generic resource anomaly (high CPU or memory)	Not profile-consistent or heartbeat degraded	S	Evidence
N	Output drift or unexpected RIC or peer contact	Target not in profile allow-list	S	Evidence

From	Trigger (signal or event)	Guard	To	Timing / action
N	Identity or SVID violation	Observed before runtime (xApp not running)	S	Evidence
N / S	Critical Falco signal (shell, sensitive file, egress, SVID material, config tamper, token access, tool execution, container escape)	Egress not profile-allowed	C	IMMEDIATE
N / S	Identity or SVID violation	xApp running	C	IMMEDIATE
N / S	Statistical confirmed (<code>resource_anomaly_t2</code>)	Dual gate: $T^2 \geq$ UCL six-of-eight and CPU rate gate	C	DWELL_30S
S	Correlated suspicious evidence	Suspicious-context ≥ 2 (correlated signals or repeat threshold)	C	DWELL_30S
C	Isolation timing IMMEDIATE	—	ISOLATED	Four-lock, probe-verified
C	Dwell elapsed (30 s) or operator isolates	Still C	ISOLATED	Four-lock, probe-verified
S	Anomaly clears	$T^2 < \text{WL}$ sustained, no critical evidence	N	Throttle lifted
ISOLATED	Verified restore	Identity, digest, config, readiness, endpoints re-validate	N	Pod recreated from clean template

Note: N = Normal, S = Suspicious, C = Compromised.

An out-of-scope event may receive the internal verdict **IGNORED**, but **IGNORED** is an event-processing result rather than an xApp trust state. Therefore, it does not extend the three-state model.

For the scope of the containment workflow, **COMPROMISED** is treated as the terminal trust state. Returning the testbed to a clean condition between repeated experimental trials is an experimental reset procedure and is not modelled as a policy-engine state transition.

3.11 Automated Multi-Lock Containment and Verified Isolation

The containment orchestrator is implemented as a standalone Flask application. The `/alert` endpoint receives behavioral or statistical threat decisions, and endpoint `/csm/containment/apply` applies the four isolation locks to the affected

xApp. Isolation is a response action and does not add a fourth state to the three-state policy model. A critical behavioral decision initiates isolation immediately. A statistically confirmed decision starts the fixed 30-second operator-intervention dwell; the operator may isolate the xApp during this interval, and otherwise isolation begins automatically when the dwell expires.

Once isolation is initiated, both paths execute the same four-lock procedure and verify the result with an active reachability probe. The trigger changes the timing and the evidence that justified the response, but not the isolation mechanism itself.

Figure 3.11 shows the containment architecture and the order in which the orchestrator applies and verifies the four locks.

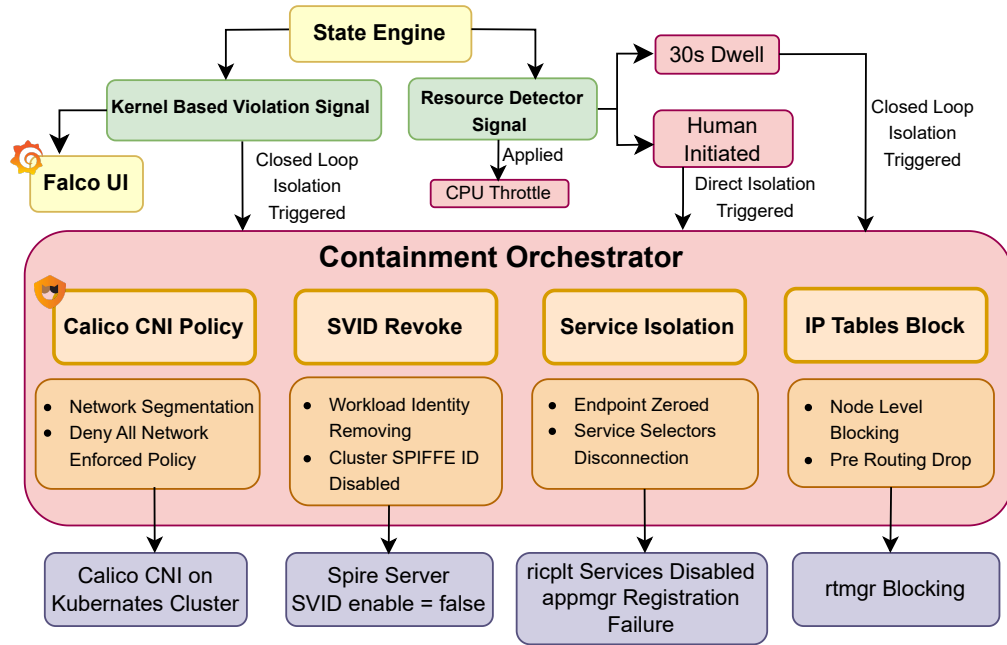


Figure 3.11: ZT-XGuard containment architecture.

On escalation to the compromised state, the orchestrator withdraws the SPIFFE identity binding to prevent renewal, applies the deny-all NetworkPolicy, isolates the service endpoints and installs the node-level packet-filter block, then confirms network unreachability with an active probe before recording the incident as contained.

Lock 1: Identity-Binding Withdrawal

In the runtime monitoring and containment testbed, the orchestrator patches the pod so it no longer carries the `svid-enabled` label that the `ClusterSPIFFEID`, `zt-xguard-ricxapp-identity` selects. The SPIRE controller consequently withdraws the rendered registration entry and stops renewing the pod's short-lived SVID (five-minute TTL). The certificate already issued to the xApp is not invalidated by this operation and remains cryptographically valid until its expiry; after that point the xApp can no longer present a valid workload identity to a service that verifies the SVID. The measured identity-lock latency is therefore the time to controller reconciliation and binding withdrawal, not the time to certificate expiry.

Immediate reachability isolation is supplied by Locks 2–4. Binding withdrawal is used instead of manual entry deletion so the controller cannot silently recreate an entry while the selector remains absent. This lock belongs to the runtime testbed and does not imply that the separate onboarding testbed’s JIT registration entry was carried into this deployment.

Lock 2: Network Micro-segmentation

A deny-all Kubernetes `NetworkPolicy` (`zt-xguard-quarantine-deny-all`) is applied to the pod together with a `zt-xguard.io/quarantine=true` label. The policy denies all ingress and egress and is enforced at the Calico/Felix CNI data plane, which programs the corresponding drop rules for the pod. Enforcement was confirmed directly with raw TCP-connect tests against the pod rather than assumed from the presence of the policy object. This severs the xApp’s reachability to the E2 termination, the RIC Shared Data Layer and any co-located platform service.

Lock 3: Service-Endpoint Isolation

The compromised xApp’s Kubernetes Service selector is swapped from its application label to a unique nonce label (`zt-xguard.io/service-isolated`), which empties the Service’s endpoint set. The original selector is preserved in an annotation in the incident evidence. This cuts reachability through the Service even for a peer that still holds a cached endpoint answer, closing a path that a network-policy lock alone can leave briefly open.

Lock 4: Direct Node-Level Network Block

Independently of the CNI policy path, the orchestrator installs drop rules for the pod’s address in a dedicated chain (`ZTX-DIRECT-QUARANTINE`) in the host node’s packet filter and confirms the rule is present on the node. Because this lock does not depend on the Calico pipeline, the two network locks (Lock 2 and Lock 4) enforce the same containment decision through independent mechanisms, so containment never rests on a single enforcement layer.

After the locks are applied, the orchestrator verifies that isolation actually took effect. It checks Service endpoint membership, network-policy selection, node-level rule presence and pod readiness, then runs an active TCP-connect probe from an uninvolved pod to the compromised xApp’s Service. Only when the probe confirms the xApp is unreachable is the incident recorded as contained. Containment is therefore measured by an active reachability test rather than inferred from an applied label.

An xApp raised only to the `SUSPICIOUS` state receives an earlier, lighter response before any isolation: a reversible cgroup CPU throttle that caps the container’s CPU and bounds resource abuse during the operator-intervention window, lifting automatically once the anomaly clears.

Forensic evidence extraction. At the moment an xApp is escalated to `COM-PROMISED`, the orchestrator captures the pod’s application logs, its full Kubernetes specification, the triggering signal and the incident timeline and writes them to a host directory outside the container’s mount namespace so the compromised pro-

cess cannot reach or tamper with them. This is forensic evidence extraction. It is not immutable storage: the current implementation does not use append-only or write-once media, does not hash or sign each artifact and does not replicate the evidence to off-node custody, so genuine immutability (append-only or WORM storage, per-artifact cryptographic hashing and signing and remote retention) remains future work.

A deliberate adversarial validation confirmed that a compromised xApp cannot defeat the network lock from inside the pod. A `kubect1 auth can-i` check executed with the `ricxapp` namespace default service account identity confirmed that this identity cannot delete the deny-all `NetworkPolicy` applied by the orchestrator, so even if the attacker steals the pod’s in-cluster service account token it cannot remove the network isolation from within the compromised environment.

3.12 Experimental Setup

The framework is validated across two distinct testbed configurations reflecting the two operational contexts in which its components operate. The secure onboarding and platform attestation were evaluated on the three-VM onboarding testbed and the runtime detection and containment were evaluated on the single-node NIST O-RAN testbed. We state plainly that these are two separate deployments: the onboarding-to-runtime pipeline was validated in two halves, not exercised end to end within one continuous deployment and unifying them is future work. Workload identity was instantiated independently in each environment: JIT Pod-UID registration was used by the final onboarding pipeline, while a label-selected `ClusterSPIFFEID` supplied the runtime testbed’s revocable identity binding.

3.12.1 Secure Onboarding Testbed

Table 3.9: Three-VM testbed configuration for secure onboarding evaluation

VM	Role	Key Components
Solution Provider	Third-party developer environment	Docker, GPG key pair, Cosign key pair, xApp descriptor, Dockerfile, image build and signing scripts
SMO	Operator management plane	GPG, Cosign, Helm, ChartMuseum, Keylime Verifier and Registrar, <code>secure-ingestion.sh</code>
Near-RT RIC	xApp runtime environment	Kubernetes (kubeadm), O-RAN SC RIC platform (<code>ricplt</code> namespace), SPIRE server and agents, cert-manager, private Docker registry, Keylime Agent, <code>registration.sh</code>

The secure onboarding and workload attestation evaluation uses three virtual machines running Ubuntu 20.04 LTS on VMware Workstation, connected over an

isolated bridged network. Each VM models one administrative domain. Table 3.9 describes the role and software stack of each.

3.12.2 Runtime Monitoring and Containment Testbed

The runtime monitoring, anomaly detection and containment evaluation uses the NIST O-RAN Automation Tool [29], a single-VM platform that integrates srsRAN, Open5GS and the O-RAN Software Community Near-RT RIC in a pre-configured environment. This platform provides a fully operational E2 interface between the srsRAN CU/DU and the Near-RT RIC, allowing the KPI monitoring xApp to receive genuine KPM indications from a simulated UE during all evaluation scenarios. The ZT-XGuard statistical anomaly detector runs as a sidecar process within the `ricplt` namespace, consuming telemetry from the KPI monitoring xApp’s `cAdvisor` metrics at 1 Hz. The containment orchestrator runs as a standalone Flask service on the same VM, listening for Falco webhook alerts and T^2 detector signals on port 16000.

Table 3.10: NIST O-RAN Automation Tool testbed configuration for runtime monitoring and containment evaluation

Component	Details
Platform	NIST O-RAN Automation Tool (single VM, VMware Workstation guest, Ubuntu 22.04 LTS, 16 GB RAM allocated)
RAN components	srsRAN CU/DU (5G, ZeroMQ virtual radio interface), Open5GS 5G core (AMF, SMF, UPF), srsUE
Near-RT RIC	O-RAN Software Community Near-RT RIC (Kubernetes, <code>ricplt</code> namespace); KPI monitoring xApp subscribed to E2SM-KPM indications
Security components	SPIRE with a label-selected <code>ClusterSPIFFEID</code> and <code>spiffe-helper</code> sidecar, Falco (Helm deployment, custom RIC rules, Falcosidekick webhook output), <code>cAdvisor</code> + Prometheus (1 Hz metric scraping), ZT-XGuard anomaly detector (Python sidecar), ZT-XGuard containment orchestrator (Flask, port 16000)
Attack tooling	stress-ng [48] (CPU and memory stressors for A1, A2, A3 and A4 scenarios), hping3 (network flood for network-layer scenarios)

The E2 interface and KPM indication flow are maintained continuously throughout all evaluation scenarios. This is a methodological requirement: the anomaly detector’s baseline model was calibrated with the xApp actively processing KPM indications and disconnecting the E2 interface during attack scenarios would introduce a confounding change in the xApp’s resource utilization profile that is unrelated to the attack itself.

Chapter 4

Results

This chapter presents the experimental results obtained from implementing and evaluating ZT-XGuard. It reports the infrastructure and onboarding results, the behavioral attack trials, the statistical detector’s clean, development and held-out datasets and the deployed containment trials. The final containment checks include an adversarial authorization test showing that the

4.1 Deployment of srsRAN and 5G Core

A fully operational end-to-end private 5G network was deployed to provide the radio environment for the security framework. The deployment consisted of the srsRAN project’s software RAN components, the Open5GS 5G core and a simulated user equipment, srsUE from the srsRAN 4G project that is compatible with the 5G network. All elements were installed in virtual machines running Ubuntu 20.04 LTS to ensure the stability and reproducibility of the testbed.

4.1.1 Deployed Components

- **5G Core:** The Open5GS 5G core was deployed as a set of Docker containers using Docker Compose. It provides the essential control-plane and user-plane network functions, including the Access and Mobility Management Function (AMF), Session Management Function (SMF) and User Plane Function (UPF). The core is responsible for subscriber authentication, session management and routing of user data between the gNB and external data networks.
- **gNodeB (gNB):** The gNB was deployed using the srsRAN 5G Project, configured as a disaggregated Central Unit (CU) and Distributed Unit (DU) pair. The CU terminates the F1-C interface towards the DU and establishes control plane connectivity with the 5G core over the N2 interface. The DU handles the lower-layer radio processing and communicates with the CU over the F1-C interface and the combined CU-DU pair forms the O-RAN-compatible gNB.
- **User Equipment:** A User Equipment (UE) was simulated using the srsRAN

4G project’s UE implementation (srsUE), which has been updated to support 5G networks. The srsUE attaches to the gNB after completing the random-access procedure and is assigned an IP address from the data network. This simulated UE generates realistic control-plane and user-plane traffic, enabling end-to-end validation of the radio link.

- **Virtual Radio:** The communication between the srsUE and the gNB is established using a ZeroMQ-based virtual radio interface, which replaces physical RF hardware with a software-based signal path. This eliminates the need for dedicated radio equipment and allows the entire RAN to run on a single host or across multiple virtual machines.

4.2 Near-RT RIC Deployment

4.2.1 Kubernetes-Based Near-RT RIC

The production-oriented deployment of the Near-RT RIC was performed on a kubeadm-based Kubernetes cluster using the it/dep repository from the O-RAN Software Community. This deployment places all RIC platform components into a namespace (ricplt), where each component, including the E2 manager, E2 termination, A1 mediator, AppMgr and O1 mediator runs as an independently managed Kubernetes pod. The use of Kubernetes enables and simplifies advanced security features such as SPIRE workload attestation, CSI driver integration, xApp quarantine and isolation mechanisms, which are essential for the project’s threat-containment objectives.

```
deshanric@deshanric-virtual-machine:~$ kubectl get pods -n ricplt
```

NAME	READY	STATUS	RESTARTS	AGE
deployment-ricplt-almediator-69dc7b8c94-66gkk	1/1	Running	24 (8h ago)	24h
deployment-ricplt-alarmanager-cbb797dfd-nt8bc	1/1	Running	0	24h
deployment-ricplt-appmgr-779896945f-rj6sc	1/1	Running	0	24h
deployment-ricplt-e2mgr-76799dc754-fbkg9	1/1	Running	10 (8h ago)	8h
deployment-ricplt-e2term-alpha-587b8bccb6-xzsmn	1/1	Running	8 (24h ago)	24h
deployment-ricplt-olmediator-59d669865-sqb74	1/1	Running	0	24h
deployment-ricplt-rtmgr-78586b8c6d-m6dct	1/1	Running	9 (8h ago)	8h
deployment-ricplt-submgr-569f9c9575-mwgdr	1/1	Running	8 (8h ago)	8h
deployment-ricplt-vespamgr-8d49dd658-792zz	1/1	Running	5 (8h ago)	24h
deployment-ricplt-xapp-onboarder-78c4698978-nkkch	2/2	Running	0	7h42m
r4-infrastructure-kong-5f597575bc-lnnpd	2/2	Running	0	8h
r4-infrastructure-prometheus-alertmanager-6755c5b76d-6829b	2/2	Running	0	24h
r4-infrastructure-prometheus-server-8676b5854f-6v4vc	1/1	Running	9 (8h ago)	24h
statefulset-ricplt-dbaas-server-0	1/1	Running	0	24h

Figure 4.1: Kubernetes-based Near-RT RIC deployment: running pods in the ricplt namespace

4.3 Hardware-Rooted Platform Attestation

The Keylime remote attestation framework was deployed in the testbed to establish a hardware-rooted trust anchor for the Near-RT RIC and srsRAN nodes before any xApp was scheduled. Virtual TPM 2.0 modules were configured within the VMware Workstation environment for each monitored VM, providing the same PCR measurement and quote-generation capabilities as a physical TPM chip.

4.3.1 Component Deployment and Registration

The four Keylime components, Agent, Registrar, Verifier and Tenant were deployed across the testbed. The Agent on the srsRAN node registered its Attestation Identity Key with the Registrar, establishing the node's hardware identity in the attestation database.

```
dilshan@dilshan-virtual-machine:~$ sudo systemctl status keylime_agent
[sudo] password for dilshan:
● keylime_agent.service - Keylime Agent Service
   Loaded: loaded (/etc/systemd/system/keylime_agent.service; enabled; vendor preset: enabled)
   Active: active (running) since Sat 2026-05-02 20:53:53 +0530; 5min ago
     Main PID: 3675 (keylime_agent)
       Tasks: 4 (limit: 4540)
      Memory: 6.8M
         CPU: 7.405s
       CGroup: /system.slice/keylime_agent.service
              └─3675 /usr/bin/keylime_agent

○ 02 20:53:53 dilshan-virtual-machine systemd[1]: Started Keylime Agent Service.
```

Figure 4.2: Keylime agent startup and registration log on the srsRAN node, confirming successful AIK binding to the registrar

```
kavinda@kavinda-virtual-machine:~$ sudo keylime_registrar
[sudo] password for kavinda:
INFO:keylime.config:Reading configuration from ['/etc/keylime/logging.conf']
2026-05-02 20:13:16.969 - keylime.registrar - INFO - Starting Keylime registrar...
2026-05-02 20:13:16.969 - keylime.registrar - INFO - Current API version: 2.5
2026-05-02 20:13:16.969 - keylime.registrar - INFO - Other supported API versions: 1.0, 2.0, 2.1, 2.2, 2.3, 2.4, 3.0
2026-05-02 20:13:16.969 - keylime.registrar - INFO - Deprecated API versions (soon to be removed): 1.0
2026-05-02 20:13:16.974 - keylime.config - INFO - Reading configuration from ['/etc/keylime/registrar.conf']
2026-05-02 20:13:16.983 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:13:16.983 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:13:16.989 - alembic.env - INFO - Migrating database registrar
2026-05-02 20:13:16.990 - alembic.runtime.migration - INFO - Context impl SQLiteImpl.
2026-05-02 20:13:16.990 - alembic.runtime.migration - INFO - Will assume non-transactional DDL.
2026-05-02 20:13:16.998 - keylime.registrar - INFO - DevID is enabled in config ('tpm_identity = default') but python-cryptography version < 38, so only the
ER will be used for device registration
2026-05-02 20:13:16.998 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:13:16.998 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:13:16.999 - keylime.shared_data - INFO - Initializing global shared memory manager
2026-05-02 20:13:17.023 - keylime.shared_data - INFO - SharedDataManager initialized successfully
2026-05-02 20:13:17.023 - keylime.shared_data - INFO - Global shared memory manager initialized
2026-05-02 20:13:17.029 - keylime.web - INFO - Listening on 0.0.0.0:8890/8891 (HTTP/S) with 4 worker processes...
2026-05-02 20:13:17.037 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:13:17.037 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:13:17.039 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:13:17.040 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:13:17.040 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:13:17.041 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:13:17.042 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:13:17.044 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
```

Figure 4.3: Keylime registrar log confirming agent registration and secure storage of the hardware identity database entry

```

kavinda@kavinda-virtual-machine:~$ sudo keylime_verifier
[sudo] password for kavinda:
INFO:keylime.config:Reading configuration from ['/etc/keylime/logging.conf']
2026-05-02 20:12:03.360 - keylime.config - INFO - Reading configuration from ['/etc/keylime/verifier.conf']
2026-05-02 20:12:03.361 - keylime.verifier - INFO - Starting Keylime verifier in PULL mode...
2026-05-02 20:12:03.361 - keylime.verifier - INFO - Current API version: 2.5
2026-05-02 20:12:03.361 - keylime.verifier - INFO - Other supported API versions: 1.0, 2.0, 2.1, 2.2, 2.3, 2.4, 3.0
2026-05-02 20:12:03.361 - keylime.verifier - INFO - Deprecated API versions (soon to be removed): 1.0
2026-05-02 20:12:03.373 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:12:03.373 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:12:03.386 - alembic.env - INFO - Migrating database cloud_verifier
2026-05-02 20:12:03.386 - alembic.runtime.migration - INFO - Context impl SQLiteImpl.
2026-05-02 20:12:03.386 - alembic.runtime.migration - INFO - Will assume non-transactional DDL.
2026-05-02 20:12:03.428 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:12:03.428 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:12:03.429 - keylime.verifier - INFO - Initializing shared memory manager in main process before server creation
2026-05-02 20:12:03.429 - keylime.shared_data - INFO - Initializing global shared memory manager
2026-05-02 20:12:03.453 - keylime.shared_data - INFO - SharedDataManager initialized successfully
2026-05-02 20:12:03.454 - keylime.shared_data - INFO - Global shared memory manager initialized
2026-05-02 20:12:03.457 - keylime.config - INFO - Reading configuration from ['/etc/keylime/ca.conf']
2026-05-02 20:12:03.458 - keylime.ca-util - WARNING - Using 'default' password option from CA configuration file
2026-05-02 20:12:03.472 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:12:03.472 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:12:03.521 - keylime.verifier - INFO - Agent ids in db loaded from file: ['e70584fd-cb91-4c83-9587-baac3a265923']
2026-05-02 20:12:03.529 - keylime.verifier - INFO - start_multi() called with operating_mode: pull
2026-05-02 20:12:03.530 - keylime.verifier - INFO - Querying agents for verifier_id: default
2026-05-02 20:12:03.531 - keylime.failure - INFO - Severity configuration set
2026-05-02 20:12:03.531 - keylime.keylime_db - INFO - database_url is set, using it to establish database connection
2026-05-02 20:12:03.531 - keylime.keylime_db - INFO - database_url is set as 'sqlite' keyword, using default values to establish database connection
2026-05-02 20:12:03.536 - keylime.verifier - INFO - Found 1 agents in database before forking
2026-05-02 20:12:03.536 - keylime.verifier - INFO - Listening on 0.0.0.0:8881 (HTTPS) with 4 worker processes...
2026-05-02 20:12:03.542 - keylime.verifier - INFO - Worker 2 assigned 0 agent(s)
2026-05-02 20:12:03.547 - keylime.verifier - INFO - Worker 0 assigned 1 agent(s)
2026-05-02 20:12:03.552 - keylime.verifier - INFO - Worker 1 assigned 0 agent(s)
2026-05-02 20:12:03.553 - keylime.verifier - INFO - Worker 3 assigned 0 agent(s)
2026-05-02 20:12:03.562 - keylime.verifier - INFO - Activating 1 agent(s) for PULL mode
2026-05-02 20:12:03.564 - keylime.verifier - INFO - Using default client_cert option for verifier
2026-05-02 20:12:03.565 - keylime.verifier - INFO - Using default client_key option for verifier
2026-05-02 20:12:03.565 - keylime.verifier - INFO - No value provided in client_key_password option for verifier, assuming the key is unencrypted

```

Figure 4.4: Keylime verifier continuous monitoring loop, issuing nonce-based TPM quote challenges and logging periodic node integrity checks

4.3.2 Trusted Provisioning and Integrity Verification

The Keylime Tenant was used to upload the Reference Integrity Manifest and initiate the attestation lifecycle. The Verifier challenged the Agent with a fresh nonce and received the current PCR values signed by the vTPM. Successful comparison of the signed quote against the stored RIM resulted in a PASS status on the monitoring dashboard, confirming that the node's software stack matched its trusted provisioning baseline.

```

dilshan@dilshan-virtual-machine:~$ sudo journalctl -u keylime_agent --no-pager | grep "Agent UUID" -A 5
@ 22 13:02:25 dilshan-virtual-machine keylime_agent[3910]: INFO keylime_agent > Agent UUID: d432fbb3-d2f1-4a97-9ef7-75bd81c00000
@ 22 13:02:25 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Building Registrar client: scheme=http, registrar=172.20.10.5:8890, TLS=false
@ 22 13:02:25 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Requesting registrar API version to http://172.20.10.5:8890/version
@ 22 13:02:25 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Requesting agent registration from http://172.20.10.5:8890/v2.5/agents/d432fbb3-d2f1-4a97-9ef7-75bd81c00000
@ 22 13:02:25 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:agent_registration > SUCCESS: Agent d432fbb3-d2f1-4a97-9ef7-75bd81c00000 registered
@ 22 13:02:26 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Requesting agent activation from http://172.20.10.5:8890/v2.5/agents/d432fbb3-d2f1-4a97-9ef7-75bd81c00000
@ 22 13:02:29 dilshan-virtual-machine keylime_agent[3910]: INFO keylime_agent > Agent UUID: d432fbb3-d2f1-4a97-9ef7-75bd81c00000
@ 22 13:02:29 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Building Registrar client: scheme=http, registrar=172.20.10.5:8890, TLS=false
@ 22 13:02:29 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Requesting registrar API version to http://172.20.10.5:8890/version
@ 22 13:02:29 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Requesting agent registration from http://172.20.10.5:8890/v2.5/agents/d432fbb3-d2f1-4a97-9ef7-75bd81c00000
@ 22 13:02:29 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:agent_registration > SUCCESS: Agent d432fbb3-d2f1-4a97-9ef7-75bd81c00000 registered
@ 22 13:02:30 dilshan-virtual-machine keylime_agent[3910]: INFO keylime:registrar_client > Requesting agent activation from http://172.20.10.5:8890/v2.5/agents/d432fbb3-d2f1-4a97-9ef7-75bd81c00000

```

Figure 4.5: Successful integrity handshake.

```
dilshan@dilshan-virtual-machine:~$ sudo tpm2_pcrread
sha1:
sha256:
0 : 0x59C18B70C713AAF005E02461AAE16782F820EA964EA6AACFA7089718586751E
1 : 0x3D458CFE55CC03E1F443F1562BEEC8DF51C75E14A9FCF9A7234A13F198E7969
2 : 0x3D458CFE55CC03E1F443F1562BEEC8DF51C75E14A9FCF9A7234A13F198E7969
3 : 0x3D458CFE55CC03E1F443F1562BEEC8DF51C75E14A9FCF9A7234A13F198E7969
4 : 0xEC053FA26E1D2BF5366044D573A89B97CB411378E4E2CF0304EBF0C48322A14
5 : 0x928075B3D080E9F0D8E25DAA40568AE71B73D0169493024E901F61C7D0921C4F4
6 : 0x3D458CFE55CC03E1F443F1562BEEC8DF51C75E14A9FCF9A7234A13F198E7969
7 : 0x9CF86A2557266A20703BA993B2E7819891809349FC86FDE98B3476B29CA4837D
8 : 0xE7803B8517ECE41C1360A73431D0583229C372031D3B3CEB0A9349A28BE5C641
9 : 0xCAF1B5C326FD0DAF71A0CCA2F1423D075669D6AC52907D16B4982E3E733D207
10 : 0xFD036E44D9464AC3D83DC06C355CF973D3DA583B36683F087293E1C95291C08
11 : 0x0000000000000000000000000000000000000000000000000000000000000000
12 : 0x0000000000000000000000000000000000000000000000000000000000000000
13 : 0x0000000000000000000000000000000000000000000000000000000000000000
14 : 0x306F9D8B94F17D93DC6E7CF8F5C79D652EB4C64D13DE2DDC24AF416E13ECA
15 : 0x0000000000000000000000000000000000000000000000000000000000000000
16 : 0x33DF53F3B75FAD52F6789B104B1FF9F08D5D1685FCEA4F7EB41D7EEA9164769
17 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
18 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
19 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
20 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
21 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
22 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
23 : 0x0000000000000000000000000000000000000000000000000000000000000000
sha384:
0 : 0x38F489A82D4DCDD0F87C75984981CFA2E0F8C8ABFD88BF0CEB048C80F3F7FC878688A06901575A50775DA79534A5C186
1 : 0x51892380F955080DA077C6AABA52280DECEDE61C599CEA6C41889CFBEA4AE4D50529D96FE4D1AFDAFB65E7F958F23C4
2 : 0x51892380F955080DA077C6AABA52280DECEDE61C599CEA6C41889CFBEA4AE4D50529D96FE4D1AFDAFB65E7F958F23C4
3 : 0x51892380F955080DA077C6AABA52280DECEDE61C599CEA6C41889CFBEA4AE4D50529D96FE4D1AFDAFB65E7F958F23C4
4 : 0x68FD38690D25985188628258082B2A2748B2E82A5E52887704852C38279AB30E6AC431735CCF3689480208043AEC
5 : 0x490FA4223CA471E194C271580A46CA833216999AC19CF1C628961193F1D414022D3CCA5C15F437198CB19E61EC8AB8
6 : 0x51892380F955080DA077C6AABA52280DECEDE61C599CEA6C41889CFBEA4AE4D50529D96FE4D1AFDAFB65E7F958F23C4
7 : 0xA715FC8AC878F4A8070F73AC3681C85714EC5FF263AFED3C4AA44F342D21B348868CD24C22D99C179040A1CD4D04D0
8 : 0x8658568E270F83CEC8CD2749875CE7A832957F7F85096859624A36753F88E038B97F187731A547408FA2BF0D15A5
9 : 0x8480D3538CAE290FC5890054005640D64D05181CE712F842E6395D0C3C097F3CECE498F772F35BCE0D19C6EC3A65320A
10 : 0xD094CF9B2AE1536F1391E972A88EF85E4008081231DECCD509B6A2823E989FD8288AF72464BE45C7744B083611CE274
11 : 0x0000000000000000000000000000000000000000000000000000000000000000
12 : 0x0000000000000000000000000000000000000000000000000000000000000000
13 : 0x0000000000000000000000000000000000000000000000000000000000000000
14 : 0x9374370F279810015F4598395CF8D20E2F36E08E3897BBA89874BF612B5DA092BADEF37F79714A60193819E384AD
15 : 0x0000000000000000000000000000000000000000000000000000000000000000
16 : 0x0000000000000000000000000000000000000000000000000000000000000000
17 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
18 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
19 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
20 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
21 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
22 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
23 : 0x0000000000000000000000000000000000000000000000000000000000000000
sha512:
0 : 0xCEBAF4C1F8041EA6BAFFD8D68F448D684E17DC83CDEB328BF33A91DA12A9D81A8BF40D7D5C34F732A323AB7FC146420C746080E782EDCC3AEBB4B2EC645
1 : 0x27EC091533C489EEA38DD14C3A3ECDEF8A99C1E564CE66DF008250154E7839B0875228FE80E8CC4C330E6AEB1A8C740708C9C9C1E268939C9D916E4513C
2 : 0x27EC091533C489EEA38DD14C3A3ECDEF8A99C1E564CE66DF008250154E7839B0875228FE80E8CC4C330E6AEB1A8C740708C9C9C1E268939C9D916E4513C
3 : 0x27EC091533C489EEA38DD14C3A3ECDEF8A99C1E564CE66DF008250154E7839B0875228FE80E8CC4C330E6AEB1A8C740708C9C9C1E268939C9D916E4513C
4 : 0x759090E2F3B2A5C232AA8EB232668D3F6D22CA0764F4D13E91C706A61498F93EFD27828463872411760C6FC8F8583E72D1768670F853235F842271DC83D0101
5 : 0x9F084E83D0F96527D50A654C7C2F03FED089F50525126E68B9D065FD368C0C344201931ACD6F1F035368488EB66A097A30FD8AC2D1985567BE35D80A6A84F9
6 : 0x27EC091533C489EEA38DD14C3A3ECDEF8A99C1E564CE66DF008250154E7839B0875228FE80E8CC4C330E6AEB1A8C740708C9C9C1E268939C9D916E4513C
7 : 0x7C9AC78055835C2660F86D966C35D03CEFA82D987C984FAEFBE6ACEA45EA41EF18A8695FAB95408AECE02A41AE745C6BEF61CC5FA4EC16C12A8E0F8EDF904D8E
8 : 0x95B110F0865A7A434670A20CA2048265D3735E91B5F67789D8F5E2EFF324D4F1F80A0F449104ED9F8ACD109AA9698796771A3FCD22D7DD174E04CDF8B63D087
9 : 0x16EE1411C6F90BF0E9808588B732D04483DC846C3594AD5CA1B618BA3A094F9C00C6812C60C2C6169690839C99E8BEA154311E67C7947341FE9784587CB243
10 : 0x94425E3FEC3D058AB7B0A3892E0529A07F6CA701D0C43546F02FD06C1E2B3DF87F11063C418A17FECF4830655575AA7B1A1724EBC4E21ED3D801384112AFA6
11 : 0x0000000000000000000000000000000000000000000000000000000000000000
12 : 0x0000000000000000000000000000000000000000000000000000000000000000
13 : 0x0000000000000000000000000000000000000000000000000000000000000000
14 : 0x397642E276D1E613AF382297484400EB4F84B18730860E2054E041D31E3724C12B1A62066C2C454355A954D52F82F2E2A26133F571947E0F78FF265
15 : 0x0000000000000000000000000000000000000000000000000000000000000000
16 : 0x0000000000000000000000000000000000000000000000000000000000000000
17 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
18 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
19 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
20 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
21 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
22 : 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
23 : 0x0000000000000000000000000000000000000000000000000000000000000000
```

Figure 4.6: Raw PCR measurements captured from the agent node's vTPM.

```
kavinda@kavinda-virtual-machine:~$ python3 ~/Desktop/FYP/status_dashboard.py
O-RAN Security Monitor
```

Resource	Value
Agent ID	e70584fd-cb91-4c83-9587-baac3a265923
O-DU IP	172.20.10.4
Status	PASS
Last Update	20:56:26

Figure 4.7: PCR-quote verification success confirmation.

4.3.3 Integrity Violation Detection

To validate the attestation path, a deliberate integrity change was injected on the agent node. A script extended PCR 23 with an unauthorized measurement value, which alters the node's reported platform-integrity state in the same way that an unexpected change to a measured component would.

```
agent@agent-virtual-machine:~/Desktop/Attack_Sim$ ./run_attack.sh
[!] ATTACK STARTING: Injecting Malicious xApp Logic...
insmod: ERROR: could not load module fake_rootkit.ko: No such file or directory
[+] Malicious module 'fake_rootkit' successfully inserted.
[!] Corrupting TPM PCR 23 (Runtime Integrity)...
[***] ATTACK COMPLETE. Check the O-RAN Dashboard on the RIC!
```

Figure 4.8: PCR manipulation on the agent node, extending PCR 23 with an unauthorized value.

```
kavinda@kavinda-virtual-machine:~$ python3 ~/Desktop/FYP/status_dashboard.py
O-RAN Security Monitor
```

Resource	Value
Agent ID	d432fbb3-d2f1-4a97-9ef7-75bd81c00000
O-DU IP	172.20.10.4
Status	FAIL
Last Update	09:53:31

Figure 4.9: Transitioning to FAIL status immediately following the PCR 23 manipulation.

The Keylime verifier compared the node's next signed TPM quote against the stored reference manifest, detected the PCR deviation on its next scheduled challenge cycle and immediately transitioned the node's attestation status to FAIL, after which secure payload delivery was automatically suspended. This demonstrates that a change to the node's measured boot and load-time integrity state is detected and acted on and that sensitive configuration material is withheld from a node whose measured integrity no longer matches its trusted baseline.

The scope of this result should be stated precisely. It shows detection of a deviation in the platform's measured-integrity state, not detection of every runtime compromise. Attacks that leave the measured boot chain unchanged, such as an in-memory-only exploit, a userspace rootkit loaded after the measured window or a hardware-level modification, fall outside what measured-boot attestation with static PCRs covers. Extending this to runtime integrity measurement, for example an IMA-based measured runtime, is future work.

4.3.4 System Admission Overhead

To evaluate the performance impact of the hardware-rooted trust anchor, system admission latency was measured across varying continuous attestation frequencies (Figure 4.10). Without attestation, baseline admission is nearly instantaneous at

0.0013459. Introducing the remote attestation pipeline adds a fixed cryptographic overhead for node registration and TPM verification, increasing latency to 1.2662 s under a 30 second polling interval.

Notably, increasing the polling frequency from 10 second down to 1 second intervals introduces minimal additional overhead, plateauing at a maximum of 1.3048 s. This demonstrates that the initial security tax is a fixed cost, allowing the system to enforce highly aggressive integrity checks without continuously degrading admission performance.

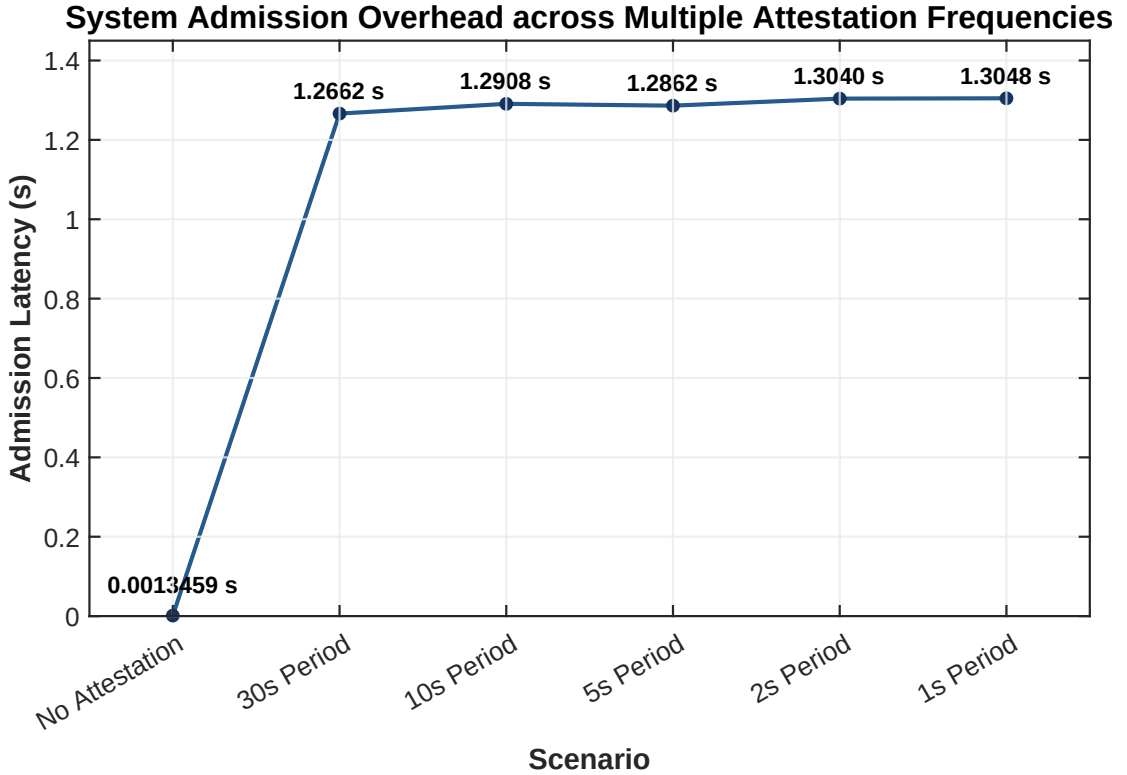


Figure 4.10: System admission overhead comparing the native baseline against continuous attestation polling intervals.

4.4 Secure Onboarding and Workload Identity Attestation

The secure onboarding pipeline was evaluated on the three-VM testbed described in Section 3.12. Two evaluation scenarios were executed: a legitimate onboarding run in which a correctly signed xApp package traverses all four gates and receives a SPIFFE workload identity, and a man-in-the-middle simulation in which a tampered package is injected into the pipeline to validate the integrity rejection mechanism.

4.4.1 Infrastructure Bootstrapping

Before any xApp deployment, the Kubernetes security infrastructure was established. `cert-manager` was deployed and configured with the `spire-ca-issuer`

ClusterIssuer, which serves as the upstream certificate authority for the entire SPIRE trust domain. The SPIRE server was started, the agent was deployed as a DaemonSet and the SPIFFE CSI driver was registered with the kubelet.

```

Near_RT RIC
agent@agent-virtual-machine:~/Desktop/ric/scripts$ kubectl get pods -n cert-manager -w
NAME                                READY   STATUS    RESTARTS   AGE
cert-manager-9f74c854d-kfqvm        1/1     Running   0           5m40s
cert-manager-cainjector-665cd78979-kg14f  1/1     Running   0           5m40s
cert-manager-webhook-65767c6f65-xbvn9  1/1     Running   0           5m40s

```

Figure 4.11: cert-manager pods running in the cert-manager namespace, confirming that the upstream CA for the SPIRE trust domain is operational

```

deshanric@deshanric-virtual-machine:~$ kubectl get pods -n spire-system
NAME                                READY   STATUS    RESTARTS   AGE
spiffe-csi-driver-fs26l            2/2     Running   18 (8h ago)  24h
spire-agent-9j7gf                  1/1     Running   3 (7h58m ago)  24h
spire-server-0                     2/2     Running   2 (165m ago)  6h51m
spire-spiffe-oidc-discovery-provider-68d6d99b79-mm5cw  2/2     Running   0           24h

```

Figure 4.12: SPIRE server, agents and CSI driver pods running in the spire-system namespace

Node attestation was performed automatically as each SPIRE agent started: the agent sent a Kubernetes Projected Service Account Token to the SPIRE server, which validated it via the TokenReview API, generated a CSR for the agent's own SVID and forwarded it to cert-manager for signing. The agent then established a secure mTLS channel to the server over the resulting SVID.

```

=====
O-RAN ZERO-TRUST: NODE ATTESTATION VERIFICATION
=====
[*] Step 1: Checking SPIRE Agent Logs (Client-Side Verification)...
[sudo] password for deshanric:
[+] SUCCESS: SPIRE Agent successfully attested to the Server!

[*] Step 2: Querying SPIRE Server Database (Server-Side Verification)...
[+] SUCCESS: SPIRE Server confirms 1 active agent connection(s)!

--- Cryptographic Identity Details ---
▶ SPIFFE ID       : spiffe://oran.fyp.local/spire/agent/k8s_psat/oran-ric-cluster/10aaf6cc-a717-4195-a203-5e00fc7c5835
  Attestation type : k8s_psat
  Expiration time  : 2026-05-01 14:10:19 +0000 UTC
=====

```

Figure 4.13: SPIRE node attestation

These logs confirm that the agent on the Near-RT RIC node successfully proved its node identity to the SPIRE server using a Kubernetes PSAT token.

4.4.2 Initial Selector-Based Onboarding Validation

This first functional onboarding validation used a SPIRE registration entry with namespace, service-account and pod-label selectors and an init container that fetched the issued SVID into a shared volume. It established that a package passing the cryptographic gates could be deployed and attested successfully in the three-VM onboarding testbed. It was not the identity arrangement used in the separate runtime containment testbed.

The Solution Provider built the xApp container image, pushed it to an external provider-managed Docker registry, signed its SHA-256 digest with Cosign in offline bundle mode, archived the descriptor files, and signed the archive with GPG. The signed tarball and its .asc signature were securely transferred to the SMO.

```
solutionprovider@ubuntu:~/Desktop/dos_xapp_package$ scp ~/Desktop/dos_xapp_package/dos-detector-d
descriptor.tar.gz* janindu@192.168.8.142:~/Desktop/vault/incoming/
janindu@192.168.8.142's password:
dos-detector-descriptor.tar.gz                                100% 662   132.7KB/s   00:00
dos-detector-descriptor.tar.gz.asc                            100% 862   155.7KB/s   00:00
```

Figure 4.14: Secure transfer of the signed xApp descriptor tarball and its GPG detached signature from solution provider to SMO

For this initial selector-based validation, the SMO executed `secure-ingestion.sh`. All four gates passed: the GPG signature verified correctly, the descriptor was parsed successfully, the Cosign bundle verified the container image in the external registry (authorizing its sync to the local RIC registry), and the Helm chart was uploaded to ChartMuseum alongside the SPIRE registration entry binding the SPIFFE ID `spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector` to the three Kubernetes selectors.

```

SMO
=====
ZERO-TRUST O-RAN INGESTION PIPELINE DASHBOARD
=====
▶ Provider ID      | sp1
▶ Payload          | dos-detector-descriptor.tar.gz
▶ Target RIC       | 192.168.8.135
=====

[ GATE 1 ] ORIGIN SIGNATURE VERIFICATION (GPG)
=====
└ Public Key      | keys/sp1_gpg.pub
✓ Status          | [ PASSED ] Origin verified. Payload is authentic.
=====

[ GATE 2 ] PAYLOAD EXTRACTION & DESCRIPTOR PARSING
=====
└ Workspace       | quarantine/ directory populated
└ xApp Name       | my-secure-dos-xapp
└ Chart Name      | dos-detector
└ Target Image    | 192.168.8.135:5000/dos-detector:1.0.1
✓ Status          | [ PASSED ] Architectural descriptors parsed.
=====

[ GATE 3 ] EDGE REGISTRY IMAGE INTEGRITY (COSIGN)
=====
└ Registry        | 192.168.8.135:5000
└ Cosign Key      | keys/sp1_cosign.pub
✓ Status          | [ PASSED ] Image cryptographically verified on Edge.
=====

[ GATE 4 ] PARALLEL ORCHESTRATION (HELM VAULT + SPIRE)
=====
└ Agent ID        | spiffe://oran.fyp.local/spire/agent/k8s_psat/k8s_psat/75
                  | dc65f3-1d27-4f06-8087-4c209a887f09
✓ Helm Vault      | [ SEALED ] Chart 'dos-detector' locked (HTTP 201)
✓ SPIRE Identity  | [ REGISTERED ] SPIFFE ID issued for 'dos-detector'
=====

[ GATE 5 ] VERIFIED XAPP READY FOR RIC DEPLOYMENT
=====
└ Chart Name      | dos-detector
└ Version         | 1.0.1
└ Helm Repo       | http://192.168.8.105:18080
└ Namespace       | ricxapp
└ RIC VM          | 192.168.8.135
=====

[ FINALIZATION ] PIPELINE CLEANUP & VERIFICATION
=====
└ Task            | Fetching verified SPIFFE Identity from Server...
└ Entry ID        | f63ca0ac-0391-4e73-8657-91daa4e16e5e
└ SPIFFE ID       | spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector
└ Parent ID       | spiffe://oran.fyp.local/spire/agent/k8s_psat/k8s_psat/75
                  | dc65f3-1d27-4f06-8087-4c209a887f09
└ Selectors       | k8s:ns:ricxapp,k8s:pod-label:app:dos-detector k8s:sa:dos
                  | -detector
└ SVID TTL        | 3600s
=====

[ ✓ SUCCESS ] xApp Trusted & Ready to Deploy
=====

```

Figure 4.15: Successful execution of the initial selector-based SMO secure ingestion pipeline

The RIC operator then executed `registration.sh`. When the xApp pod started, the init container called `spire-agent api fetch x509`. The SPIRE agent con-

firmed that the pod's namespace, service account and pod label matched the registration entry selectors, attested the workload successfully and delivered the X.509 SVID to the shared volume at /svid-data. The main xApp container was then able to use this SVID for authenticated communication with AppMgr and other RIC platform services.

```

=====
NEAR-RT RIC: xAPP DEPLOYMENT & ATTESTATION DASHBOARD
=====
▶ xApp Name      | dos-detector
▶ Version        | 1.0.1
▶ Repo           | http://192.168.8.105:18080
=====

[ STEP 1 ] CONNECTING TO SMO CHARTMUSEUM & SERVICEACCOUNT VERIFICATION
=====
↳ Repo URL      | http://192.168.8.105:18080
✓ Status        | [ PASSED ] Successfully synced with SMO repository.
✓ Status        | [ PASSED ] ServiceAccount is already owned by Helm.
=====

[ STEP 2 ] INSTANTIATING XAPP VIA HELM & DEPLOYMENT ROLLOUT
=====
↳ Helm          | [ UPGRADED ] Existing xApp updated to v1.0.1.
↳ Active Pod    | dos-detector-9464c47c6-pw24t
↳ ServiceAccount | dos-detector
✓ SA Match      | [ PASSED ] Container is using the correct ServiceAccount
=====

[ STEP 3 ] ZERO-TRUST WORKLOAD ATTESTATION
=====
↳ Layer A      | SPIRE server entry check...
✓ Status       | [ PASS ] Registration entry found on SPIRE server (2 total).
↳ SPIFFE ID    | spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector
↳ Layer B      | SVID issuance confirmation (spiffe-helper logs)...
✓ Status       | [ PASS ] Sidecar log confirms SVID fetched & written:
time="2026-08-06T10:09:20Z" level=info msg="Using configuration file: \"/etc/spiffe-helper/helper.conf\" system=spiffe-helper
time="2026-08-06T10:09:20Z" level=info msg="Launching daemon" system=spiffe-helper
time="2026-08-06T10:09:20Z" level=info msg="Watching for X509 Context" system=spiffe-helper
time="2026-08-06T10:09:21Z" level=info msg="Received update" spiffe_id="spiffe://oran.fyp.local/ns/ricxapp/sa/dos-detector" system=spiffe-helper
time="2026-08-06T10:09:21Z" level=info msg="X.509 certificates updated" system=spiffe-helper
=====

[ SUMMARY ] ATTESTATION RESULTS
=====
✓ Result        | [ ATTESTATION COMPLETE ] Both layers passed. xApp identity proven.
=====

[ ✓ SUCCESS ] xApp is Active and Cryptographically Secured
=====

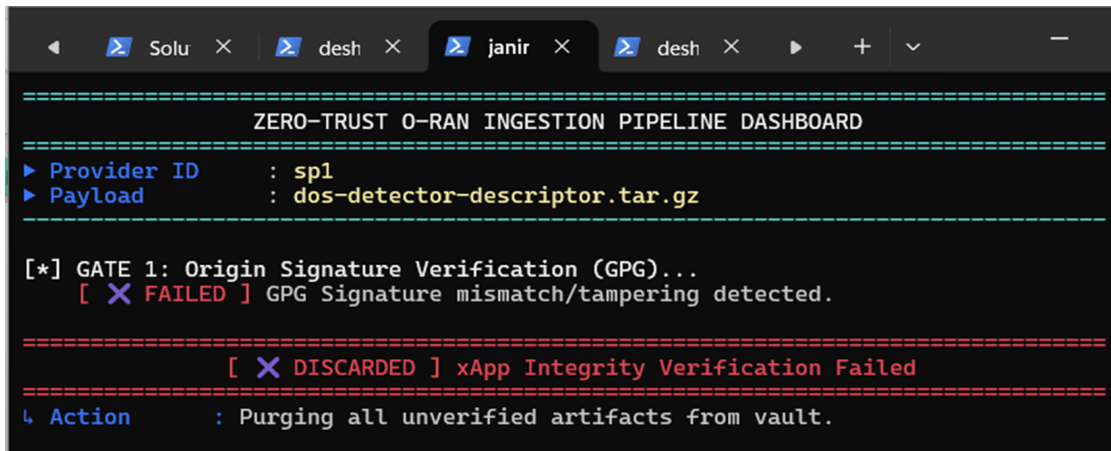
```

Figure 4.16: Initial onboarding validation: successful xApp deployment and selector-based SPIRE workload attestation on the Near-RT RIC

4.4.3 Man-in-the-Middle Simulation

To validate Gate 1’s integrity rejection capability, the legitimate descriptor tarball was modified by altering the container image tag inside `xapp-descriptor.json` to an unauthorized value. The tarball was re-archived with the same filename but the original, unaltered GPG signature file was retained alongside it, exactly as an attacker who intercepts the delivery channel and modifies the payload without access to the provider’s private key would do.

The modified package was placed in the SMO’s ingestion directory and `secure-ingestion.sh` was executed. Gate 1 detected the signature mismatch immediately, the signed digest embedded in the `.asc` file no longer matched the digest of the modified tarball and terminated the pipeline before unpacking the archive. The ingestion and quarantine directories were automatically purged.



```
=====
ZERO-TRUST O-RAN INGESTION PIPELINE DASHBOARD
=====
▶ Provider ID      : sp1
▶ Payload          : dos-detector-descriptor.tar.gz
=====

[*] GATE 1: Origin Signature Verification (GPG)...
  [ X FAILED ] GPG Signature mismatch/tampering detected.

=====
  [ X DISCARDED ] xApp Integrity Verification Failed
=====
↳ Action          : Purging all unverified artifacts from vault.
```

Figure 4.17: Man-in-the-middle simulation result

Gate 1 of the SMO ingestion pipeline detects a GPG signature mismatch on the tampered descriptor tarball and terminates the pipeline, leaving no trace of the malicious payload in the ingestion environment. This result confirms that the dual-signature model cryptographically prevents unauthorized modifications from reaching the RIC regardless of whether the attacker preserves the original filename, archive structure or signature file, since the signature is bound to the exact byte content of the archive at the time of signing.

4.4.4 Onboarding Latency and Performance Overhead

The final JIT sidecar onboarding pipeline was compared with native deployment trial by trial across 200 experimental runs in Figure 4.18. The native deployment has a very low mean of 2.53s and a stable, narrow distribution, with very few deployments outside the 2s–3s range. The zero-trust pipeline has a wider distribution from approximately 3 seconds to 17 seconds and a mean latency of 9.91s. This additional latency and variance arise from the serial cryptographic verification operations, capture of the active Pod UID and the asynchronous SPIRE agent handshake during pod initialization.

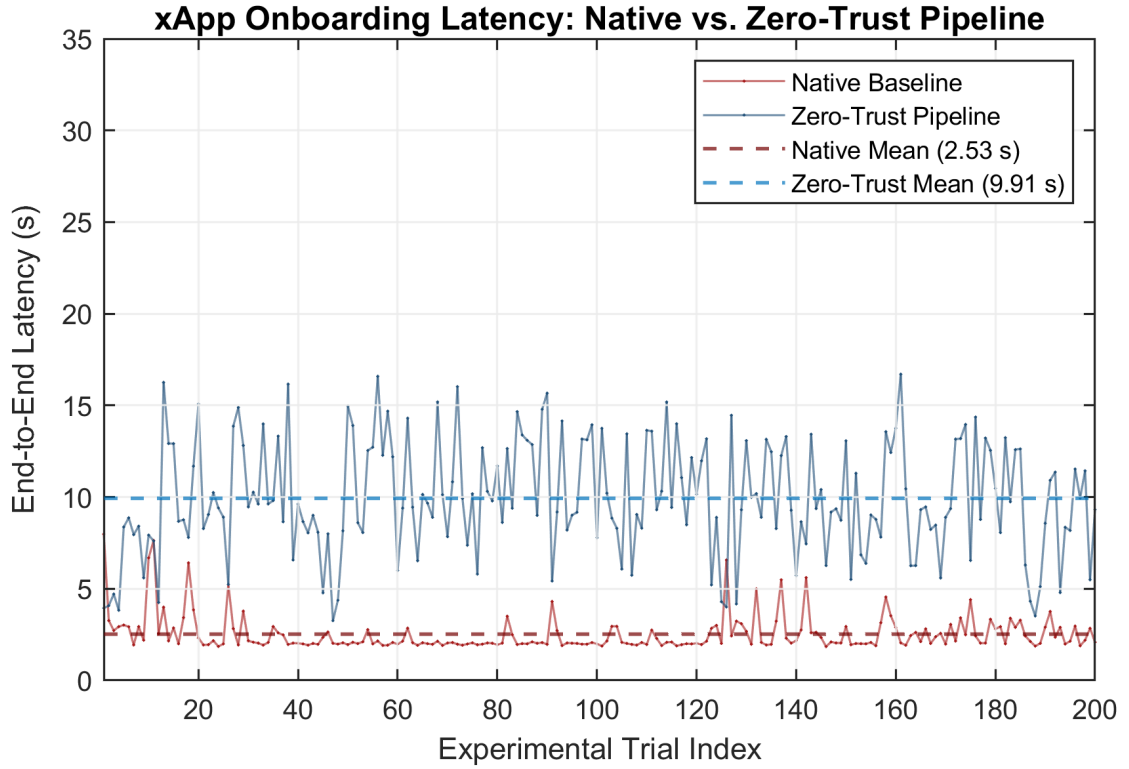


Figure 4.18: Comparison of xApp onboarding latency across 200 trials for baseline vs. zero-trust pipeline.

The onboarding process is approximately 10 seconds and has an added overhead, but is still within acceptable operational tolerances for initial xApp deployment. This latency trade-off is not an issue for establishing a cryptographically verified zero-trust boundary as onboarding and attestation are lifecycle events, unlike the continuous 10 ms - 1 s near-real-time control loops.

4.4.5 Security Implementations and Threat Mitigation

- **Mitigation of supply chain attacks:** The dual-signature model ensures that only packages signed by a known, pre-trusted Solution Provider are accepted. An attacker cannot forge a valid GPG or Cosign signature without possessing the developer’s private keys. The SMO’s gateway script enforces this by rejecting any package that fails either verification step.
- **Infrastructure defence via cert-manager:** cert-manager automates the lifecycle of the trust domain’s root CA. All SPIRE-issued certificates such as agent SVIDs, workload SVIDs are signed by the same spire-ca-issuer. This unified trust chain enables mutual TLS verification across all components without manual certificate distribution.
- **Bounded onboarding-identity lifetime:** SPIRE issues short-lived X.509 SVIDs in the onboarding testbed. Removing the JIT registration entry prevents renewal, so after the current SVID expires the workload can no longer authenticate to a service that enforces SPIFFE identity. The run-time containment testbed implements the corresponding revocation objec-

tive through its separate `ClusterSPIFFEID` label binding, as described in Section 3.11.

This secure onboarding framework provides four independent security layers:

1. Package integrity (GPG on the descriptor)
2. Image integrity (Cosign on the container image)
3. Admission gating (the SMO verifies both artifacts before releasing the chart for deployment)
4. Runtime identity (JIT Pod-UID workload attestation with a short-lived, renewable SVID)

A malicious xApp must bypass all four layers to operate in the RIC, which is cryptographically infeasible without access to the developer’s private keys.

Table 4.1: Attack scenarios and points of failure in the zero-trust onboarding pipeline

Attack scenario	Point of failure	Pipeline response	Security significance
Unsigned xApp package	Gate 1: GPG verification	The package was rejected because a valid provider signature was unavailable.	Blocked an unauthenticated package before processing.
Tampered signed archive	Gate 1: GPG verification	The modified archive no longer matched its original GPG signature.	Detected changes made after package signing.
Malformed xApp descriptor	Gate 2: Descriptor validation	The package passed signature verification but failed descriptor parsing or field validation.	Blocked invalid deployment metadata.
Unsigned or tampered image	Gate 3: Cosign verification	The referenced image digest did not have a valid trusted Cosign signature.	Prevented an unverified image from reaching the RIC.
Invalid SPIFFE registration	Gate 4: SPIRE registration	The requested identity or selector information violated the configured SPIRE policy.	Prevented creation of an unauthorised identity.
Runtime selector mismatch	Gate 5: Workload attestation	The running pod did not match its registered namespace, service account or label selectors.	Prevented the workload from receiving the trusted SVID.

An adversarial evaluation was performed in a controlled manner to evaluate how the Zero Trust xApp onboarding pipeline responds to any attacks aimed at the controls for package authentication, descriptor validation, container image authenticity, SPIRE identity registration, and workload attestation at runtime. Through this evaluation, each one of these scenarios was set up to pass through the previous

controls to the point where it triggered the security gate under test. Table 4.1 summarises the evaluated scenarios and their corresponding points of failure.

The results of the assessment shown in Table 4.1 proves that the onboarding mechanism worked in the form of a fail-closed verification process. Unauthorized or altered packages could not be processed at Gate 1, descriptors with incorrect syntax could not get through Gate 2, and unverified container images were not allowed to enter the Near-RT RIC at Gate 3.

Any attacks that had to do compromise in identity management were restricted by the last two steps of the process. Registration information provided by SPIFFE could not be verified at Gate 4, whereas identity selectors that did not match the Kubernetes workload were detected at Gate 5. Through this pipeline, the xApp became a trusted workload after successful verification of its software artifacts and identity.

4.5 Behavioral Threat Detection

The behavioral channel was evaluated on the NIST O-RAN Automation Tool testbed against the live KPI monitoring xApp (kpimon-go) while it received genuine E2 KPM indications throughout. Eight adversary scenarios, each reproduced 50 times for 400 trials in total, exercise the critical-tier detection paths: an in-container shell, sensitive-file access, ServiceAccount-token access, SVID or SPIFFE material access, malicious tool execution, external egress, profile or configuration tamper and a container-escape attempt. The scenarios use real tooling, with the container-escape and tool-execution cases reproducing Peirates [49] techniques. Each attack is launched by a real system call inside the live pod rather than by injecting a synthetic alert, so the measured latency covers the full path from the kernel event through Falco, alert forwarding and the decision webhook, starting from attack onset. The rule set, its severity assignments and the trust-state mapping were all frozen before the evaluation began. Table 4.2 maps each evaluated attack to its MITRE ATT&CK technique [50], tying the corpus to a recognized adversary-technique taxonomy.

Table 4.2: Evaluated attacks mapped to MITRE ATT&CK techniques

Evaluated attack	MITRE ATT&CK technique
In-container shell	T1059 Command and Scripting Interpreter
Sensitive-file access	T1552 Unsecured Credentials
ServiceAccount-token access	T1528 Steal Application Access Token
SVID / SPIFFE material access	T1552 Unsecured Credentials
Malicious tool execution	T1105 Ingress Tool Transfer
External egress	T1041 Exfiltration Over C2 Channel
Configuration tamper	T1565 Data Manipulation
Container-escape attempt	T1611 Escape to Host
Resource exhaustion (statistical channel)	T1499 Endpoint Denial of Service

4.5.1 Detection and Containment Effectiveness

Every one of the 400 trials was contained. There were no detection misses and no containment failures across the eight scenarios. Table 4.3 reports, per scenario, the detection decision time and the time to complete application of the four containment locks, both measured from attack onset. The active reachability probe is reported separately.

Table 4.3: Behavioral-channel detection and containment across the eight attack scenarios (50 trials each, mean)

Attack scenario	Contained	Detection (ms)	Four-lock application (s)
Unexpected shell	50/50	153	4.96
Sensitive file access	50/50	228	4.65
ServiceAccount token access	50/50	211	4.47
SVID / SPIFFE material access	50/50	209	4.49
Malicious tool execution	50/50	200	4.46
External egress	50/50	163	4.16
Profile or config tamper	50/50	184	5.47
Container-escape attempt	50/50	215	4.52
All scenarios	400/400	196	4.73

4.5.2 Detection and Containment Latency

The detection decision, which classifies an incoming event as an attack, is a small and stable part of the response, with per-scenario means from 153 ms (unexpected shell) to 228 ms (sensitive file access) and an overall mean of 196 ms. From attack onset, application of all four locks completes in a mean of 4.73 s and ranges from 4.16 s (external egress) to 5.47 s (profile or configuration tamper). This includes withdrawal of the identity binding but not expiry of the already issued SVID. The unexpected shell reaches the network lock fastest, in about one second, because its evidence appears at the very first system call, although all four locks still take about five seconds once SPIRE controller reconciliation is included. Figure 4.19 pairs the per-scenario detection decision with this four-lock application latency.

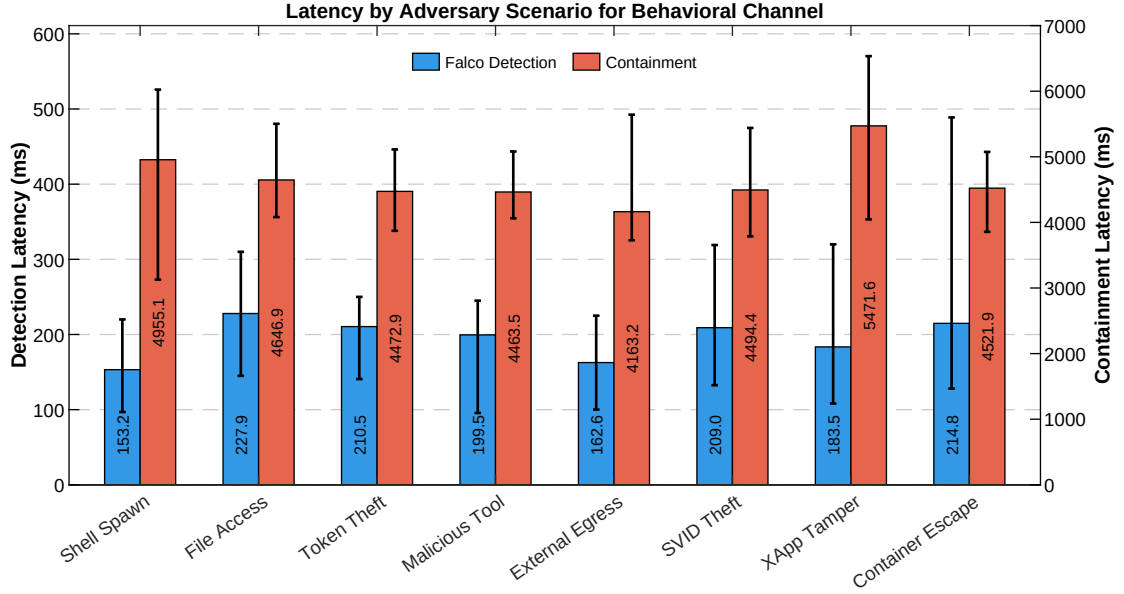


Figure 4.19: Behavioral channel per attack scenario: detection decision latency (left axis) and attack-onset-to-four-lock-application latency (right axis) across the 400 trials.

4.5.3 Per-Mechanism Containment Latency

Breaking the response down by lock shows where the containment time is spent (Table 4.4). The deny-all NetworkPolicy, the service-endpoint isolation and the direct node-level packet-filter block all land within roughly 2.4 to 3.0 s, while SPIFFE identity-binding withdrawal is the longest single mechanism at about 4.4 s because it waits on the SPIRE controller to reconcile the removed selector binding. This measurement does not include the residual lifetime of the already issued SVID. After the locks are in place, the active TCP-connect probe confirms the xApp is genuinely unreachable in a mean of 5.05 s, which is the point at which the incident is recorded as contained.

Table 4.4: Per-mechanism containment latency from attack onset (mean across the 400 behavioral trials)

Containment mechanism	Latency (s)
Deny-all NetworkPolicy (micro-segmentation)	2.39
Service-endpoint isolation	2.83
Direct node-level iptables block	2.95
SPIFFE identity-binding withdrawal	4.36
Probe-confirmed unreachable (recorded contained)	5.05

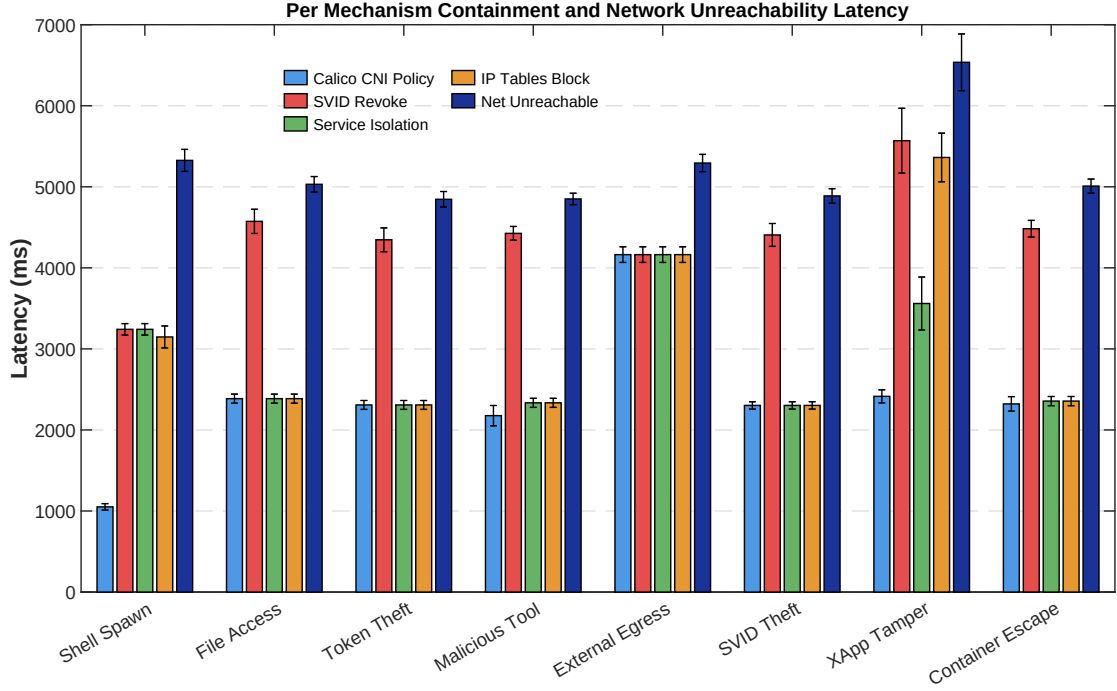


Figure 4.20: Per-mechanism containment and network-unreachability latency (mean with 95% confidence interval) across the 400 behavioral trials.

4.5.4 Behavioral Monitoring Overhead

The behavioral channel’s steady-state cost is small relative to the Near-RT RIC compute budget. Across the 400 trials the Falco eBPF monitor used a median of 12.4% of one core (maximum 16.2%) and about 109 MB of resident memory and it dropped zero events even under attack load. Each containment episode issued five Kubernetes API writes. After an incident, the xApp is recreated from the clean deployment template and this recreation completes in a median of 24.6 s (p95 30.6 s), after which the fresh pod passes the re-validation checks and returns to service.

4.6 Statistical Resource-Anomaly Detection

The ZT-XGuard MEWMA/ T^2 resource-anomaly detector was evaluated against six controlled scenarios on the NIST O-RAN Automation Tool testbed. The KPI monitoring xApp was running and receiving E2 KPM indications throughout all scenarios. The detector model and its calibration limits were fitted on clean baseline data and the CPU rate-gate window and rate were then fixed during a development phase that used two attack datasets (A1 and A3). After that point every parameter was frozen and no parameter was adjusted after observing any of the held-out evaluation datasets, which are the clean held-out sets (D1 to D3) and the held-out attack sets (A2 and A4). We separate development datasets from evaluation datasets explicitly, so that the blind results reported below rest only on data that never influenced any parameter.

4.6.1 Evaluation Dataset Overview

Table 4.5 summarizes all datasets used in the evaluation, their roles and their relationship to the model fitting and calibration phases.

Table 4.5: Evaluation dataset inventory: roles, sizes and relationship to model fitting, calibration and evaluation phases

Dataset	Role	Size	Notes
Clean-Baseline-Fit Dataset 1 (N1) Clean-Baseline-Fit Dataset 2 (N2)	Model fitting	1800 each	MEWMA center, scale and Ledoit-Wolf covariance matrix estimated exclusively from these two datasets. All parameters frozen immediately after fitting.
Clean-Baseline-Calibration (N3)	Threshold calibration	1800	WL and UCL derived from the 97.5th and 99.5th empirical percentiles of the T^2 score distribution on this dataset. CPU floor derived from N1, N2 and N3 pooled ($n = 5400$) using the Wilks tolerance bound.
Clean-Heldout Dataset 1 (D1) Clean-Heldout Dataset 2 (D2)	Held-out clean evaluation	1800 each	Never used in any fitting or calibration step. Used in the final blind evaluation to verify 0.00% FPR on genuinely unseen clean data.
Benign-Elevated Load (D3)	Blind control	1500	No attack active. Legitimate KPM processing load elevated artificially. Collected after model freeze for the final legitimate elevated workload blind evaluation.
CPU-Exhaustion (A1)	Development attack dataset	1800	Sustained CPU exhaustion using stress-ng at 20% of available CPU for 900 seconds within the collection window. Used together with A3 to select the CPU rate-gate window ($L = 60$ s) and rate ($r = 0.15$); it is therefore a development dataset and detection results on it are not blind.
CPU-Memory Combined (A2)	Held-out attack evaluation	5700	Multi-stressor attack combining CPU stress at 10% with sustained memory pressure across a 58-minute active attack phase. Never used in any calibration or development step; evaluated directly on the frozen model.

Dataset	Role	Size	Notes
Low-and-Slow (A3)	Development / boundary dataset	4800	Four-step intensity staircase at $0.7\times$, $1.0\times$, $1.3\times$ and $1.6\times$ the frozen CPU floor (56.638 mCPU), each step sustained for 15 minutes. Used with A1 to select the rate-gate window and rate and to map the detection boundary; it is a development dataset, not a blind evaluation.
Burst-Pulse-Train (A4)	Pulse-duration boundary characterization	2800	Five burst pulses of increasing duration (5s, 10s, 20s, 40s, 60s) separated by five-minute inter-pulse baselines. Designed to test the frozen dual gate around its CPU-rate count boundary: a 5s pulse is below the ten above-floor observations required by Gate B, while 10s is the shortest tested duration capable of supplying that count, subject to sampling alignment and simultaneous satisfaction of the 6-of-8 T^2 gate.

Of the gate components, the CPU floor is the only one derived purely from clean data (the Wilks tolerance bound over the pooled clean baseline); the rate-gate window and rate are development-phase choices informed by A1 and A3. The blind results that follow are therefore of two kinds. The false-positive rate rests on the genuinely unseen clean datasets D1, D2 and D3, none of which touched any parameter and the held-out attack-detection results rest on A2 and A4, two attack classes never seen during tuning. Detection figures reported for A1 and A3 are development results and are labeled as such wherever they appear.

4.6.2 Clean Baseline Specificity

Before evaluating attack detection, the model’s false positive rate on genuinely clean, unseen data must be established. Table 4.6 reports the T^2 score behavior across the two baseline heldout, clean datasets (D1 and D2) and benign elevated-load dataset (D3). A false positive is defined as a per-second state classified as SUSPICIOUS or COMPROMISED during a phase where no attack was active.

Table 4.6: False positive rate on clean and benign-elevated-load evaluation datasets (n = total scored samples after warmup exclusion)

Dataset	Type	N scored	False positives	FPR
D1	Held-out clean	1745	0	0.00%
D2	Held-out clean	1745	0	0.00%
D3	Benign elevated load	907	0	0.00%
Aggregate		4,397	0	0.00%

Across D1–D3, the detector produced zero false-positive states in 4,397 scored clean-phase samples, giving an observed sample-level FPR of $0/4,397 = 0.00\%$. This is an empirical result for these held-out collections, not a claim that the population false-positive probability is exactly zero. The clean-derived CPU floor has a nominal one-sided Wilks coverage target of at least 99.9% with at least 95% confidence under the same distribution as the calibration data. That tolerance statement does not by itself guarantee zero state-level false positives because the full decision also depends on the T^2 persistence gate and future workloads may differ from the calibration population. The held-out result nevertheless provides direct evidence that the complete frozen detector remained specific for the two unseen clean runs and the separately collected benign elevated-load run. It is also consistent with the intended role of the directional-excess transform in suppressing harmless downward baseline drift.

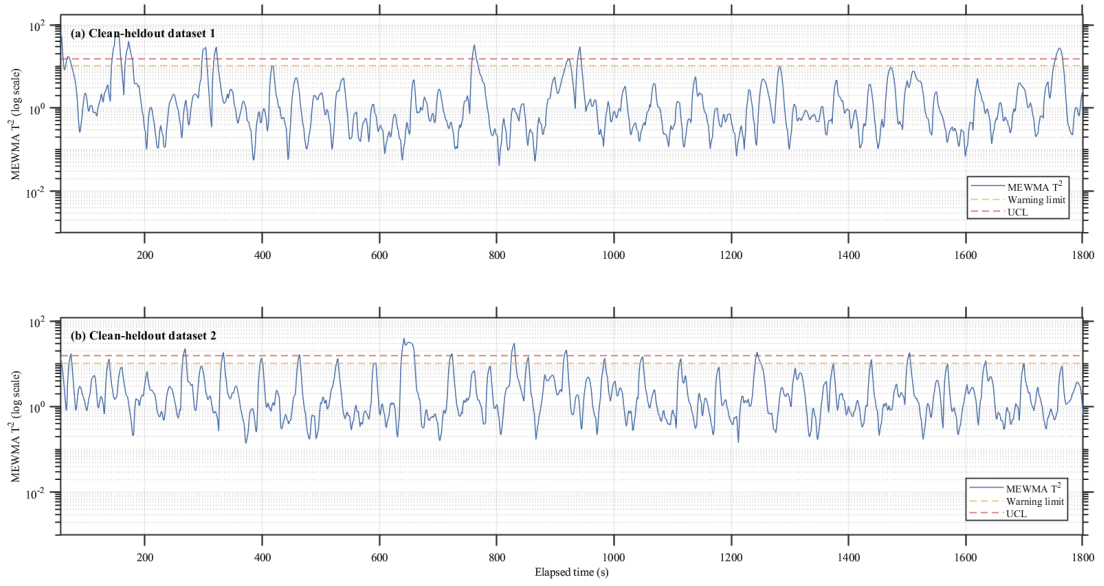


Figure 4.21: MEWMA/ T^2 pressure score on held-out clean datasets D1 (top) and D2 (bottom) across 1745 scored samples each

The score remains below the Warning Limit ($WL = 10.4364$) throughout both collections, producing 0 false positive states. WL and UCL lines are shown for reference.

4.6.3 Resource Exhaustion Adversary Scenarios

The detector was evaluated against four sustained attack scenarios. For each, the ground-truth attack window boundaries were determined from the collection timeline files recorded at the time of data collection. True Positives are per-second states classified as SUSPICIOUS or COMPROMISED during the active attack window; False Negatives are attack-window samples not detected; True Negatives are correctly classified clean-phase samples; False Positives are clean-phase samples incorrectly classified as anomalous.

Table 4.7: Confusion matrix for the three sustained scenarios (A1 and A3 development; A2 held out)

Scenario	N scored	TP	FN	FP	TN	TPR
CPU Exhaustion (A1)	1444	856	44	0	544	95.11%
CPU-Memory Exhaustion (A2)	4317	3464	18	0	835	99.48%
Low and Slow Attack (A3)	4144	3560	40	0	544	98.89%

(n = total scored samples per scenario, baseline + attack window only; recovery-phase samples are excluded from this count and discussed separately as a bounded de-escalation tail)

All three sustained scenarios achieve $\text{TPR} > 95\%$, with 0 false positives observed across 4,397 clean-phase samples. A2 achieves the highest TPR despite operating at lower per-metric intensity than A1, demonstrating the multivariate detector’s ability to fuse distributed weak signals across CPU, memory and network dimensions.

Sustained CPU Exhaustion Adversary Scenario

The A1 CPU-exhaustion scenario served as a development dataset. Together with A3, it was used to select the CPU rate-gate window $L = 60$ s and rate threshold $r = 0.15$. The stress-ng CPU stressor ran at 20% of available CPU for 900 seconds within an 1800-second collection, with 600 seconds of pre-attack baseline and 300 seconds of post-attack recovery. Its 95.11% true-positive rate therefore characterizes development performance and is not presented as a blind or independently held-out result.

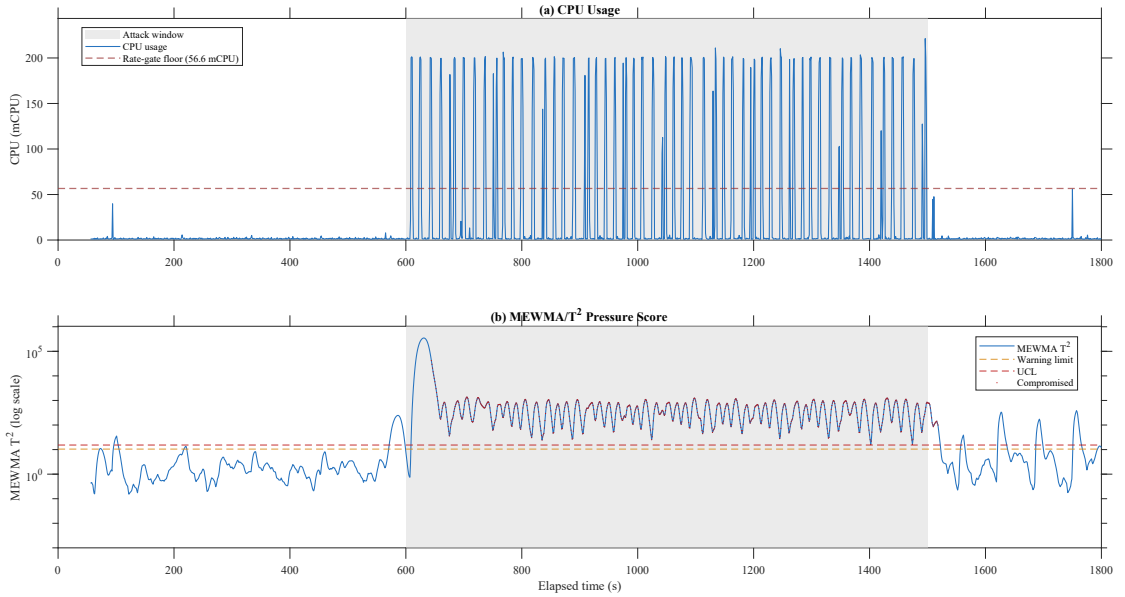


Figure 4.22: A1 development dataset (sustained CPU exhaustion at 20% utilization), used with A3 to select the CPU rate-gate window and rate.

(a) raw CPU usage in mCPU with the rate-gate floor at 56.638 mCPU; (b) MEWMA/ T^2 pressure score on a log scale, showing rapid escalation above the UCL (15.3084) at attack onset and return of the per-sample statistical output below the warning threshold during the recovery phase. TPR: 95.11% (n=900 attack-window samples).

The detector correctly classified 856 of the 900 attack-window samples as SUSPICIOUS or COMPROMISED, achieving 95.11% TPR. The MEWMA state accumulates sustained CPU pressure until the six-of-eight persistence rule and the CPU rate gate both confirm it. After the attack ends, the detector’s per-sample statistical output falls below the warning threshold during the recovery phase. This decay of the statistical signal is not a transition of a contained xApp from COMPROMISED back into service.

Combined CPU and Memory Exhaustion Adversary Scenario

The A2 scenario combined a CPU stressor at 10% utilization with a sustained memory pressure component for a 58-minute active attack phase, with 15 minutes of pre-attack baseline and 12 minutes of post-attack recovery, for a total collection duration of 95 minutes. This scenario was held out from all calibration and development phases; the model had never been exposed to any A2 data before this evaluation.

The detector achieved 99.48% TPR with a detection latency of 21.0 seconds, which is notably faster than A1 despite the per-metric CPU intensity being only half that of the A1 scenario. This held-out result is consistent with the motivation for multivariate fusion: CPU, memory-growth and network deviations contribute jointly through the Ledoit-Wolf precision matrix, so coordinated pressure can raise the T^2 score even when no single component is as large as the dominant component of a different attack. Because A1 and A2 differ in both composition and duration, their latency difference supports but does not by itself isolate the causal contribution of multivariate fusion; the

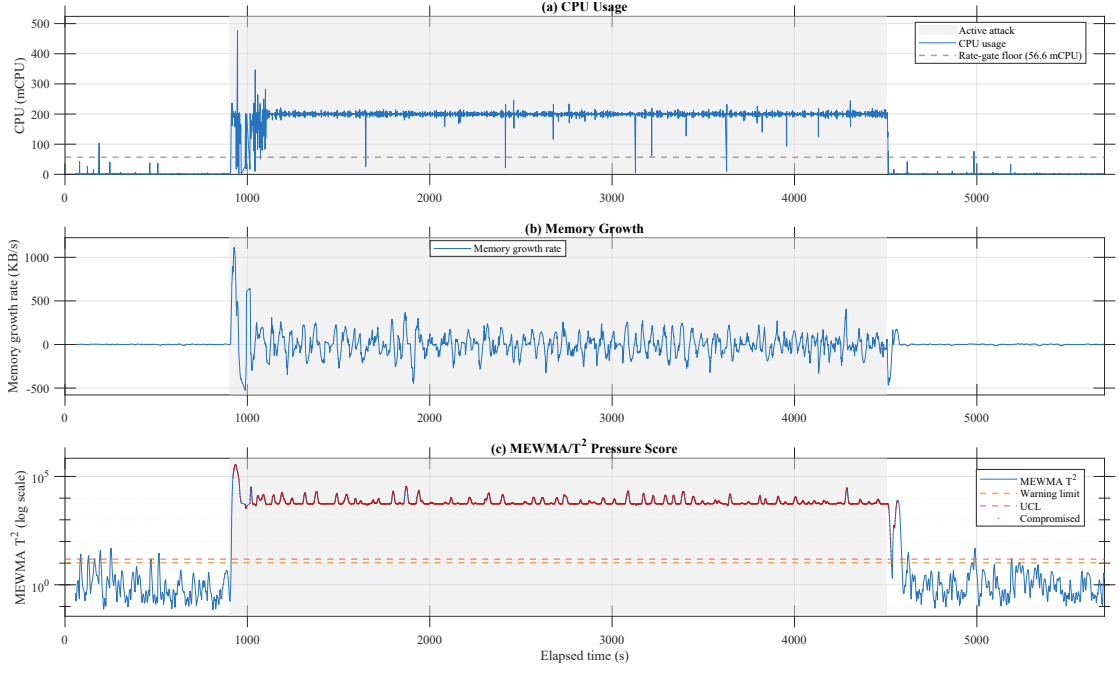


Figure 4.23: A2 held-out evaluation (combined CPU + memory exhaustion)

(a) CPU usage, (b) memory growth rate and (c) MEWMA/ T^2 pressure score over 5700 seconds. The detector maintains a sustained anomalous per-sample classification throughout the 58-minute attack phase, after which its statistical output falls below the warning threshold during recovery. This is detector-signal decay, not restoration of a contained xApp. TPR: 99.48% ($n=3482$ attack-window samples).

Low-and-Slow Intensity Staircase Adversary Scenario

A3 served as a development and boundary-characterization dataset. It was used together with A1 to select the CPU rate-gate window and rate; therefore, its aggregate true-positive rate and per-step behavior are development results rather than independently held-out performance. The scenario ran CPU stress at four intensity levels defined relative to the frozen CPU floor: $0.7\times$ (39.6 mCPU), $1.0\times$ (56.6 mCPU, at the floor), $1.3\times$ (73.6 mCPU, above the floor) and $1.6\times$ (90.6 mCPU, comfortably above the floor). Each step ran for 15 minutes, back-to-back with no cooldown between escalating levels. The purpose of the staircase design was to characterize the detection boundary empirically relative to the calibrated CPU floor.

The detector achieved 98.89% TPR across the full staircase. Contrary to a naive expectation that a $0.7\times$ -floor mean intensity would evade detection entirely, Step 1 alone was already classified SUSPICIOUS for 95.6% of its samples (860 of 900). This is because the CPU rate gate evaluates the fraction of individual 1-second samples exceeding the instantaneous floor within a 60-second window, not the mean load over that window: a bursty stress-ng workload targeting a $0.7\times$ *average* intensity still produces enough individual samples above the floor to satisfy the 15% exceedance threshold. Steps 2–4 begin already in a confirmed SUSPICIOUS

state carried over from Step 1 (verified directly: the first three samples of each of Steps 2–4 are already SUSPICIOUS), so only Step 1 constitutes a genuine fresh detection onset in this design; the 98.89% aggregate TPR is attributable to Step 1’s onset. This development result shows that the selected rate gate detects the tested bursty workload even when its target mean is below the instantaneous floor. It does not establish resistance to every adaptive duty-cycling strategy, whose outcome would depend on its per-sample exceedance rate and simultaneous T^2 trajectory.

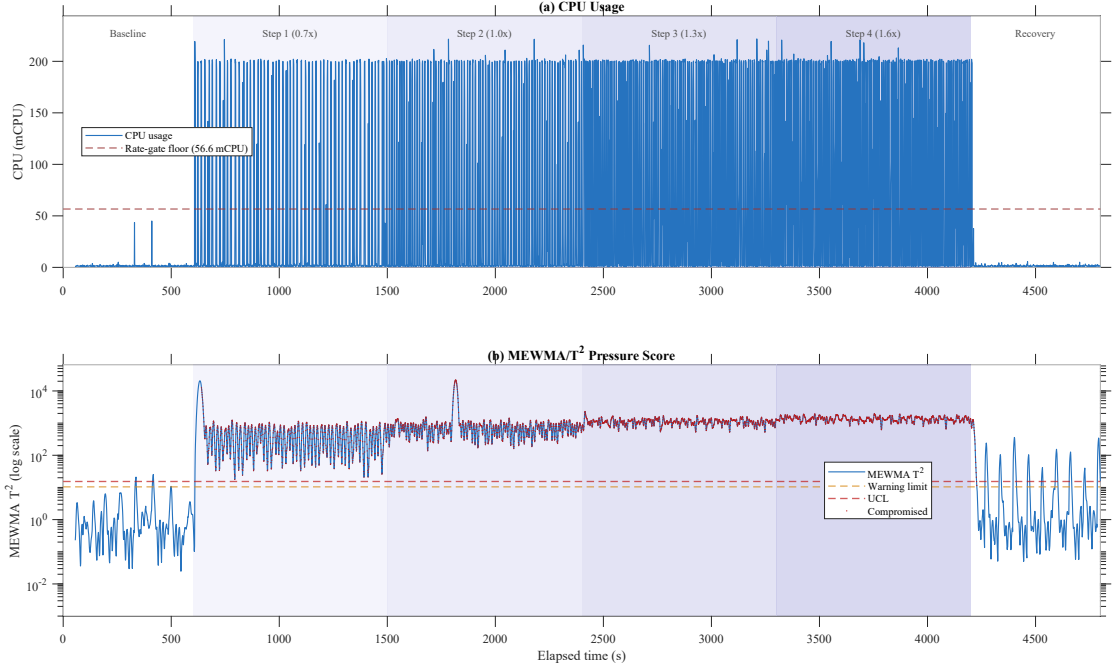


Figure 4.24: A3 development and boundary-characterization dataset (4-step stealth intensity staircase)

(a) CPU usage showing the four intensity steps annotated relative to the frozen CPU floor (56.638 mCPU); (b) MEWMA/ T^2 score. Detection begins already within Step 1 ($0.7\times$ floor mean intensity) at 95.6% of samples, driven by the rate-gate’s sensitivity to per-sample bursts rather than mean load, not at Step 2 as a naive mean-based reading of the staircase design would suggest. Detection is sustained through Steps 2–4. Aggregate TPR: 98.89%, with detection beginning at Step 1 onset.

4.6.4 Burst-Duration Boundary Characterization

The A4 scenario characterized the detector’s response over five tested pulse durations using a train with escalating burst lengths: 5s, 10s, 20s, 40s and 60s. Each pulse was separated by a five-minute inter-pulse baseline period to allow the MEWMA state to return to a clean level before the next pulse began.

Under the implemented strict rate condition, a full 60-sample history needs at least ten individual CPU observations above the floor; nine observations produce a rate of exactly 0.15 and therefore do not pass Gate B. Ten above-floor observations are

only a necessary CPU-corroboration condition, not a sufficient pulse-duration rule. The detector must also satisfy the 6-of-8 T^2 persistence gate at the same instant, and the observed onset is affected by one-second sampling alignment, the 8-second moving mean and the MEWMA trajectory. Consequently, A4 tests the frozen detector around the configured count boundary and brackets the tested transition between 5s and 10s; it does not establish a universal minimum detectable attack duration.

Table 4.8: A4 per-pulse outcomes around the frozen dual-gate boundary

Pulse	Duration (s)	Detected?	Latency	Notes
P1	5	No	—	Complete dual gate was not satisfied in this run
P2	10	Yes	11.0s	Shortest tested pulse detected; latency includes gate and sampling alignment
P3	20	Yes	10.0s	Detected in this run
P4	40	Yes	10.0s	Detected in this run
P5	60	Yes	11.0s	Detected in this run

Figure 4.25 shows the raw pulse train underlying Table 4.8: five CPU pulses of increasing duration against the T^2 response; Table 4.8 summarizes the corresponding detected/not-detected outcomes.

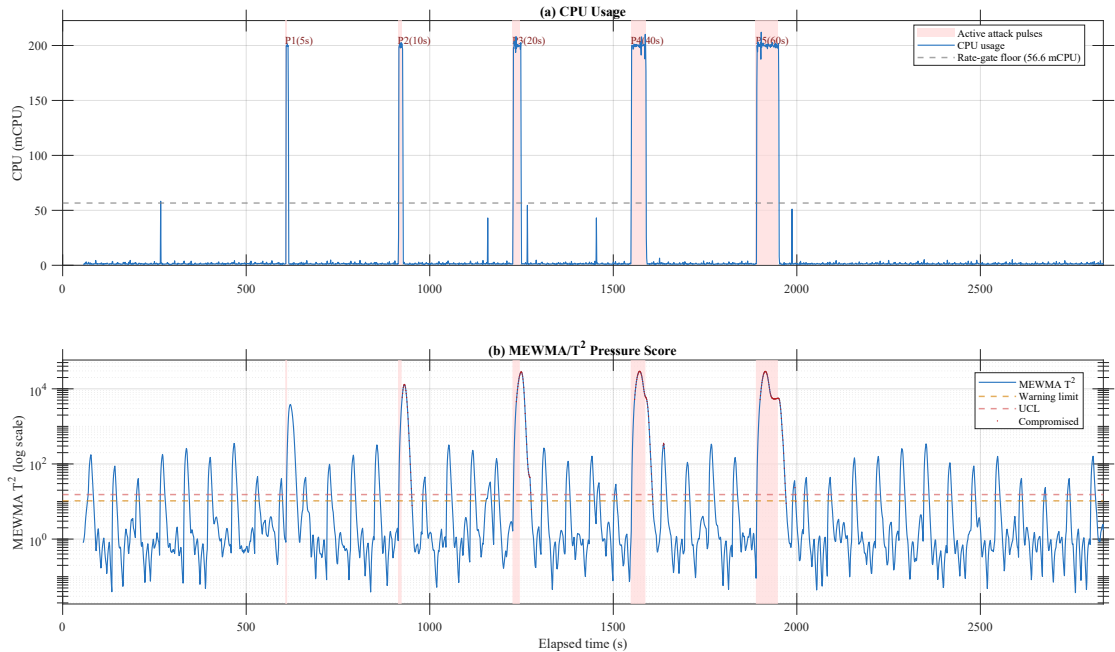


Figure 4.25: A4 raw pulse-train evaluation

(a) CPU usage across the five pulses (5s, 10s, 20s, 40s, 60s) with the rate-gate floor (56.638 mCPU) for reference; (b) MEWMA/ T^2 pressure score, with

confirmed-COMPROMISED periods shown as shaded bands. Only the pulses of 10s and above (P2–P5) produce a confirmed-COMPROMISED band; the 5s pulse (P1) produces a T^2 rise but never persists long enough for the 6-of-8 persistence and CPU rate gates to be satisfied simultaneously in this run.

In this A4 collection, P1 was not detected and P2–P5 were detected with onset latencies of 10–11 s. These observations bracket the tested transition between the 5s and 10s pulses and agree with the strict CPU-gate requirement that ten above-floor samples are needed. They do not prove that every 10s waveform will be detected or that every shorter waveform will evade detection: the result also depends on how many sampled values exceed the floor and whether the smoothed T^2 persistence gate overlaps the CPU gate.

The undetected 5s resource-only pulse is therefore reported as a deliberate sensitivity-specificity trade-off and a bounded evasion case for the statistical channel, not as proof that such a pulse is harmless. This experiment did not measure the service impact of that pulse. A short attack that also causes a high-confidence syscall violation remains detectable by the independent behavioral channel, whereas a resource-only pulse that does not satisfy the dual gate can evade statistical escalation.

4.6.5 Ablation: Contribution of Each Detector Stage

To show what each part of the detector contributes and not only that the whole works, we ablate it along two axes: the two detection channels and the two confirmation gates inside the statistical channel.

Channel ablation. Used on its own, each detection channel is one of the prior-art baselines this framework builds on and each is structurally blind to a threat class the other catches. The exploitation events the behavioral channel is built for, an in-container shell, an SVID read, an unexpected egress connection or a container-escape attempt, move no resource metric, so a statistical-only detector cannot see them. Conversely, overt and duty-cycled resource abuse produces no rule-violating system call, so a behavioral-only detector cannot see it. This is a property of the evidence each channel observes, not a tuning gap that more calibration would close. Only the dual-channel design covers both classes, which is the central reason the framework pairs them rather than choosing one.

Dual-gate ablation. Within the statistical channel, the six-of-eight persistence rule on the T^2 score and the CPU rate gate play distinct roles, which a K -of-8 sweep makes explicit. Sweeping the persistence requirement K from 1 to 8 with the CPU gate removed, persistence alone never reaches zero false positives on benign data: even at the strictest setting (8 of 8), several benign runs still produced false positives, ranging from a few to several hundred per run. Adding the CPU rate gate makes the fused rule zero-false-positive on every benign run at every K from 1 to 8. The two gates therefore contribute different things. The persistence rule governs how many raw T^2 exceedances become confirmed detections and its hit volume falls monotonically as K rises (one benign run, for example, drops from 53 raw exceedances at $K = 1$ to 5 at $K = 6$), while the CPU corroboration gate is what drives the false-alarm rate to zero. Neither gate alone delivers both

sensitivity and a zero false-positive rate; the combination does.

Gate-choice ablation. The CPU rate was chosen as the corroboration dimension over rate gates built on the memory-growth and network dimensions. Comparing the three, the CPU rate gate gave the cleanest separation between attack and benign behavior, which is why the memory and network rate gates are computed but excluded from the final decision. This makes the corroboration gate a deliberate design choice with an ablation behind it, rather than an arbitrary pick.

4.6.6 Deployed Detection and Automated Containment

The dataset evaluation above characterizes the detector on recorded telemetry. This subsection reports the detector running inside the deployed framework, where a confirmed anomaly drives the same verified containment the behavioral channel uses. Five scenarios were run live against the KPI monitoring xApp, ten trials each: a benign control, a sustained CPU flood, a burst pulse train, a stealth intensity staircase and a combined CPU-and-memory attack. Each trial measures the full path from attack onset through detection, the reversible CPU throttle, the fixed operator-intervention dwell and full four-lock isolation.

The statistical channel operates in two tiers. Partial statistical evidence raises the xApp to **SUSPICIOUS** and applies a reversible cgroup CPU throttle within approximately 30 ms. When the complete dual gate, consisting of the persistence condition and the CPU rate gate, is satisfied, the xApp is raised to **COMPROMISED** and the fixed 30-second operator-intervention dwell begins. The operator may initiate isolation during this interval; otherwise, four-lock isolation begins automatically when the dwell expires. The dwell therefore delays the containment action, not the **COMPROMISED** decision.

Table 4.9 reports the result. Every attack scenario was detected and escalated to **COMPROMISED**, while the benign control did not escalate. That control outcome is consistent with the separately observed 0/4,397 false-positive result in Section 4.6. Attack-onset-to-detection ranges from 11.4 s to 59.8 s. The combined CPU-and-memory scenario is detected fastest, an outcome consistent with the multivariate fusion mechanism, although the differing attack compositions do not isolate fusion as the sole cause. The burst pulse train is slowest because its inter-pulse gaps repeatedly interrupt the six-of-eight persistence count.

Once a resource anomaly is confirmed, the system deliberately holds a fixed 30 second dwell before it auto-isolates the xApp. This dwell is an intentional design choice, not a mechanism delay. Because the statistical evidence is probabilistic rather than a single decisive event, the dwell gives an operator a window to intervene before a control xApp is removed from the RIC, and it is a constant hold that is identical for every scenario. We therefore report the containment latency without this deliberate dwell. Once the dwell elapses, the four-lock containment itself completes in 4.5 to 5.1 s across the scenarios (Table 4.9). This is the same enforcement path used by the behavioral channel, and its post-dwell mechanism latency is comparable to the behavioral containment measurements.

Table 4.9: Deployed statistical-channel detection and post-dwell containment per scenario.

Scenario	Escalated to compromised	Detection (s)	Post-dwell containment (s)
Benign control	0/10 (TN)	—	—
Sustained CPU flood	10/10	37.0 ± 5.7	5.1
Burst pulse train	10/10	59.8 ± 0.4	4.9
Stealth staircase	10/10	33.4 ± 3.9	4.9
Combined CPU-and-memory	10/10	11.4 ± 0.6	4.5

Detection is the mean $\pm$ standard deviation over 10 trials; the containment column is the four-lock mechanism latency measured after the fixed 30 s operator-intervention dwell, which is a deliberate design choice excluded from the reported latency.

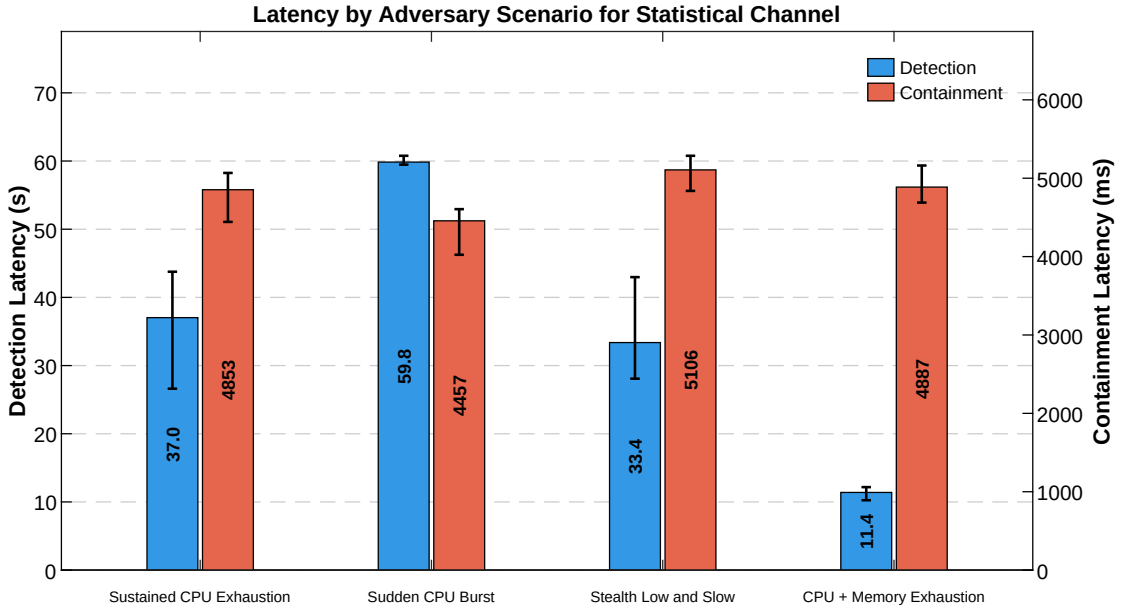


Figure 4.26: Deployed statistical channel per scenario: attack-onset-to-detection (left axis) and post-detection containment latency (right axis). The fixed 30 s dwell that precedes isolation is constant across scenarios and is omitted for clarity.

Before any isolation, the suspicious-tier CPU throttle is real enforcement rather than an alert. Figure 4.27 shows the container CPU held near the 40 mCPU cap within 16.7 to 27.2 ms of detection, down from the 200 to 260 mCPU the attack was driving. The throttle lifts automatically once the anomaly clears, so a transient that never reaches the dual gate is bounded without any service removal. Container CPU is capped near 40 mCPU within 16.7 to 27.2 ms of detection.

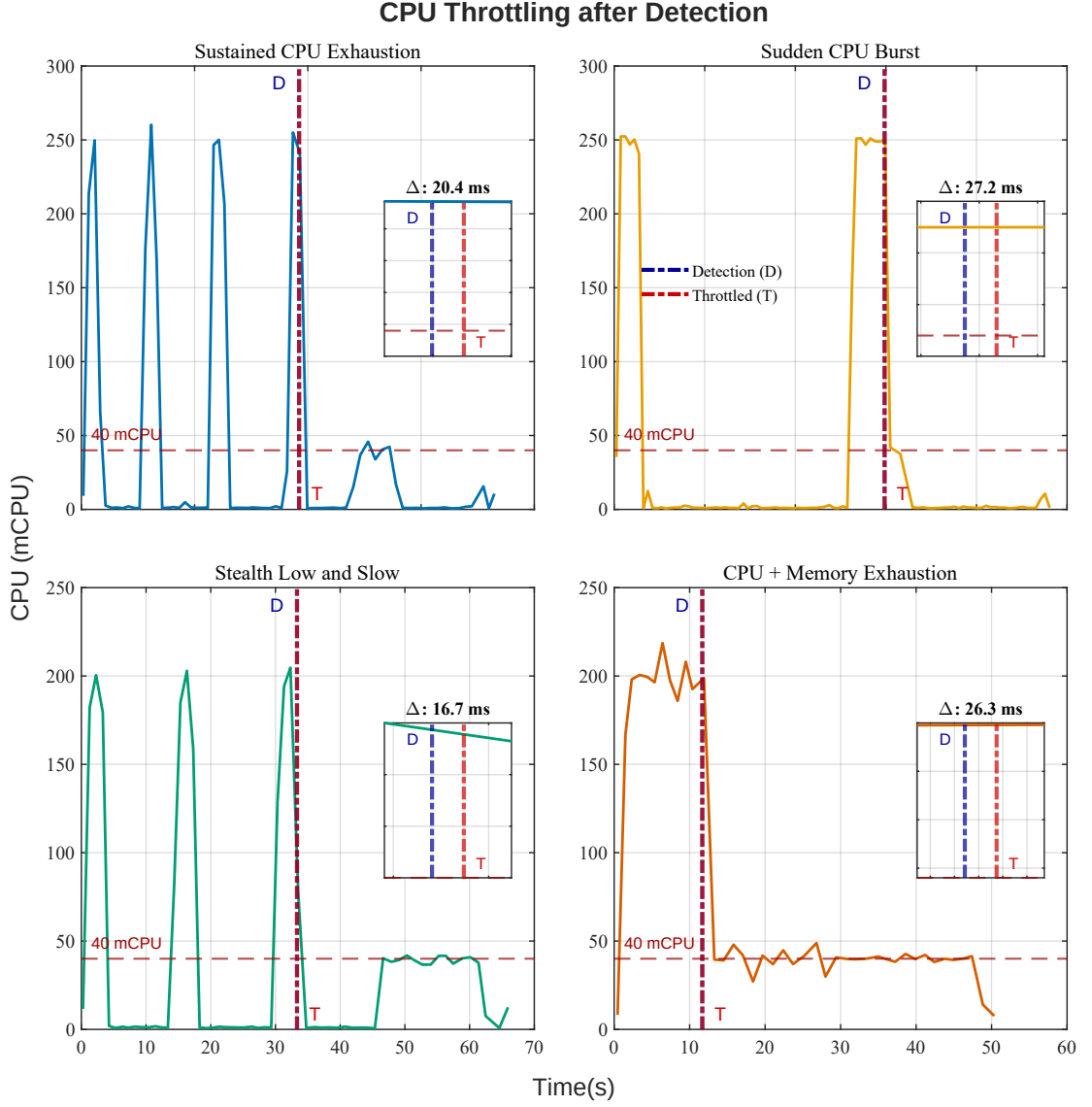


Figure 4.27: Reversible CPU throttle after escalation to suspicious from the resource channel.

4.7 Automated Containment and Isolation

Once a critical behavioral event or a statistically confirmed resource anomaly raises an xApp to **COMPROMISED**, the framework transitions from observation to active mitigation. The behavioral route initiates isolation immediately, whereas the statistical route uses the 30-second operator-intervention dwell. When isolation begins, the orchestrator runs the common four-lock process defined in Section 3.11 and verifies that the isolation took effect. The per-scenario detection and containment latencies are reported in Sections 4.5 and 4.6.6, and the per-mechanism breakdown appears in Table 4.4.

4.7.1 Verified Removal from the RIC Control Loop

The purpose of containment is not that a policy object exists but that the xApp genuinely leaves the control loop. After isolation, the xApp’s periodic E2 RIC-indication traffic stops and its received throughput falls by roughly half (Figure 4.28). More directly, the active TCP-connect probe from an uninvolved pod confirms that the xApp’s Service is unreachable and only then is the incident recorded as contained. This is a probe-verified outcome rather than an assumption that an applied label was honored by the data plane. Across the 400 behavioral trials and the 50 statistical trials, every confirmed compromise reached this probe-verified unreachable state.

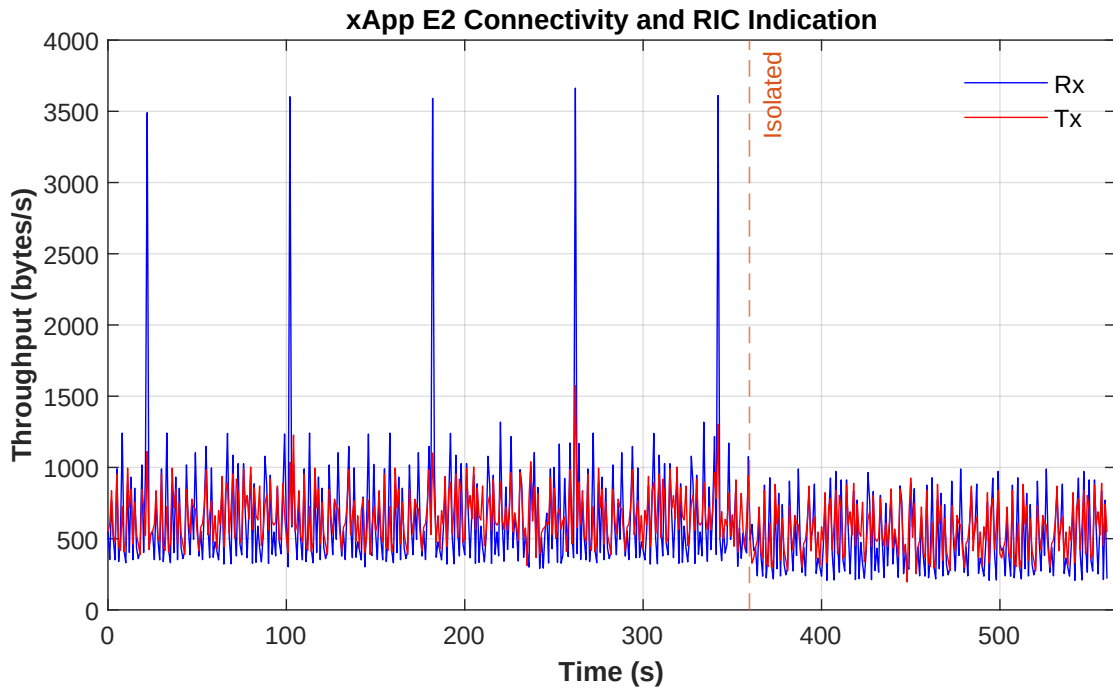


Figure 4.28: xApp E2 connectivity carrying RIC indication and reporting.

The periodic RIC indication peaks stop after isolation and received throughput drops by about half.

4.7.2 Resilience of the Isolation to the Compromised xApp

Because the attacker holds the xApp’s own in-cluster identity, containment must not be something the xApp can undo from inside its own pod. A `kubectl auth can-i` check with the `ricxapp` namespace default service account confirmed that this identity cannot delete the deny-all `NetworkPolicy` the orchestrator applies, so even a stolen `ServiceAccount` token does not let the compromised xApp lift its own network isolation. The independent node-level packet-filter block provides a second network lock that does not depend on the CNI pipeline, so isolation does not rest on a single mechanism.

4.7.3 Monitoring and Orchestration Overhead

The response machinery is lightweight. The detector and containment orchestrator run below 8 mCPU during normal monitoring, with only a brief sub-second spike while the locks are applied, after which usage returns to baseline. Combined with the Falco monitor’s steady-state cost reported in Section 4.5 (a median 12.4% of one core and about 109 MB of memory), the framework protects the xApps without measurably competing with healthy workloads for compute.

4.7.4 Experimental Reset Between Trials

Between trials, the testbed was manually returned to its clean baseline so that the same xApp scenario could be repeated. This experimental reset is not modelled as a policy-engine state transition and is not evaluated as part of the proposed containment strategy.

4.8 Scalability Evaluation

The earlier results establish detection and containment correctness for a single protected xApp. This section measures how ZT-XGuard behaves as load grows. It first sets out what scale means for one Near-RT RIC, then reports three separate experiments, one for each security function that could become a bottleneck.

4.8.1 Evaluation Scope and Industry Context

Neither the O-RAN architecture nor the public record of operator deployments fixes a universal number of xApps that a Near-RT RIC must host. Published deployments and experimental platforms instead show that application cardinality depends on the use case, the workload, the platform, and the available resources. ZT-XGuard therefore does not select its test range from a supposed industry-standard xApp count. It evaluates the scalability of each security function on its own terms.

Commercial activity confirms that RIC-based intelligence has moved into operational networks. Rakuten has reported nationwide deployment of RIC applications across its commercial mobile network in Japan, together with third-party rApp integration [51]. That 2026 announcement concerns RIC applications and third-party rApps, and is cited here only as evidence that RIC-hosted intelligence has progressed into large-scale commercial operation, not as a production Near-RT xApp count. Samsung and TELUS have announced commercial deployment of an AI-powered RIC with open multi-vendor support and an application suite [52]. Near-RT RIC and xApp operation over the native E2 interface has been demonstrated by Nokia and AT&T on operator infrastructure, reported as a trial rather than a production deployment [53]. None of these disclosures publishes an exact count of concurrently hosted Near-RT xApps, so none provides a directly comparable numerical capacity benchmark.

The specification record is consistent with this. O-RAN WG3 defines the Near-RT RIC architecture, the E2 interface, the xApp APIs, and the associated require-

ments and conformance procedures, but its public scope does not prescribe a maximum number of simultaneously hosted xApps [54]. The O-RAN Software Community Traffic Steering use case composes five xApps, namely KPI Monitoring, Anomaly Detection, Traffic Steering, QoE Prediction, and RAN Control [55]. That figure describes one reference workflow, not a capacity ceiling, and it is directly relevant here because the testbed RIC is built on O-RAN Software Community components and runs the KPIMON-GO xApp. A peer-reviewed O-RAN testbed provides a comparable small-cardinality precedent: Sultana et al. evaluate an xApp conflict-mitigation framework on an OTIC testbed using two interacting xApps [56], a precedent for meaningful multi-xApp validation using a small number of functionally distinct interacting applications rather than an xApp-capacity recommendation. Dedicated RIC scalability research has reached larger populations by scaling the underlying infrastructure: FlexScale uses Kubernetes horizontal and vertical pod autoscaling with load balancing across multiple Near-RT RIC pods and reports scaling a lightweight xApp workload from 38 to 89 instances [57]. That result characterizes a specialized resource-scaling architecture and is not directly comparable with the logical detector-context benchmark used here.

Given this evidence, the present work does not ask how many xApps a Near-RT RIC can host. It asks three separate questions, each exercising a different resource. Experiment 1 measures whether the statistical detector computation keeps up as the number of independently maintained protected workload contexts grows. Experiment 2 measures the offered request rate at which the policy decision plane begins to saturate. Experiment 3 measures how the real four-lock containment path behaves when one to four genuinely deployed xApp workloads are compromised at the same time. These three axes are not interchangeable. The first stresses statistical computation, the second stresses decision-plane request processing and queuing, and the third stresses live Kubernetes, SPIRE, Service, host packet-filter, and reachability enforcement. They are reported separately for that reason.

The concurrent-enforcement experiment uses the four xApp workloads actually deployed in the testbed: `hw-go`, `hw-python`, `kpimon-go`, and `traffixapp`. Only `kpimon-go` is genuinely E2 and KPM active; the other three are real deployed workloads without live E2 KPI traffic. The four-workload configuration lies within the small multi-application cardinality represented by existing reference workflows and real multi-xApp experimental studies, while remaining small enough that every target and every non-target can be instrumented and independently verified. It is not presented as representative of all production RICs.

4.8.2 Protected-Workload Detector Scalability

Only one statistical model is validated in this work, the model fitted for the KPI monitoring xApp. To measure computational scaling without introducing unvalidated models, the benchmark instantiates N independent detector contexts. All contexts share that single trained model, but each keeps its own MEWMA state, Hotelling T^2 statistic, trust state, and detector history. This mirrors the arrangement a real multi-xApp deployment would use, with one detector context per pro-

tected xApp. The context count is swept over $N \in \{1, 2, 3, 4, 5, 10, 20, 40, 60, 80\}$, five repetitions per level. Each repetition begins with eight warm-up ticks that initialize the MEWMA and history state and are excluded from the latency and deadline statistics, followed by 300 measured scoring cycles. Because the detector operates on a project-defined 1 Hz evaluation cadence, the complete N -context scoring cycle must complete within the 1000 ms evaluation period. The input is a real recorded metrics trace replayed from a single-UE diagnostic run and filtered to admissible samples, not synthetic data.

The metric needs a precise definition. For one logical scoring tick the harness starts a timer, calls `StreamingScorer.update()` once for each of the N contexts in turn, and stops the timer after the last context returns. The recorded value is therefore the time to rescore all N protected contexts once, that is the complete N -context scoring-cycle compute latency. It excludes telemetry acquisition, E2 transport, the wait for the next telemetry sample, detection dwell, policy evaluation, and containment. It is a compute-cost measurement, not an attack-detection latency. The 1000 ms reference is a project-defined processing budget tied to the 1 Hz detector cadence; it is not an O-RAN standardized latency requirement.

Across all ten levels the benchmark issued 346,500 scorer update invocations in total, counting the warm-up ticks. The latency and deadline statistics reported here are computed over the measured scoring cycles only, 300 per repetition and 1,500 per N . No measured cycle missed the processing budget at any N , and the decision-equivalence check passed at every N : each context produced the same trust-state sequence whether it ran alone or alongside others. Figure 4.29 plots per-cycle latency against N . At the top of the sweep, with 80 concurrent contexts, the median cycle took 3.56 ms, the p95 was 7.62 ms, the p99 was 12.0 ms, and the worst observed cycle was 50.7 ms, corresponding to approximately 5.1% of the 1000 ms processing budget. No computational saturation appeared through the $N=80$ range. The result demonstrates detector-compute headroom. It does not demonstrate that the physical Near-RT RIC can host 80 xApps, and the upper context counts are deliberately a high-load stress regime for the detector arithmetic alone.

Table 4.10: Detector computational scalability.

N contexts	Updates	p50 (ms)	p95 (ms)	p99 (ms)	Max (ms)
1	1,540	0.04	0.07	0.13	0.68
2	3,080	0.10	0.24	0.33	0.96
3	4,620	0.13	0.30	0.44	0.89
4	6,160	0.17	0.34	0.45	1.62
5	7,700	0.21	0.48	0.91	1.56
10	15,400	0.40	0.82	1.27	2.12
20	30,800	0.82	1.93	2.77	3.51
40	61,600	1.72	3.93	5.85	14.05
60	92,400	2.76	5.65	9.57	21.33
80	123,200	3.56	7.62	12.00	50.67
Total	346,500	—	—	—	—

Per-cycle scoring latency for N independent detector contexts against the 1 Hz (1000 ms) processing budget, five repetitions per level. “Updates” is the total scorer invocations including warm-up; the latency percentiles and “Max” are computed over the measured scoring cycles only (warm-up excluded).

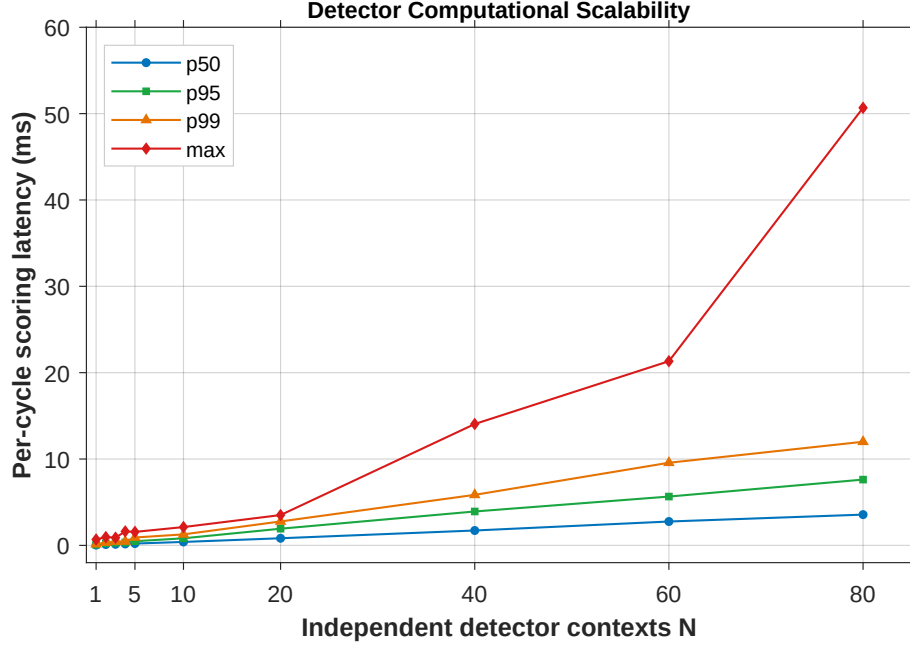


Figure 4.29: Detector per-cycle scoring latency (p50, p95, p99 and worst case) against the number of independent detector contexts.

Statistics are over the measured scoring cycles (1,500 per N , warm-up excluded); at $N=80$ the p50, p95 and p99 are 3.56, 7.62 and 12.0 ms and the worst single cycle is 50.7 ms, all far below the 1000 ms budget.

4.8.3 Policy Decision-Plane Throughput and Saturation

The trust-evaluation path was measured under open-loop offered load. Requests were issued on a fixed schedule from a low rate up to 80 evaluations per second, five repetitions per rate, distributed across R distinct synthetic protected-workload identifiers at each offered rate R . Every request traverses the real policy path: request parsing, evidence handling, the state machine, and the state update. The experiment does not drive live telemetry acquisition, Falco, or containment, so it measures policy decision-plane throughput rather than xApp capacity.

Two latency quantities are recorded per request. The request-response service latency is the interval from request submission to response receipt. End-to-end latency is measured from each request’s scheduled arrival time to its response, so it also captures any client-side queuing that open-loop load produces once the server falls behind. The deadline-miss criterion uses the end-to-end quantity against a 1 s bound, which avoids the coordinated-omission bias that a purely server-side measure would introduce.

The path held a zero deadline-miss rate through 20 evaluations per second and

remained close to it at 25 (1.96%). Beyond that it saturated quickly. The end-to-end miss rate reached 33% at 30 evaluations per second, 73% at 35, and 98% at 80. No HTTP errors occurred and the evaluator process did not crash at any rate. Two operating criteria frame this result. Strict capacity is the highest offered rate at which no request returned an error or timeout, no decision was incorrect, and no request exceeded the 1000 ms end-to-end budget. Operational capacity is the highest offered rate that still completed at least 99% of requests with under 1% budget misses, no incorrect decisions, and no evaluator restart. Under the evaluator’s configured one-core CPU allocation both criteria are met at 20 evaluations per second, with an empirical saturation knee near 25 to 30; evaluator CPU rose from about 0.6 of a core at 20 evaluations per second to its one-core ceiling by 35, where it stayed. Figure 4.30 shows the deadline-miss curve as the primary SLO view. The supporting panels in Figure 4.31 reinforce the same knee from two angles: the achieved completion rate tracks the offered rate up to about 25 per second and then stops following the ideal $y=x$ line, plateauing near 30 per second, while service latency climbs steeply through the same region. The three views describe one saturation result, not three independent effects.

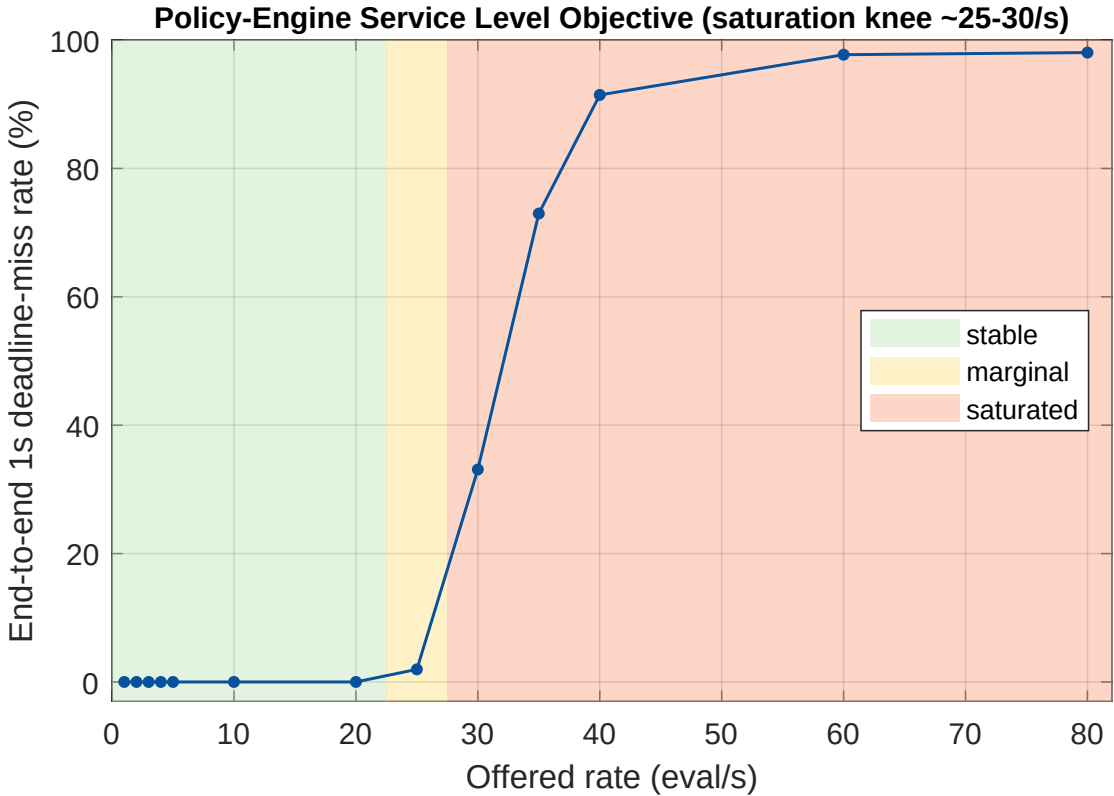


Figure 4.30: Policy decision-plane end-to-end deadline-miss rate against offered evaluation rate.

The path is stable through about 20 evaluations per second and saturates sharply beyond a knee near 25 to 30 under the evaluator’s configured one-core CPU allocation

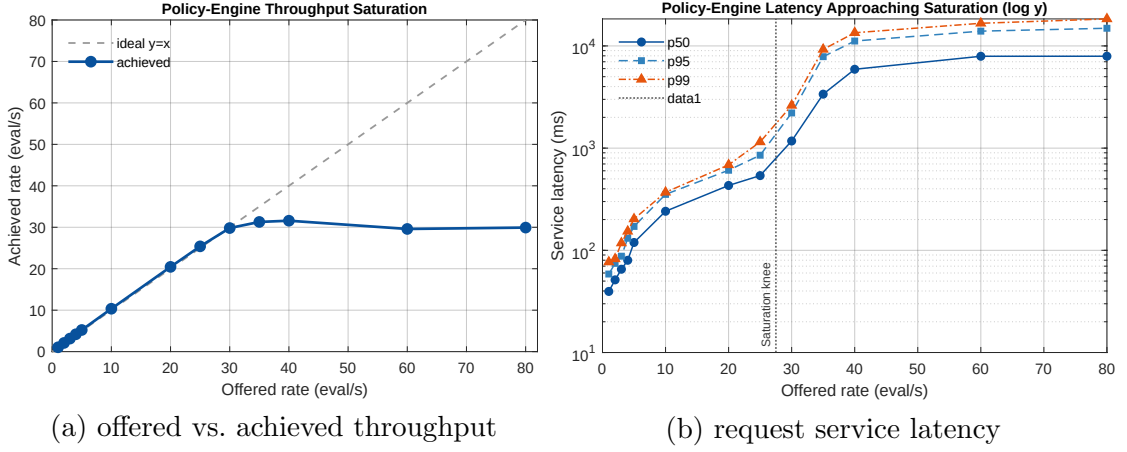


Figure 4.31: Supporting evidence for the decision-plane saturation knee.

In (a) the achieved completion rate departs from the ideal $y=x$ line and plateaus near 30 per second; in (b) service latency (submission to response) rises steeply through the same 25 to 30 per second region.

4.8.4 Concurrent Enforcement on Deployed xApps

The decisive scalability question for containment is whether the real enforcement path still isolates several compromised xApps at once. In each trial a barrier-synchronized controlled in-container attack action was injected into K of the four deployed xApp workloads simultaneously, for K from 1 to 4, with the target set rotated across trials so that no result is tied to one xApp. The action is an unexpected in-container shell execution representative of a compromised workload, not the exploitation of K independent software vulnerabilities, and it drives the full pipeline: the Falco and Falcotick behavioural path, the evaluator and responder, followed by SPIRE identity-binding withdrawal, Kubernetes and Calico quarantine-policy selection, Service isolation, node packet filtering, and active reachability verification. Isolation is confirmed by an active TCP-connect probe against the target pod. The formal dataset consists of 100 trials per K , for 400 valid trials. Launch skew stayed below 2 ms in every formal trial, and no trial was rejected for an invalid precondition.

Containment behaviour is reported through enforcement milestones and the final probe confirmed isolation. The identity milestone is an identity-binding withdrawal that removes renewal eligibility; it does not instantly revoke an already issued SPIFFE SVID. The NetworkPolicy milestone is the point at which the target, once labelled for quarantine, is selected by the pre-existing `zt-xguard-quarantine-deny-all` policy; it marks policy selection rather than confirmed packet-level enforcement, which the reachability probe verifies independently. The policy-decision timestamp itself is not reported for this experiment. The COMPROMISED decision time was not captured for any of the 60 formal targeted workload instances, and the ISOLATED update time is not a valid substitute for it, so no attack-onset-to-decision latency is claimed here.

Table 4.11 gives the median completion time of each enforcement milestone by concurrency level, and Figure 4.32 plots the per-stage times against K with the

individual trials shown. The containment makespan, measured from attack onset to probe-verified unreachable, increased monotonically with concurrency, from a 5.25 s mean at $K=1$ to 11.55 s at $K=4$.

Table 4.11: Mean enforcement-milestone completion time (seconds, attack-onset origin) by concurrency level, over the four deployed xApps. 100 formal trials per K .

K	SVID Revoke	Calico CNI Policy	Service Isolation	IP Table Block	Network Unreachability
1	3.20	1.05	3.15	3.10	5.25
2	4.48	1.47	4.41	4.34	7.35
3	5.76	1.89	5.67	5.58	9.45
4	7.04	2.31	6.93	6.82	11.55

Figure 4.32 displays the per-level mean for each enforcement milestone and the final probe-verified-unreachable time, with error bars indicating the standard deviation across trials.

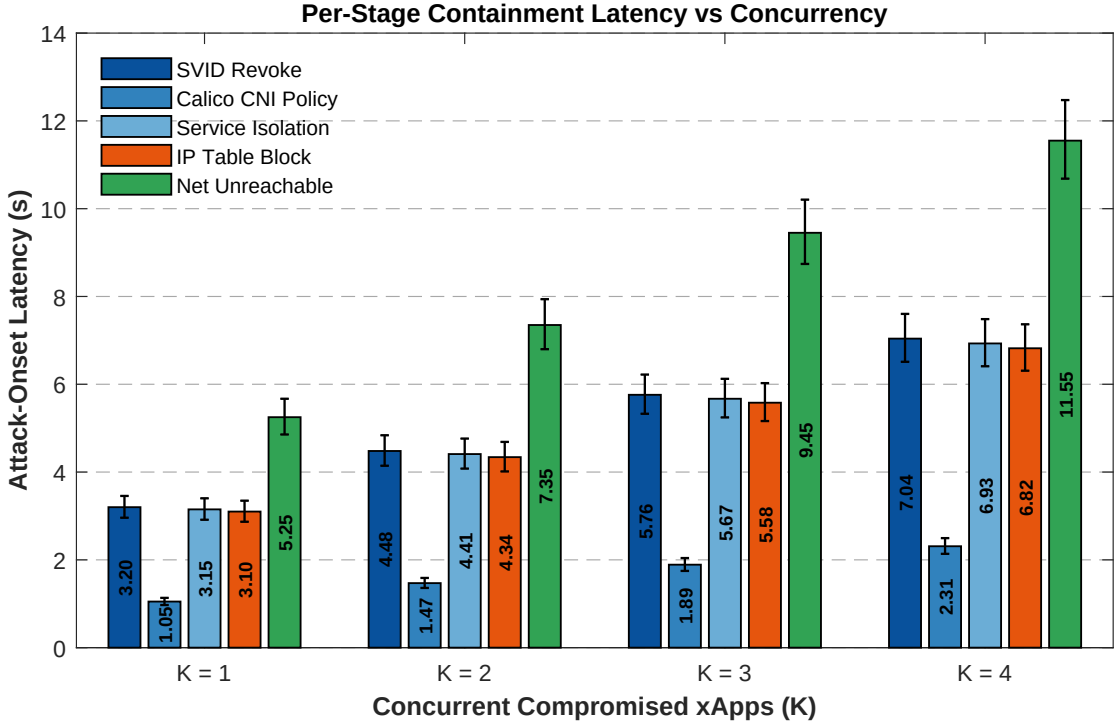


Figure 4.32: Per-stage containment latency against the number K of deployed xApp workloads subjected to simultaneous controlled attack injection.

4.8.5 Scalability Summary and Validity Boundaries

The three experiments show different scaling behaviour at each layer of ZT-XGuard. The statistical detector kept substantial processing headroom through 80 independent logical contexts. No measured scoring cycle missed the 1000 ms

processing budget; 346,500 scorer invocations were executed in total, including warm-up. The policy decision plane showed the first measurable computational saturation: the operational criterion held through 20 evaluations per second, with a saturation knee near 25 to 30. The real enforcement path isolated the targets in all formal trials, while the containment makespan grew with the number of simultaneous compromises. Taken together, these measurements characterize the scalability of ZT-XGuard’s security functions, not the maximum xApp-hosting capacity of the underlying Near-RT RIC.

The boundaries of that claim should be read precisely. Experiment 1 exercises 80 logical detector contexts, not 80 physically deployed xApps. Experiment 2 offers up to 80 evaluations per second at the top of the sweep, which is a request rate and not a count of xApps. Experiment 3 uses four actually deployed workload targets, not four independent E2 control loops. All three run on a single node. The results establish a tested single-node operating envelope for the security functions. They do not, on their own, establish production scalability for every Near-RT RIC deployment, and no claim of superiority over commercial RICs or other scalability research is drawn from them. Multi-node placement, fleets of fully E2-active xApps, and a queue-backed or horizontally scaled evaluator ingestion path are left as future work.

4.9 Runtime Overhead of the Policy Engine

The detection and containment logic contributed by this project runs in the ZT-XGuard policy engine, so its runtime cost on the Near-RT RIC was measured against a vanilla RIC hosting the same four xApps. Per-pod CPU and memory were sampled at 1 Hz from the node’s cgroup v2 accounting over a fifteen-minute steady-state window with no attack active. As shown in Figure 4.33, the additional overhead is small. For CPU usage, the two traces are visually indistinguishable: the policy engine increases the node mean only from 0.94% to 1.01%, corresponding to approximately 0.004 of a core, while the shared spikes originate from the RIC platform rather than the security layer. Memory usage increases by a fixed 98 MB, from 1398 MB to 1496 MB, and remains stable throughout the measurement window. Since the policy engine evaluates the detector once per second and operates outside the E2 near-real-time control path, this measured overhead remains small relative to the resources of the node hosting the RIC and its xApps.

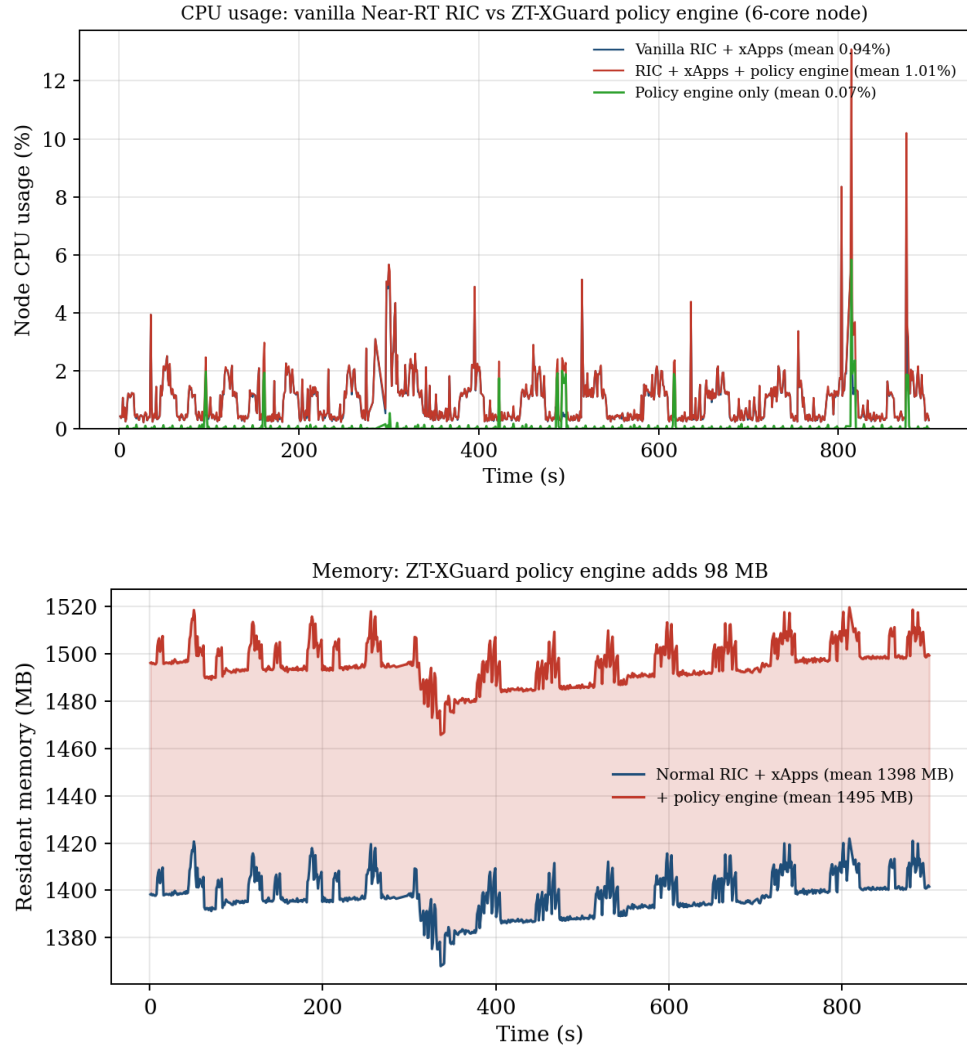


Figure 4.33: ZT-XGuard policy-engine overhead on the Near-RT RIC during a 15-minute steady-state measurement at 1 Hz. The upper plot compares CPU usage for the vanilla RIC and the ZT-XGuard-enabled RIC, while the lower plot compares resident-memory usage. The CPU overhead remains small, whereas the policy engine introduces an approximately 98 MB fixed resident-memory increase.

Chapter 5

Discussion

This chapter interprets the experimental results presented in Chapter 4 within the context of O-RAN security requirements and the zero-trust design principles that guide the framework. The discussion is organized around the four main areas of the implementation: the radio access network, the Near-RT RIC platform, the secure onboarding pipeline and the runtime monitoring layer.

5.1 Deployment of srsRAN and the 5G Core

The successful deployment of a fully containerized end-to-end 5G network validates that a software-defined RAN testbed can be built entirely from open-source components, providing a realistic environment for security experimentation. The use of Open5GS, srsRAN CU/DU and srsUE over a ZeroMQ virtual radio eliminated the need for dedicated RF hardware while preserving the protocol interactions that a malicious xApp would exploit. The measured throughput spike during the simulated DoS attack confirmed that the radio interface generated real telemetry, which is essential for testing the detection logic of the xApps. This setup constitutes a viable and reproducible platform for evaluating threat-containment mechanisms which are under controlled.

5.2 Near-RT RIC Deployment

The evaluation of two Near-RT RIC deployment models, Docker Compose and Kubernetes, revealed a clear trade-off between operational simplicity and security capability. The Docker-based RIC was straightforward to deploy and adequate for functional testing of xApps, but it lacked the fine-grained orchestration primitives required for workload identity attestation and network-level isolation. Migrating to a Kubernetes-based deployment was a necessary step to integrate SPIRE, cert-manager, the CSI driver, and the custom isolation policies that form the core of the framework. This underscores that security cannot be enforced onto a lightweight orchestration platform; the platform itself must offer APIs for identity, attestation and policy enforcement. The Kubernetes-based RIC, while more complex to operate, proved to be the correct architectural choice for a zero-trust Near-RT RIC.

5.3 Impact and Analysis of Remote Attestation

The integration of remote attestation using Keylime provided a verifiable layer of security to the O-RAN deployment. By utilizing TPM technology simulated via vTPM in virtual machines the system was able to check the integrity of monitored nodes periodically.

Technical logs confirmed that PCR values were collected successfully and attestation checks were performed at the configured intervals. The architecture, consisting of the agent, verifier, registrar, dashboard and tenant modules, was validated through successful communication and node registration.

This mechanism is important for Open RAN environments, where components like the Near-RT RIC and edge nodes may be distributed across diverse platforms. The ability to detect unauthorized changes to the platform’s measured boot and load-time integrity strengthens the overall trustworthiness of the deployment, within the scope noted in the results: it catches deviations in the measured boot chain, not runtime-only compromises that leave those measurements unchanged. Furthermore, the Keylime framework includes features for automated revocation of payload file which can be utilized to isolate nodes if an integrity failure is detected.

5.4 Secure Onboarding and Workload Attestation

The three-VM onboarding testbed supported an iterative implementation process. The initial selector-based, init-container run documented in Section 4.4 established functional workload attestation after the cryptographic gates. The later JIT Pod-UID and sidecar implementation hardened that same onboarding workflow and supplied the pipeline evaluated in the 200 latency trials. Neither onboarding iteration was connected to the independently deployed runtime containment testbed.

Evolution in the Onboarding Testbed

The final onboarding implementation, reflected in the `secure-ingestion.sh` and `registration.sh` scripts, diverged from the initial onboarding design in four important ways:

- **Manual Helm chart generation.** The `dms_cli` tool proved unreliable in the lab environment; therefore the pipeline now constructs the Helm chart directly from the xApp descriptor, guaranteeing that a valid chart is always produced and uploaded to ChartMuseum.
- **Validated selector-based intermediate design.** Early experiments replaced an invalid colon-chained selector with three independent selectors (namespace, service account and pod label). This produced the successful initial attestation run reported in the Results chapter and established the functional baseline for the later JIT design.
- **Sidecar-based credential delivery and JIT attestation.** Within the

onboarding testbed, the initial init container and label-based selectors were replaced by a dynamically injected `spiffe-helper` sidecar and a JIT entry bound to the active Kubernetes Pod UID. The sidecar renews the short-lived SVID in the background, avoiding a one-time credential fetch and binding the identity to one concrete pod instance without requiring changes to the main xApp code.

- **Man-in-the-middle (MiM) simulation.** A tampering test, in which the descriptor archive was modified while retaining the original GPG signature, was performed to validate Gate 1. The pipeline correctly rejected the payload, confirming that the signature verification is effective and that any unauthorized modification is detected before the image is even inspected.

Interpretation of the Results

The successful execution of the final onboarding pipeline demonstrated that the combination of GPG and Cosign provides a robust dual-signature supply-chain verification process that can operate in an air-gapped environment. In the final onboarding iteration, SPIRE reliably issued short-lived X.509 SVIDs with the expected SPIFFE ID and validity period, as evidenced by the SPIRE and sidecar logs. The JIT Pod-UID selector binds the issued identity to the active pod instance, and removing that JIT registration entry prevents subsequent renewal. The result therefore validates the onboarding testbed’s package verification to identity issuance path.

Relationship to Runtime Identity and Containment

The runtime monitoring and containment testbed instantiated workload identity independently. It used a controller-managed `ClusterSPIFFEID` selected by the `svid-enabled` label and a sidecar-managed SVID. Removing that label during containment causes the SPIRE controller to withdraw the rendered registration entry, preventing renewal after the current SVID expires. This second experiment validates identity binding withdrawal and renewal prevention as one containment lock; it does not reuse or revoke the SVID issued by the onboarding testbed.

5.5 Runtime Behavioral Monitoring

The runtime monitoring layer demonstrated that Falco can detect container-level security events, such as unauthorized access to sensitive files, within seconds of their occurrence. The custom rules tailored to the `ricplt` and `ricxapp` namespaces worked as intended, producing real-time security alerts visible on the Falco UI dashboard. This capability directly addresses the O-RAN WG11 threat landscape regarding compromised xApps attempting lateral movement, privilege escalation or credential exfiltration at the system call level.

5.6 Statistical Resource-Anomaly Detection

The evaluation results reported in Section 4.6 address the gap identified in Section 2.3 directly: no prior published Near-RT RIC security framework had empirically characterized a per-xApp, multivariate statistical detector’s true and false positive performance or the pulse-duration boundary produced by its configured persistence and rate gates. The observed clean runs and one benign elevated-load control (D1–D3) results are matters operationally: a detector with a non-trivial false-positive rate against legitimate bursty xApp behavior, such as the KPM-processing spikes in D3, would be unsuitable for automatic containment because each false escalation could interrupt a legitimate control loop. The outcome is consistent with the intended effects of the directional-excess transform and conservative CPU-floor calibration. The nominal Wilks (1941) [47] statement targets at least 99.9% population coverage with at least 95% confidence under the calibration distribution; it does not guarantee a zero future false-positive rate under workload shift, and temporal correlation limits a literal independent-sample interpretation. The held-out runs instead provide direct empirical evidence that the complete frozen detector produced states in those three separately collected evaluations.

Across the three sustained attack scenarios, the reported sample-level TPR was above 95%. These results have different inferential roles: A1 and A3 are development results because they informed selection of the CPU rate-gate window and threshold, whereas A2 is the independently held-out sustained attack evaluation. A2 achieved the highest TPR (99.48%) and a shorter detection latency than A1 despite using a lower CPU stress setting while also introducing memory pressure. This supports the intended multivariate advantage: the Ledoit-Wolf precision matrix [42] encodes how CPU, memory growth and network throughput co-vary, allowing the Hotelling T^2 statistic [20] to combine coordinated deviations that separate per-metric thresholds would evaluate independently. Because A1 and A2 differ in both attack composition and duration, this comparison supports, but does not by itself isolate or prove, the causal contribution of multivariate fusion. The gate ablation in Section 4.6.5 provides complementary component level evidence.

The low-and-slow staircase scenario (A3) revised the project’s working assumption that its $0.7\times$ -floor mean-intensity step would evade the rate gate. Step 1 was classified SUSPICIOUS for 95.6% of its samples because the gate counts individual one-second observations above the floor rather than comparing a windowed mean. The tested bursty stress workload therefore crossed the floor often enough to satisfy the selected gate rate. This is useful development evidence that the rate formulation resists the specific “stay below the mean floor” strategy exercised by A3. It does not prove resistance to every adaptive duty cycle: an adversary controlling its per-sample exceedance rate and T^2 trajectory may remain below one or both confirmation gates.

The A4 pulse-train result is narrower but still useful. Because the CPU-rate comparison is strict, a full 60-sample window needs ten, not nine, above-floor observations; this is a necessary Gate B count rather than a universal duration

threshold. In the single A4 collection, the 5s pulse was not detected and the 10s, 20s, 40s and 60s pulses were detected with observed onset latencies of 10–11 s. The experiment therefore brackets the tested transition between 5s and 10s under the frozen waveform and sampling alignment. It does not estimate detection probability at each duration or prove that every 10s attack is detected. The experiment measures the service impact of the undetected 5s pulse. That case is reported as a bounded resource-channel evasion and an explicit sensitivity specificity trade-off, while the independent behavioral channel retains coverage when a short attack also generates a high-confidence syscall violation.

These results should be read alongside three limitations. First, the model was fitted and calibrated against a single xApp (the KPI Monitoring xApp); the claim that per-xApp calibration is necessary (Section 4.6) has not itself been validated by fitting and evaluating the same procedure against a second, behaviourally distinct xApp, so generalization across xApp types is inferred from the design rather than demonstrated. Second, the evaluated resource scenarios (A1, A2, A3 and A4) generate pressure through external stress-generation tooling rather than a genuinely compromised xApp codebase adapting its behavior in response to detection. An adaptive adversary aware of the gate thresholds was not evaluated. A4 also contains only one pulse at each tested duration, so its 5s-to-10s transition is a boundary observation, not a repeated estimate of detection probability. Third, the bounded recovery-phase decay tail observed after attack cessation (up to 46 samples, 3.85%, in A2) reflects the MEWMA state’s intentional exponential memory and means a small number of post-attack samples remain classified as SUSPICIOUS after the underlying resource pressure has genuinely ended; this is an accepted consequence of the same smoothing mechanism that gives the detector its sensitivity to sustained low-intensity attacks, but it is a design trade-off rather than a cost-free property of the model.

5.7 Automated Multi-Lock Containment

The containment results reported in Section 4.7 show that either independent detection channel can establish COMPROMISED when its own confirmation guard is satisfied. Critical behavioral compromise initiates immediate isolation, whereas statistically confirmed compromise introduces a 30-second operator intervention dwell. During this interval, the operator may isolate the xApp early; otherwise, isolation begins automatically when the dwell expires. Both routes apply the same four independent locks and confirm with an active probe that the xApp is unreachable before the incident is recorded as contained.

Measured from attack onset, the deny-all network lock was applied in a mean of 2.3 s and application of all four containment locks completed in a mean of 4.73 s across the 400 behavioral trials. The identity figure measures withdrawal of the controller-managed binding; the current five-minute SVID remains valid until expiry, while the network and endpoint locks provide immediate reachability isolation. The statistical route additionally includes its attack-onset-to-detection time and the deliberate 30-second dwell before isolation. These latencies bound the window in which a compromised xApp can act, but they do not eliminate it.

Because the Near-RT RIC control loop operates within 10 ms to 1 s, even a few seconds can include multiple control iterations. The suspicious-tier CPU throttle limits resource consumption during the statistical route’s dwell but does not prove that the xApp cannot affect the control loop.

The orchestrator’s CPU usage remains below 8 mCPU outside the brief containment spike. These results demonstrate a verified detection-to-isolation loop: ZT-XGuard does not merely generate an alert; it applies independent containment controls, confirms unreachability and extracts forensic evidence for subsequent investigation.

5.8 Challenges and Solutions

Building and securing a working Near-RT RIC on a real radio testbed surfaced a number of problems that were not visible in design. This section key issues recorded in the implementation and how each was resolved, so that a later project reproducing this setup can avoid the same dead ends.

5.8.1 Getting Real KPM Telemetry across the E2 Interface

The first hard problem was simply obtaining live Key Performance Measurement (KPM) data from a real gNB through the Near-RT RIC, rather than replaying recorded traces. The testbed pairs a srsRAN OCUDU gNB with the OSC Near-RT RIC (platform image v6.0.8) and the `kpimon-go` monitoring xApp. At the transport level everything looked healthy: the gNB opened its SCTP association, `e2mgr` logged the node as `CONNECTED`, and E2 Setup completed. The xApp itself, however, entered a `CrashLoopBackOff` and aborted with a `SIGABRT` during ASN.1 decoding, reporting `munmap_chunk(): invalid pointer` and a null `measID` while building the subscription. We reported this as a reproducible defect on the official NIST O-RAN-Testbed-Automation project, where it is publicly tracked as issue #13 [58] (<https://github.com/usnistgov/O-RAN-Testbed-Automation/issues/13>).

The root cause had two layers. First, a genuine memory-management bug in OSC’s `kpimon-go`: its `wrapper.c` allocated a buffer outside a loop and then double-freed it. The testbed maintainers confirmed our diagnosis, patched the double-free, and merged the fix upstream in a commit that references the report (commit `fda2d67`, “Update automation tool (#13)”). Second, and more fundamental, a protocol-version mismatch: the OCUDU gNB emits E2AP v03.00 and E2SM-KPM v05.00, while `kpimon-go` was built against the legacy E2SM-KPM v2.0 data structures, so its C bindings could not decode the modern payload geometry.

Even after the crash was fixed the xApp still received no indications, because its F1AP helper decoded the F1 Setup message but returned empty Cell and PLMN identifiers, so the subscription’s `ActionToBeSetupList` was empty and the request was rejected. We resolved this ourselves by patching `kpimon-go` to fall back to the PLMN and NR Cell identity carried in the decoded F1AP message (PLMN 00f110, NR Cell 00066c0000) when the helper returned blanks. With

that fallback in place the xApp built a valid Format 1 cell-level action, the RIC accepted the subscription (`Successfully subscription done`), and `kpimon-go` began receiving live `RIC_INDICATION` messages and decoding real cell metrics such as `RRU.PrbAvailDl`, `RRU.PrbUsedUl` and `DRB.UEthpDl`, which it wrote to InfluxDB while a UE passed real traffic through the network. The maintainers confirmed in the s issue thread that `kpimon-go` now interoperates with the OCUDU gNB, and the issue was closed as resolved. Beyond unblocking our own work, this contribution gave the project a genuine, end-to-end KPM feed instead of synthetic input, and fed a fix back into the public testbed that other O-RAN researchers depend on.

5.8.2 Separating Attacks from Legitimate RIC Control Traffic

Once the behavioural monitoring layer was switched on, it repeatedly quarantined perfectly healthy xApps. The cause was the external-egress rule firing on `kpimon-go`'s own normal RMR traffic to RIC platform services such as `e2term`, `appmgr`, `almediator`, `submgr` and the SDL Redis store. RMR connects by raw IP, and at the moment Falco observes the `connect` the reverse-DNS name is not yet resolved, so `fd.sip.name` was empty and the destination looked external. The problem was made worse by two practical details: the `submgr` and `DBAAS` services are headless, so they have no stable `ClusterIP` and their backing-pod addresses drift on every restart, and a cluster-wide pod-churn event once invalidated the enumerated addresses and triggered a wave of false quarantines. We also found the hard way that Falco's `fd.sip.name` supports only exact match and that CIDR ranges are not valid inside `fd.sip in (...)`, so a simple subnet rule was not an option.

The solution was to make the notion of “known internal destination” explicit and self-maintaining. We enumerated each RIC platform service both by DNS name and by stable `ClusterIP`, added CoreDNS and loopback as recognized internal targets, and, for the services whose addresses change, added a background reconciler (`ztx_falco_rule_sync.py`) that updates the pod-IP entries whenever they drift. Only destinations outside this maintained set are now treated as external egress. A related fix corrected `kpimon-go`'s `DBAAS_SERVICE_HOST` environment variable, which was missing the `.svc.cluster.local` suffix and had been causing slow DNS lookups and intermittent liveness-probe timeouts.

5.9 O-RAN WG11 Threat Coverage

The O-RAN Alliance Security Working Group (WG11) defines the threat model this project targets, so it is worth showing directly how ZT-XGuard covers each of its main xApp threat categories. Table 5.1 maps the WG11 threats to the detection channel that observes them and the response that contains them. The framework covers the runtime and resource-abuse categories through its two detection channels and four-lock containment, and it covers the platform-integrity category through measured-boot attestation within the scope stated in the following section.

Table 5.1: Coverage of O-RAN WG11 xApp threat categories by ZT-XGuard

WG11 threat category	Detection	Containment response
Malicious or compromised xApp (shell, credential and token theft, SVID abuse)	Behavioral (critical Falco rules)	Identity revocation and four-lock isolation
Resource exhaustion / denial of service against co-located components	Statistical (MEWMA/ T^2 dual gate)	CPU throttle, then four-lock isolation
Breach of container isolation / container escape	Behavioral (runtime-socket access rule)	Immediate isolation before host compromise completes
Lateral movement and unauthorized cross-component access	Behavioral (external egress rule)	Deny-all NetworkPolicy and node-level block
Data exfiltration and information disclosure	Behavioral (external egress)	Network locks and identity-binding withdrawal
Configuration and integrity tampering	Behavioral (config-tamper rule) and measured-boot attestation	Probe-verified four-lock isolation and forensic evidence extraction
Compromised infrastructure node	Keylime measured-boot attestation	Node marked untrusted, encrypted payloads withheld

5.10 Limitations

The results in this report hold within a scope that is worth stating plainly, both to bound what has been demonstrated and to define the future work that follows from it.

Detection is characterized on a curated corpus. The behavioral rules are authored per xApp and per attack class, so the 400-trial result shows that curated exploitation evidence is detected and contained, not that the rules generalize to novel or obfuscated techniques. A behavioral false-positive rate on long-running benign xApp operation was not separately measured.

The statistical false-positive result is measured on one xApp. The observed sample-level result of 0 false-positive states in 4,397 samples is established on the KPI monitoring xApp’s held-out clean and elevated-load telemetry. It characterizes that workload rather than the full space of legitimate behavior across

xApps with different resource profiles. Per-xApp calibration is a deliberate design choice, but automatically learning and validating baselines for a large fleet is left to future work.

The pulse-duration boundary is sparsely sampled. A4 contains one pulse at each of five durations. Its 5s-to-10s transition is an observed boundary for that waveform and sampling alignment, not a repeated estimate of detection probability or a universal minimum detectable duration.

Containment limits, but does not eliminate, the exposure window. The Near-RT RIC control loop closes in tens of milliseconds to one second, so even the achieved network-cutoff latency of a few seconds leaves room for a compromised xApp to influence several control-loop iterations before quarantine. The suspicious-tier CPU throttle bounds resource abuse during this window but does not close it for the resource channel.

The two halves were validated on separate testbeds. Secure onboarding and platform attestation were evaluated on the three-VM testbed, whereas runtime detection and containment were evaluated on the single-node NIST testbed. The identity mechanisms were instantiated independently: the final onboarding pipeline used JIT Pod-UID registration, while the runtime testbed used a label-selected `ClusterSPIFFEID` as its revocation anchor. The onboarding-to-runtime pipeline was therefore validated in complementary parts rather than exercised end to end with one shared SVID lifecycle.

Attestation is scoped to measured boot. The Keyline result demonstrates detection of a deviation in the platform’s measured boot and load-time integrity. It does not detect runtime-only compromises that leave the measured boot chain unchanged, such as in-memory-only exploits or userspace rootkits loaded after the measured window.

Forensic evidence extraction is not immutable storage. Evidence is captured to a host path the compromised process cannot reach, but the current implementation does not provide append-only or WORM storage, per-artifact cryptographic hashing and signing or off-node retention.

Scale is bounded to a single node. The scalability evaluation in Section 4.8 establishes a tested single-node envelope for ZT-XGuard’s security functions rather than the xApp-hosting capacity of the Near-RT RIC. The statistical detector retained substantial processing headroom through 80 independent logical detector contexts with no measured processing-budget misses. The policy decision plane satisfied both predefined capacity criteria through 20 evaluations per second and exhibited an empirical saturation knee at approximately 25 to 30 evaluations per second. Concurrent enforcement was evaluated across four deployed xApp workloads for $K=1$ to 4, yielding successful complete trials out of 24 formal trials; only `kpimon-go` was E2/KPM-active. These bounds do not establish multi-node or production-wide scalability. Multi-node placement, fleets of fully E2-active xApps, and horizontally scaled or queue-backed policy ingestion remain future work.

5.11 Summary of Discussion

The experimental results confirm that the proposed zero-trust framework provides effective defense in depth for the Near-RT RIC. The srsRAN testbed offers a realistic radio environment, the Kubernetes-based RIC provides the necessary orchestration primitives, the secure onboarding pipeline reliably rejects tampered payloads and issues verifiable workload identities and the runtime monitoring layer detects both security and performance anomalies. The onboarding evaluation documents both the initial selector-based, init-container validation and the final JIT Pod-UID, sidecar-managed pipeline. Independently, the runtime testbed demonstrates `ClusterSPIFFEID`-based identity-binding withdrawal as one containment lock. Together these results demonstrate complementary identity issuance and binding withdrawal without implying a shared cross-testbed SVID lifecycle. The iterative implementation also demonstrates the practical challenges of integrating security tools into an evolving open-source ecosystem while preserving resilience and offline operation. Across the development and held-out attack datasets, the statistical resource-anomaly detector reports true-positive rates above 95%. The multi-lock containment mechanism closes the loop from detection to verified isolation. Critical behavioral compromise initiates immediate isolation, while statistical compromise permits optional early operator intervention before automatic isolation at the end of the 30-second dwell. The remaining required manual step in the current implementation is narrower than initially scoped: it is the SMO to RIC deployment trigger within the secure onboarding pipeline, not the containment response, which the results in this chapter confirm operates fully automatically.

Chapter 6

Timeline and Resources Required

6.1 Timeline

The project followed a phased plan and the phases below describe the work as it was carried out. It began with a detailed literature review of xApp security in Open RAN, the principles of zero-trust architecture, remote attestation, workload identity and container isolation, which fixed the project scope and the threat model. The security architecture was then designed as four coordinated layers: hardware-rooted platform attestation, secure onboarding with workload identity, dual-channel runtime detection and automated containment.

Hardware-rooted attestation was implemented first, using Keylime with a virtual TPM to verify each node’s measured boot integrity before any xApp was scheduled. The secure onboarding pipeline followed, spanning the solution provider, the SMO and the Near-RT RIC, with Cosign image signing, GPG descriptor signing, a four-gate SMO ingestion pipeline and SPIFFE/SPIRE workload attestation that binds a short-lived SVID to each xApp.

The largest single effort was the statistical resource-anomaly detector. This covered feature selection, robust median and MAD centering, the directional-excess transform, sliding-window smoothing, MEWMA tracking, Ledoit-Wolf shrinkage covariance and Hotelling’s T^2 fusion, followed by nonparametric calibration of the control limits and the dual-gate decision logic on clean baseline data, producing a frozen model. In parallel, the behavioral channel was built on Falco with per-xApp rules generated from declarative profiles and both channels were fused by the policy engine into the three-state trust model.

Automated multi-lock containment and active verification were then implemented and wired to the policy engine. The final phase was testing and evaluation: 400 behavioral attack trials and 50 statistical attack trials on the live testbed, the offline model characterization across the clean and attack datasets, the evasion-boundary and ablation analyses and the compilation of the results into this report.

Table 6.1: Project Timeline

Task	2025–2026																																							
	Nov				Dec				Jan				Feb				Mar				Apr				May				Jun				Jul				Aug			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4				
Selection of the Project Topic																																								
Discussion with Supervisor and Co-Supervisors																																								
Study on Research Work (Papers, Journals, Conferences)																																								
Prepare Project Proposal and Presentation																																								
Study on Existing O-RAN Test Frameworks																																								
Initial Testbed Setup and Design																																								
Remote Attestation using TPM																																								
Design xApp Integrity and Workload Attestation																																								
Statistical Resource-Anomaly Detector Development and Calibration																																								
Behavioral Detection (Falco) and Dual-Channel Policy Engine																																								
Automated Multi-Lock Containment and Verification																																								
Integration Demonstration and Visualization																																								
Testing and Evaluation																																								
Final Report Submission and Presentation																																								

6.2 Resources Required

Hardware Resource Requirements

1. Host workstations capable of running multiple virtual machines and a Kubernetes cluster.
2. A three-VM testbed (Ubuntu) for secure onboarding, modelling the solution provider, the SMO and the Near-RT RIC as separate administrative domains.
3. A single-node NIST O-RAN testbed VM for runtime detection and containment, hosting the 5G core, the gNB and the Kubernetes-based Near-RT RIC.
4. Virtual TPM (vTPM) modules for hardware-rooted remote attestation, providing TPM 2.0 functions within the virtualized environment.
5. A software-simulated gNB and UE for E2 connectivity, so that the KPI monitoring xApp receives genuine KPM indications during the runtime evaluation.

Software Resource Requirements

1. Ubuntu Linux as the operating system across all nodes.
2. srsRAN Project (gNB), Open5GS (5G core) and the O-RAN Software Community Near-RT RIC, integrated through the NIST 5G open-source testbed automation platform, with Kubernetes (kubeadm) and the Calico CNI.
3. Keylime with a virtual TPM for remote attestation and Cosign, GPG, Helm and ChartMuseum for the secure onboarding pipeline.
4. SPIFFE/SPIRE and cert-manager for workload identity attestation and short-lived SVIDs.
5. Falco with eBPF and Falcosidekick for behavioral monitoring and cAdvisor with Prometheus for resource telemetry.
6. A Python and Flask policy engine and containment orchestrator and the frozen statistical model implemented with NumPy and scikit-learn (Ledoit-Wolf shrinkage).
7. stress-ng and Peirates for reproducible attack generation and MATLAB for figure generation.
8. A web-based Incident Investigation Center dashboard for reviewing forensic evidence and initiating early isolation during the statistical dwell.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The transition to Open RAN introduces a fundamental tension: the disaggregation and openness that enable innovation also expand the attack surface, particularly through third-party xApps running on the Near-RT RIC. A single compromised xApp can disrupt network control loops, leak sensitive data or degrade service for multiple users. This project set out to design and validate a security framework that mitigates that risk using zero-trust principles.

The framework was built around four complementary layers. Platform attestation, implemented with Keylime and virtual TPMs, ensures that the underlying infrastructure boots into a known-good state and remains untampered. A secure onboarding pipeline, driven by cryptographic verification of both the xApp descriptor and the container image, prevents tampered or unauthorized packages from being deployed. Workload identity attestation, using SPIFFE/SPIRE, issues short-lived X.509 certificates to xApp pods and binds those identities to specific Kubernetes attributes, enabling mutual TLS and fine-grained access control. The onboarding and runtime testbeds instantiated this identity layer independently for issuance and binding withdrawal respectively; they did not share one SVID or registration entry. Runtime protection then combines two independent detection channels with an automated response. A behavioral channel built on Falco and eBPF flags system-level indicators such as unexpected shells, credential access and service-account token theft, while a statistical channel tracks CPU, memory-growth and network telemetry through a robust MEWMA and Hotelling T^2 detector to catch resource-exhaustion attacks that leave no discrete syscall signature. A three-state policy engine fuses both channels at the decision level. Critical behavioral compromise initiates immediate isolation, whereas statistically confirmed compromise starts a 30-second operator intervention dwell before automatic isolation. Both paths use a four-lock containment mechanism and actively verify that the xApp is unreachable.

All layers were implemented and experimentally validated across two reproducible environments. The three-VM onboarding testbed rejected the tampered payload in the man-in-the-middle simulation and issued a Pod-UID-bound workload iden-

tity through the final JIT sidecar pipeline. The separate NIST runtime testbed exercised its own `ClusterSPIFFEID` binding as a revocable identity anchor during verified containment. These results validate complementary identity-issuance and identity-binding-withdrawal properties without claiming that the two testbeds shared one SVID lifecycle. The behavioral channel detected every attack scenario, with a mean detection latency of 196 ms and application of all four locks completing in a mean of 4.73 s across 400 live trials. The identity-lock time measures binding withdrawal rather than expiry of the already issued five-minute SVID; network unreachability is enforced and verified independently. Across the development and held-out attack datasets, the statistical channel produced true-positive rates from 95.11% to 99.48%. A1 (95.11%) and A3 (98.89%) are development results because they informed selection of the CPU rate-gate parameters. The independently held-out A2 scenario achieved a 99.48% true-positive rate, while A4 independently bracketed the observed transition between the tested 5s and 10s pulses under the frozen dual gate. The held-out benign D1–D3 datasets produced zero false positives across 4,397 scored samples. In deployed operation, the statistical path applied a reversible CPU throttle on partial evidence and reached `COMPROMISED` when the complete dual gate was satisfied. Both component deployments operated under air-gapped constraints, demonstrating offline operation of each evaluated half without claiming that one integrated end-to-end pipeline was exercised. This capability is frequently overlooked but is critical for real world O-RAN deployments.

7.2 Future Work

The current implementation has limitations that define the remaining work. The statistical detector was fitted and calibrated against a single xApp, the KPI Monitoring xApp, so the claim that per-xApp calibration is necessary has not yet been tested by fitting and evaluating a second workload, and validating this across a diverse xApp population is the natural next step. Because the framework reasons over system-level and resource-level evidence, attacks expressed entirely through an xApp’s legitimate RAN-control semantics, such as E2 data poisoning or malicious but well-formed control logic, fall outside its current scope and would need application-level models of correct control behavior. The one manual step that remains is narrower than the containment loop the response now closes automatically: it is the SMO-to-RIC deployment trigger in the onboarding pipeline. Extending the policy engine from its current state-based decisions toward dynamic, risk-based access control is a further direction. The project has demonstrated that a layered, zero-trust security architecture can be built from open-source components and tailored to the unique constraints of the Near-RT RIC, and it provides an experimentally grounded reference for operators and standards bodies seeking to raise the security posture of Open RAN without sacrificing the flexibility that makes the architecture valuable.

References

- [1] O-RAN Working Group 1, “O-RAN Architecture Description,” O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v15.00, Oct. 2025.
- [2] O-RAN Working Group 3, “Near-RT RIC Architecture,” O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v07.00, Feb. 2025.
- [3] M. Polese, L. Bonati, S. D’oro, S. Basagni, and T. Melodia, “Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges,” *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1376–1411, 2023.
- [4] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. Di Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta *et al.*, “A Comprehensive Tutorial and Survey of O-RAN: Exploring Slicing-aware Architecture, Deployment Options, Use Cases, and Challenges,” *IEEE Communications Surveys & Tutorials*, 2025.
- [5] O-RAN Alliance, “O-RAN Alliance Official Website,” <https://www.o-ran.org/>.
- [6] —, “O-ran architecture description,” O-RAN Alliance, Technical Specification O-RAN.WG1.TS.OAD-R004-v15.00, 2025.
- [7] O-RAN ALLIANCE, “The O-RAN ALLIANCE Security Working Group Continues to Advance O-RAN Security,” O-RAN ALLIANCE Security Working Group (WG11), Feb. 2024, accessed: Aug. 2026. [Online]. Available: <https://www.o-ran.org/blog/the-o-ran-alliance-security-working-group-continues-to-advance-o-ran-security>
- [8] C.-F. Hung, Y.-R. Chen, C.-H. Tseng, and S.-M. Cheng, “Security Threats to xApps Access Control and E2 Interface in O-RAN,” *IEEE Open Journal of the Communications Society*, vol. 5, pp. 1197–1203, 2024.
- [9] M. Liyanage, A. Braeken, S. Shahabuddin, and P. Ranaweera, “Open RAN Security: Challenges and Opportunities,” *Journal of Network and Computer Applications*, vol. 214, p. 103621, 2023.
- [10] P. Kuure, “Security and Trust in O-RAN,” Master’s thesis, South-Eastern Finland University of Applied Sciences, Finland, 2024.

- [11] O-RAN ALLIANCE, “O-RAN Security Requirements Specification,” O-RAN ALLIANCE, WG11 Security Work Group, Technical Specification O-RAN.WG11.Security-Requirements-Specification-v04.00, Oct. 2022, version 4.00.
- [12] O-RAN Working Group 11, “O-RAN Security Threat Modeling and Risk Assessment,” O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v07.00, Oct. 2025.
- [13] —, “Study on Security for Near Real Time RIC and xApps,” O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v06.00, Oct. 2025.
- [14] National Vulnerability Database (NVD), “CVE-2023-42358,” <https://nvd.nist.gov/vuln/detail/CVE-2023-42358>, 2023.
- [15] National Institute of Standards and Technology, “CVE-2023-41628 Detail,” National Vulnerability Database (NVD), 2023, CVE-2023-41628, published Sep. 1, 2023, accessed Aug. 8, 2026. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2023-41628>
- [16] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” National Institute of Standards and Technology, NIST Special Publication 800-207, 2020.
- [17] O-RAN Working Group 11, “Study on Zero Trust Architecture for O-RAN,” O-RAN ALLIANCE e.V., Tech. Rep. TR.0-R004-v04.00, Oct. 2025.
- [18] Cloud Native Computing Foundation, “The spiffe project: Secure production identity framework for everyone,” <https://spiffe.io>, 2024.
- [19] C. A. Lowry, W. H. Woodall, C. W. Champ, and S. E. Rigdon, “A multivariate exponentially weighted moving average control chart,” *Technometrics*, vol. 34, no. 1, pp. 46–53, 1992.
- [20] H. Hotelling, “Multivariate quality control, illustrated by the air testing of sample bombsights,” in *Techniques of Statistical Analysis*, C. Eisenhart, M. W. Hastay, and W. A. Wallis, Eds. New York: McGraw-Hill, 1947, pp. 111–184.
- [21] T. O. Atalay, S. Maitra, D. Stojadinovic, A. Stavrou, and H. Wang, “Securing 5G OpenRAN with a scalable authorization framework for xApps,” in *Proc. IEEE INFOCOM*, 2023.
- [22] P. Fernando, P. Porambage, M. Liyanage, K. Steenhaut, and A. Braeken, “Securing xApps in open RAN: A hierarchical approach to authentication and authorisation,” in *Proc. IEEE Conf. Communications and Network Security (CNS)*, 2025.
- [23] C.-F. Hung, C.-H. Tseng, and S.-M. Cheng, “Anomaly detection for mitigating xApp and E2 interface threats in O-RAN near-RT RIC,” *IEEE Open Journal of the Communications Society*, 2025.

- [24] A. S. Abdalla, J. Moore, N. Adhikari, and V. Marojevic, “ZTRAN: Prototyping zero trust security xApps for open radio access network deployments,” *IEEE Network*, 2024.
- [25] H. Jiang, H. Chang, S. Mukherjee, and J. E. Van der Merwe, “OZTrust: An O-RAN Zero-Trust Security System,” in *2023 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. IEEE, 2023, pp. 129–134.
- [26] D. Lin, S. Bhargav, A. Chiejina, M. I. Ibrahim, and V. K. Shah, “ZT-RIC: A zero trust RIC framework for ensuring data privacy and confidentiality in open RAN,” in *Proc. IEEE Consumer Communications and Networking Conference (CCNC)*, 2025.
- [27] M. H. Eiza, B. Akwirry, A. Raschella, M. Mackay, and M. K. Maheshwari, “A Hybrid Zero Trust Deployment Model for Securing O-RAN Architecture in 6G Networks,” *Future Internet*, vol. 17, no. 8, p. 372, Aug. 2025.
- [28] The Falco Project (CNCF), “Falco: Cloud native runtime security,” [Online]. Available: <https://falco.org>, accessed: Aug. 2026.
- [29] National Institute of Standards and Technology, “NIST 5G open-source testbed automation tool,” [Online]. Available: <https://www.nist.gov/services-resources/software/nist-5g-open-source-testbed-automation-tool>. Source code: <https://github.com/usnistgov/O-RAN-Testbed-Automation>, accessed: Aug. 2026.
- [30] “srsRAN Documentation,” <https://docs.srsran.com/projects/project/en/latest/index.html>, srsRAN Project.
- [31] Open5GS Project, “Open5gs: Open source implementation of 5G core and EPC,” [Online]. Available: <https://open5gs.org>, accessed: Aug. 2026.
- [32] O-RAN Software Community, “O-RAN SC RIC Platform Deployment Documentation,” <https://docs.o-ran-sc.org/projects/o-ran-sc-ric-plt-ric-dep/en/latest/>.
- [33] —, “O-RAN SC Official Website,” <https://o-ran-sc.org/>.
- [34] Sigstore Project, “Cosign: Container signing, verification and storage in an OCI registry,” [Online]. Available: <https://github.com/sigstore/cosign>, accessed: Aug. 2026.
- [35] “Keylime: Bootstrap & Maintain Trust on the Edge/Cloud and IoT,” <https://keylime.dev/>, Keylime Project.
- [36] Trusted Computing Group, “Trusted Computing Group Official Website,” <https://trustedcomputinggroup.org/>.
- [37] The Falcosidekick Project, “Falcosidekick: A simple daemon to connect falco to your ecosystem,” [Online]. Available: <https://github.com/falcosecurity/falcosidekick>, accessed: Aug. 2026.

- [38] Google, “cadvisor: Analyzes resource usage and performance of running containers,” [Online]. Available: <https://github.com/google/cadvisor>, accessed: Aug. 2026.
- [39] Prometheus Authors (CNCF), “Prometheus: Monitoring system and time series database,” [Online]. Available: <https://prometheus.io>, accessed: Aug. 2026.
- [40] The Kubernetes Authors (CNCF), “Kubernetes: Production-grade container orchestration,” [Online]. Available: <https://kubernetes.io>, accessed: Aug. 2026.
- [41] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [42] O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2003.
- [43] Tigera, “Project calico: Networking and network policy for kubernetes,” [Online]. Available: <https://www.tigera.io/project-calico/>, accessed: Aug. 2026.
- [44] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.
- [45] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.
- [46] P. J. Rousseeuw and C. Croux, “Alternatives to the median absolute deviation,” *Journal of the American Statistical Association*, vol. 88, no. 424, pp. 1273–1283, 1993.
- [47] S. S. Wilks, “Determination of sample sizes for setting tolerance limits,” *The Annals of Mathematical Statistics*, vol. 12, no. 1, pp. 91–96, 1941.
- [48] C. I. King, “stress-ng: A tool to load and stress a computer system,” [Online]. Available: <https://github.com/ColinIanKing/stress-ng>, accessed: Aug. 2026.
- [49] InGuardians, “Peirates: Kubernetes penetration testing tool,” [Online]. Available: <https://github.com/inguardians/peirates>, accessed: Aug. 2026.
- [50] The MITRE Corporation, “MITRE ATT&CK knowledge base,” [Online]. Available: <https://attack.mitre.org>, accessed: Aug. 2026.
- [51] Rakuten Symphony, “Rakuten mobile and rakuten symphony advance open RAN innovation with third-party rApp integration and nationwide RIC deployment,” Press release, Rakuten Symphony, Feb. 2026, accessed: Aug. 2026.

- [52] Samsung Electronics, “Samsung and TELUS collaborate on canada’s first AI-powered RAN intelligent controller to optimize network performance and energy efficiency,” Press release, Samsung Newsroom, Sep. 2025, accessed: Aug. 2026.
- [53] Nokia, “Nokia and AT&T leverage advanced intelligence of the open RAN compliant near-real-time RAN intelligent controller with native E2 in successful trial,” Press release, Nokia, Mar. 2023, accessed: Aug. 2026.
- [54] O-RAN ALLIANCE, “Near-real-time RAN intelligent controller and E2 interface workgroup (WG3),” O-RAN ALLIANCE working group webpage, <https://www.o-ran.org>, accessed: Aug. 2026.
- [55] O-RAN Software Community, “Traffic steering use case (xApp) user guide,” O-RAN Software Community documentation, accessed: Aug. 2026.
- [56] A. Sultana *et al.*, “Experimental evaluation of xApp conflict mitigation framework in O-RAN: Insights from testbed deployment in OTIC,” in *Proc. IEEE INFOCOM Workshops (INFOCOM WKSHPS)*, 2025.
- [57] S. Kumar, R. Zitouni, A. Akbas, and C. H. Foh, “FlexScale: Scalable and efficient management approach for near-RT RIC in O-RAN,” in *Proc. IEEE Wireless Communications and Networking Conference (WCNC)*, 2025.
- [58] NIST O-RAN Testbed Automation contributors, “ASN.1 decoding error (SIGABRT) in kpimon-go with srsRAN E2SM-KPM payload,” GitHub issue #13, [usnistgov/O-RAN-Testbed-Automation](https://github.com/usnistgov/O-RAN-Testbed-Automation), <https://github.com/usnistgov/O-RAN-Testbed-Automation/issues/13>, 2026, accessed: Aug. 2026.